UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

MANUAL DE EJERCICIOS PARA LA MATERIA LENGUAJES DE PROGRAMACIÓN

ACTIVIDAD DE APOYO A LA DOCENCIA

QUE PARA OBTENER EL TÍTULO DE:

LICENCIADO EN CIENCIAS DE LA COMPUTACIÓN

PRESENTA:

LUIS MIGUEL MUÑOZ BARÓN

TUTOR:

JAVIER ENRIQUEZ MENDOZA

CIUDAD UNIVERSITARIA, CDMX. 2026.

Nota al lector

El presente trabajo es una compilación de ejercicios teóricos para la materia de lenguajes de programación de la carrera en ciencias de la computación con clave de asignatura 1536, siguiendo el plan de estudios impartido desde el año 2013 en la Facultad de Ciencias de la Universidad Nacional Autónoma de México.

El objetivo principal de este material es que sirva como guía para desarrollar los contenidos visitados en este curso, siendo un apoyo didáctico para profesores y alumnos donde cada ejercicio planteado durante el capítulo esté acompañado de la base teórica para poder seguir el desarrollo del mismo y ser entendido en su totalidad.

Cada capítulo cuenta con dos secciones, una sección de teoría autocontenida con ejercicios resueltos y una sección de ejercicios sin respuesta para el lector al final del mismo.

Las demostraciones y desarrollo de los contenidos teóricos serán discutidos de forma laxa dejando a discreción del lector el estudio a profundidad de la información aquí presentada concentrándonos únicamente en la teoría necesaria para poder dar solución a los ejercicios, asimilar los conceptos y definiciones, familiarizarnos con la explicación, planteamiento y respuesta del material aquí presentado.

Para un estudio en profundidad se refiere a las personas interesadas a consultar [1], [2], [5], [12], [22] y [26] de la bibliografía que se encuentra al final del presente manual.

Finalmente, espero que las personas que hagan uso de este material encuentren útiles y fructífero los desarrollos y conceptos escritos en las páginas de este texto.

Miguel Barón.

"A language is not just words. It's a culture, a tradition, a unification of a community, a whole history that creates what a community is. It's all embodied in a language"

-Noam Chomsky

Índice general

1. Introducción **1**

1. Las partes del lenguaje 2
2. Gramáticas y semántica de expresiones 3
   - 2.1. El lenguaje EAL 4
3. Clasificación de los lenguajes de programación 6
4. Ejercicios para el lector 8

2. Herramientas matemáticas **10**

1. Juicios lógicos 11
2. Reglas de inferencia 12
3. Estructuras recursivas e inducción estructural 15
4. Sistemas de transición 19
5. Ejercicios para el lector 23

3. Sintaxis **29**

1. Sintaxis concreta 30
2. Sintaxis abstracta 31
   - 2.1. Análisis sintáctico 32
   - 2.2. El operador `if` 33
3. El operador let 34
   - 3.1. Sintaxis abstracta de orden superior 34
   - 3.2. Clasificación de variables en el operador let 35
4. Sustitución y α-equivalencias 39
5. Ejercicios para el lector 41

4. Semántica **44**

1. Semántica estática 45
2. Semántica dinámica 47
   - 2.1. Semántica operacional 48
   - 2.2. Semántica de paso pequeño 49
   - 2.3. Semántica de paso grande 51
3. La función eval 57
4. Ejercicios para el lector 58

5. Cálculo Lambda **61**

1. Sintaxis del Cálculo Lambda 62
2. α-equivalencia en el Cálculo Lambda 64
3. Semántica operacional del Cálculo Lambda 64
4. Definibilidad Lambda 65
   - 4.1. Booleanos y operadores lógicos 66
5. Aritmética del Cálculo Lambda 67
   - 5.1. Numerales de Church 67
   - 5.2. Funciones aritméticas 68
6. Datos estructurados en el Cálculo Lambda 71
   - 6.1. Pares 71
   - 6.2. Listas 72
7. Propiedades semánticas del Cálculo Lambda 73
   - 7.1. No terminación 73
   - 7.2. No determinismo 73
   - 7.3. Confluencia 74
8. Combinadores de punto fijo 74
9. Ejercicios para el lector 76

6. MinHS **79**

1. Sintaxis de MinHS 80
   - 1.1. Sintaxis concreta 80
   - 1.2. Sintaxis abstracta 81

2. Semántica de MinHs 82
2.1. Sistemas de tipos 82
2.2. Semántica estática 82
2.3. Semántica dinámica 86
3. Propiedades de MinHS 87
3.1. Seguridad del lenguaje 87
3.2. No terminación 88
4. Ejercicios para el lector 88

7. Inferencia de tipos **90**
1. Estandarización de variables 91
2. Generación de restricciones 92
3. Algoritmo de unificación 94
4. Algoritmo de inferencia de tipos 95
5. Ejercicios para el lector 100

8. Máquinas abstractas **102**
1. La máquina $\mathcal{H}$ 103
1.1. Marcos y la pila de control 103
1.2. Estados 104
1.3. Transiciones 105
2. La máquina $\mathcal{J}$ 108
2.1. Marcos 109
2.2. Ambientes 109
2.3. Estados 109
2.4. Transiciones 110
2.5. Alcance 112
2.6. Cerraduras 113
2.7. Transiciones con cerradura para $\mathcal{J}$ 113
3. Ejercicios para el lector 116

9. TinyC **118**

1. Sintaxis 119
2. La Máquina $\mathcal{C}$ 120
2.1. Marcos 120
2.2. Estados 121
2.3. Ambientes 122
2.4. Transiciones 122
3. Ejercicios para el lector 128

10. Herencia y subtipos **130**
1. Orientación a objetos 131
2. Subtipos 132
2.1. Subtipificado de tipos primitivos 133
2.2. Subtipificado de funciones 134
2.3. Subtipificado para suma y producto 134
2.4. Subtipificado para registros 135
2.5. Elementos máximos 135
3. Conversión de tipos 136
3.1. Conversión ascendente *(Upcasting)* 136
3.2. Conversión descendente *(Downcasting)* 136
4. Ejercicios para el lector 137

11. Java Peso Pluma **138**
1. Sintaxis de Java Peso Pluma 139
2. Tablas de clase 142
3. Semántica dinámica 143
4. Semántica estática 144
5. Propiedades de Java Peso Pluma 146
5.1. Preservación de tipos 146
5.2. Progreso 146
5.3. Seguridad 146
6. Cómo se relaciona Java Peso Pluma con Java? 147
7. Ejercicios para el lector 147

Capítulo 1

Introducción

En el contexto de la computación, los lenguajes de programación fungen como una interfaz entre programador y procesador para establecer una comunicación sobre lo que queremos que la computadora ejecute por nosotros.

Diferentes paradigmas de lenguajes de programación, estilos, reglas y convenciones se han adoptado en las últimas décadas para ajustarse al mejor planteamiento posible de un problema, mantener un estándar declarativo, un nombramiento homogéneo de variables y métodos, o un estilo idéntico compartido entre los diferentes programas que se agrupan en estas categorías.

Iniciaremos el estudio de este manual planteando los conceptos claves que nos permitirán definir, clasificar y analizar a los lenguajes de programación que existen en el contexto de las ciencias de la computación. Estos conceptos nos permitirán definir nuestro propio lenguaje

de programación desde cero.

Por último analizaremos sus propiedades y estudiaremos sus características más importantes para poder dar una implementación robusta aplicando las definiciones que iremos presentando en el desarrollo de esta introducción.

Objetivo

Comenzar el estudio formal de los lenguajes de programación brindando un primer acercamiento a la construcción básica de uno, partiendo de las definiciones de los conceptos que los componen, específicamente; sintaxis, semántica, y pragmática, así como estudiar la clasificación que agrupa y distingue a los diferentes lenguajes computacionales en la actualidad[1].

Planteamiento

Este capítulo inicia con el estudio de los lenguajes de programación, definiendo diferentes gramáticas para generar lenguajes pequeños y familiarizarnos con los elementos que los conforman.

Es de particular interés analizar nuestro primer caso de estudio, el lenguaje EAL: Expresiones aritmético-lógicas proporcionando la gramática y esbozando como podemos interpretar las expresiones que pertenecen al mismo mediante la función de evaluación `eval`.

Para cerrar el capítulo se discute brevemente la clasificación de los lenguajes de programación mas importantes de la actualidad basados en las características y paradigma de programación al que pertenecen.

1 Las partes del lenguaje

La lingüística tiene como propósito definir una serie de reglas que rijan la estructura fundamental que compone un lenguaje. Las reglas que estudian dicha composición pueden ser clasificadas en sintácticas (estructura gramatical), semánticas (significado) y pragmáticas (contexto de uso).

Definición 1.1. (Sintaxis). En el contexto de las ciencias de la computación, la sintaxis de un lenguaje son las reglas que definen las combinaciones de símbolos que se consideran declaraciones o expresiones correctamente estructuradas[a].

[a] Definición extraída de [29], [30] y [31]

[1] Para las personas que poseen una madurez matemática superior a aquella que la que un estudiante de cuarto semestre de la Facultad de Ciencias pudiera tener se recomienda encarecidamente iniciar el estudio de este manual en el capítulo 3: Sintaxis. Los temas recomendados para omitir los primeros dos capítulos del manual son: estructuras discretas, juicios lógicos, reglas de inferencia e inducción estructural.

Definición 1.2. (Semántica). En el contexto de las ciencias de la computación, la semántica describe los procesos que sigue una computadora al ejecutar un programa en un lenguaje específico. Esto se puede mostrar describiendo la relación entre la entrada y la salida de un programa, o una explicación de cómo se ejecutará el programa en una determinada plataforma, creando así un modelo de cálculo[a].

[a] Definición extraída de [20], [21] y [28]

Definición 1.3. (Pragmática). En el contexto de las ciencias de la computación la pragmática se refiere a todos aquellos elementos del lenguaje que nos permiten obtener el significado de una expresión bajo un determinado contexto.

Un ejemplo de esto son las bibliotecas, que nos permiten importar métodos previamente definidos. Así, la interpretación de una instrucción de esta biblioteca solo tiene un significado válido bajo el contexto que la importa[a].

[a] Extraído de [36] y [37]

2 Gramáticas y semántica de expresiones

Los lenguajes (no solo los de programación) requieren de un conjunto de reglas que nos permita formar las expresiones que pertenecen a ellos y un método de interpretación que nos diga el significado de las expresiones formadas a partir de dicho conjunto de reglas.

Las gramáticas de los lenguajes de programación constan de dos niveles: la sintaxis concreta y la sintaxis abstracta. Estudiaremos con especial atención la primera mientras que las sintaxis abstracta será presentada a detalle en el capítulo 3: Sintaxis.

En cursos como autómatas y lenguajes formales[2] se revisan este tipo de estructuras y las reglas que las generan, estas usualmente son representadas como gramáticas libres de contexto. A continuación revisaremos algunos ejemplos aplicados para estructuras matemáticas.

Ejercicio 2.1. Dado el alfabeto $A = \{a, b\}$ define la gramática para generar $\{a^n b^n \mid n \in \mathbb{N}\}$[a].

$$S ::= aSb \mid \varepsilon$$

[a] Ejercicio consultado de [135]

Ejercicio 2.2. Define la gramática para generar n tal que, n $\in$ $\mathbb{N}$[a].

$$\begin{aligned} S &::= BT \mid A \\ T &::= AT \mid A \\ A &::= B \mid 0 \\ B &::= 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9 \end{aligned}$$

[a] Ejercicio consultado de [134]

[2] Conforme al plan de estudios que se imparte desde el 2013 en la Facultad de Ciencias de la Universidad Nacional Autónoma de México con clave de asignatura 1425.

Ejercicio 2.3. Define la gramática para generar expresiones aritméticas parentizadas con notación in-fija para el operador "+"[a]. (puedes suponer definidos los números naturales a partir del ejercicio anterior)[b].

$$S ::= S \ + \ A \mid A$$
$$A ::= (S) \mid \mathbb{N}$$

[a]Ejercicio consultado de [136]

[b]Aquí hacemos abuso de la notación en la última regla puesto que $\mathbb{N}$ no es un elemento de la gramática por si mismo, este hace referencia a los número naturales generados del ejercicio 2.2.

2.1 El lenguaje EAL

Vamos a definir la base del lenguaje que utilizaremos para el resto del manual, este se denomina EAL (expresiones aritméticas y lógicas). Poco a poco iremos extendiendo y agregando instrucciones para hacerlo más robusto, por el momento nos limitaremos a definir los operadores aritméticos básicos: "+", "-","*" y los operadores lógicos: "<", "isZero" y "not". De igual forma definimos las constantes `true`, `false` y los números naturales.

Definición 2.1 (Sintaxis de Expresiones aritmético-lógicas: EAL)**.**

$$e \ ::= \ n \ \mid \ \texttt{true} \ \mid \ \texttt{false} \ \mid \ e + e \ \mid \ e - e \ \mid \ e * e \ \mid \ e < e$$
$$\mid \ \texttt{isZero}\ e \ \mid \ \texttt{not}\ e$$

Nuevamente hacemos abuso en la notación para n, en donde $n \in \mathbb{N}$[a].

[a]Definición formulada de [1], [5]. [12], [26] y [39].

Definición 2.2 (Semántica de EAL)**.** Función de evaluación para el lenguaje de expresiones aritmético-lógicas.

$$[\![\cdot]\!] : \mathsf{EAL} \to \mathbb{N} \cup \mathit{Bool}$$

Esto quiere decir que la función de evaluación recibe una expresión del lenguaje EAL y regresa un número natural ó un booleano

$$\begin{aligned}
[\![n]\!] &= n \\
[\![\texttt{true}]\!] &= \texttt{true} \\
[\![\texttt{false}]\!] &= \texttt{false} \\
[\![e_1 + e_2]\!] &= [\![e_1]\!] +_{\mathbb{N}} [\![e_2]\!] \\
[\![e_1 - e_2]\!] &= [\![e_1]\!] -_{\mathbb{N}} [\![e_2]\!] \\
[\![e_1 * e_2]\!] &= [\![e_1]\!] \times_{\mathbb{N}} [\![e_2]\!] \\
[\![e_1 < e_2]\!] &= [\![e_1]\!] <_{\mathbb{N}} [\![e_2]\!] \\
[\![\texttt{isZero}\ e]\!] &= \mathsf{zero}_{\mathbb{N}}[\![e]\!] \\
[\![\texttt{not}\ e]\!] &= \neg[\![e]\!]
\end{aligned}$$

En donde $n \in \mathbb{N}$ y las constantes `false`, `true` $\in \mathit{Bool}$.
Los operadores $+_{\mathbb{N}}, \times_{\mathbb{N}}, <_{\mathbb{N}}$ y $\mathsf{zero}_{\mathbb{N}}$ representan las funciones de suma, producto, menor que y el *test* de cero definidas para los naturales respectivamente, mientras que $\neg$ es la negación lógica para Bool[a].

[a]Definición extraída de [1], [5]. [12], [26] y [39].

A partir de la definición anterior se puede implementar un intérprete para expresiones del lenguaje EAL mediante una función recursiva `eval` como sigue:

```
eval n           = n
eval true        = true
eval false       = false
eval (e1 + e2)   = eval e1 + eval e2
eval (e1 - e2)   = eval e1 - eval e2
eval (e1 * e2)   = eval e1 * eval e2
eval (e1 < e2)   = eval e1 < eval e2
eval (not e)     = not eval e
eval (isZero e)  = eval e == 0
```

Con esta definición ahora podemos evaluar algunas expresiones pertenecientes a nuestro lenguaje EAL.

Ejercicio 2.4. Evalúa la expresión: `not` $3 < 5 + 7$ haciendo uso de la implementación del intérprete anteriormente definido para EAL.

```
eval not 3 < 5 + 7  =
not eval 3 < 5 + 7  =
not eval 3 < eval 5 + 7  =
not eval 3 <  eval 5 + eval 7 =
not 3 < 5 + 7  =
not 3 < 12  =
not true  =
false
```

Ejercicio 2.5. Evalúa la expresión: 3 * `true` > 8 haciendo uso de la implementación del intérprete anteriormente definido para EAL.

```
eval 3 * true > 8       =
eval 3 * true > eval 8  =
eval 3 * eval true >  8 =
3 * true > 8
```

En este caso, aún cuando la expresión EAL está bien formada, no existe una regla en nuestro intérprete que nos permita continuar con la evaluación, por lo tanto, no se puede obtener un resultado. Más adelante estudiaremos mecanismos para evitar que casos como este figuren en la evaluación de un lenguaje.

Definición 2.3 (Gramática ambigua)**.** Sea G Una gramática libre de contexto, decimos que que G es una gramática ambigua si existe una cadena para la cual se pueda tener más de una derivación a la izquierda[a].

[a] Definición extraída de [5]. [12], [40] y [41].

Ejercicio 2.6. ¿La gramática definida para EAL es ambigua? Argumenta tu respuesta.
Sí porque podemos derivar de dos formas distintas la cadena: $3 + 4 * 5$

$$\begin{array}{rcl} e & \rightarrow & e + e \\ & \rightarrow & 3 + e \\ & \rightarrow & 3 + e * e \\ & \rightarrow & 3 + 4 * e \\ & \rightarrow & 3 + 4 * 5 \end{array}$$

$$\begin{array}{rcl} e & \rightarrow & e * e \\ & \rightarrow & e * 5 \\ & \rightarrow & e + e * 5 \\ & \rightarrow & e + 4 * 5 \\ & \rightarrow & 3 + 4 * 5 \end{array}$$

Ejercicio 2.7. ¿Cómo eliminarías la ambigüedad de esta gramática?

Para eliminar la ambigüedad, se requiere de la introducción de paréntesis para los operadores binarios. Este cambio permite la asociatividad y hace explícita la precedencia en la cuál las operaciones se tienen que evaluar. La gramática de EAL es entonces modificada como sigue:

$$\begin{array}{rcl} e & ::= & n \mid \texttt{true} \mid \texttt{false} \mid (e + e) \mid (e - e) \mid (e * e) \\ & & \mid (e < e) \mid \texttt{isZero}\ e \mid \texttt{not}\ e \end{array}$$

En el ejemplo anterior la ambigüedad queda eliminada en su totalidad al forzar la asociatividad de los operadores dependiendo de cual se quiera resolver primero:

$$(3 + (4 * 5)) \text{ ó } ((3 + 4) * 5)$$

3 Clasificación de los lenguajes de programación

En computación existen diferentes clasificaciones que nos permiten agrupar lenguajes de programación según sus características principales, paradigma de programación al que pertenecen, tecnología involucrada en el proceso de traducción a lenguaje de máquina, etc. A continuación se en listan algunas de estas categorías:

- **Lenguajes compilados**: que como el nombre lo indica precisan de un compilador para traducir el código de alto nivel a instrucciones de máquina que el procesador pueda ejecutar. Generalmente suelen ser mas lentos al momento de ejecutarse que el resto de las categorías ya que requieren de un sistema sofisticado de inferencia de tipos, optimizaciones en la traducción de las instrucciones y en ocasiones entornos especiales para la ejecución de los programas como la máquina virtual de Java (*JVM* por sus siglas en inglés)[3].

- **Lenguajes interpretados**: en donde las instrucciones son mapeadas a su traducción equivalente en el lenguaje de máquina cuando la línea que contiene dicha instrucción

[3]Información obtenida de [43], [44], [45], [46]

se vaya a ejecutar por el procesador[4]. Generalmente son menos robustos a la hora de encontrar errores de tipo, errores en la tipografía, caracteres faltantes, etc. Sin embargo este tipo de lenguajes de programación resultan particularmente útiles por que permiten el modelado y la realización de programas mucho más rápido que los lenguajes compilados[5].

- **Lenguajes funcionales**: este tipo de lenguajes se enfocan en la especificación de un problema mediante condiciones. La programación funcional es un paradigma de programación en el que los programas se construyen aplicando y componiendo funciones. Íntimamente relacionado con el paradigma de programación declarativa en el que las definiciones de funciones son árboles de expresiones que asignan valores a otros valores, en lugar de una secuencia de declaraciones imperativas que actualizan el estado de ejecución del programa[6].

- **Lenguajes imperativos**: que contrastan con la clasificación anterior por su naturaleza secuencial, podemos encapsular sus características más importantes en la siguiente definición: "Un paradigma de programación de software que utiliza declaraciones que cambian el estado de un programa. De la misma manera que el modo imperativo en los lenguajes naturales expresa órdenes, un programa imperativo consta de órdenes que debe ejecutar la computadora. La programación imperativa se centra en describir cómo funciona un programa paso a paso[7] en lugar de descripciones de alto nivel de sus resultados esperados."[8]

  El término se utiliza con frecuencia como un contrapunto de la programación declarativa, siendo el primero un conjunto de instrucciones para solucionar un determinado problema mientras que el segundo se centra en la especificación del problema mismo.

- **Lenguajes orientados a objetos:** que modelan las entidades de un programa con su identidad, métodos, variables y que brindan características deseables como la herencia, el polimorfismo y el encapsulamiento de datos.

  Esta clasificación pertenece al paradigma imperativo dado que se trabaja estrechamente con el estado del programa en cada paso de la ejecución, sus direcciones de memoria y sus sentencias de control[9].

- **Lenguajes lógicos**: los programas que se escriben en esta categoría de lenguajes se pueden definir de la siguiente forma: "Un programa, base de datos o base de conocimientos en un lenguaje de programación lógica es un conjunto de oraciones, predicados o afirmaciones que expresan hechos y reglas sobre un dominio específico."[10]

  Generalmente este tipo de programas se apoyan definiendo predicados y objetos a los cuales estos predicados pueden ser aplicados así como instrucciones y operadores lógicos para dar solución a los problemas planteados en este modelo.

[4]También es posible interpretar las expresiones de un lenguaje como el valor asociado a dicha expresión una vez que se evalúa. En este curso estudiaremos este proceso en el capítulo 4: Semántica.
[5]Información obtenida de [43], [44], [45], [46]
[6]Definición extraída de [51] y [52]
[7][[53]
[8]Extraído de [53], [54], [55] y [56]
[9]Definición extraída de [47], [48], [49]
[10]Definición extraída de [57]

Ejercicio 3.1. Da una implementación del algoritmo `selectionSort` para arreglos en algún lenguaje de programación imperativa.

```
void swap(int *xp, int *yp){
    int temp = *xp;
    *xp = *yp;
    *yp = temp;
}

void selectionSort(int arr[], int n){
    int i, j, min_idx;
    for (i = 0; i < n-1; i++){
        min_idx = i;
        for (j = i+1; j < n; j++)
        if (arr[j] < arr[min_idx])
            min_idx = j;
        swap(&arr[min_idx], &arr[i]);
    }
}
```

Ejercicio 3.2. Da una implementación del algoritmo `selectionSort` para listas en algún lenguaje de programación declarativa.

```
ssort :: Ord t => [t] -> [t]
ssort [] = []
ssort xs = let { x = minimum xs }
           in  x : ssort (delete x xs)
```

4 Ejercicios para el lector

Ejercicio 4.1. El uso del carácter especial ';' al terminar una instrucción de programa en lenguajes de programación como C y Java, a qué tipo de regla corresponde?

Ejercicio 4.2. Ejemplifica la aplicación de la pragmática aplicada en el contexto de los lenguajes de programación.

Ejercicio 4.3. En el contexto de los lenguajes de programación muchas veces la semántica de una expresión es explicada en lenguaje natural en la documentación del lenguaje.

Explica por qué esto no es una semántica formal.

Ejercicio 4.4. Escribe una gramática libre de contexto para generar cada uno de los siguientes lenguajes[a].

$$\begin{aligned} L_1 &= \{a^n b^m \mid n > 0,\ m \geq n\} \\ L_2 &= \{a^m b^n c^p d^q \mid m + n = p + q\} \end{aligned}$$

[a] Ejercicio extraído de [138]

Ejercicio 4.5. Escribe una gramática libre de contexto para el lenguaje

$$L = \{w \in \{a, b\}^* \mid w = w^R\}$$

En donde w^R representa la reversa de la cadena w.

Ejercicio 4.6. Escribe una gramática libre de contexto para el siguiente lenguaje:

$$L = \{w \in \{(,)\}^* \mid \forall i \in \{1, 2, \ldots, |w|\},\ \#_i(w) \geq 0,\ \#_i(w) = 0 \text{ si y solo si } i = |w|\}$$

En donde $\#_i(w)$ denota el número de paréntesis abiertos menos el número de paréntesis cerrados hasta la posición i en la cadena w.

Ejercicio 4.7. Evalúa la expresión del lenguaje EAL utilizando la función `eval`.

$$(7 * ((8 < 3) * 9))$$

Es posible obtener un resultado de la evaluación la expresión anterior? argumenta tu respuesta.

Ejercicio 4.8. Evalúa la expresión del lenguaje EAL utilizando la función `eval`.

$$\texttt{not(isZero}((7 * 4) + (1 * 0)))$$

Es posible obtener un resultado de la evaluación la expresión anterior? argumenta tu respuesta.

Ejercicio 4.9. Escribe el algoritmo de la búsqueda binaria sobre una lista ordenada en el paradigma de programación imperativa.

Ejercicio 4.10. Escribe el algoritmo de la búsqueda binaria sobre una lista ordenada en el paradigma de programación declarativa.

Capítulo 2

Herramientas matemáticas

Para estudiar a los lenguajes de programación es necesario definir una estructura que nos permita capturar su esencia matemática y nos proporcione un mecanismo para demostrar características y propiedades asociadas a estos.

Una manera de definir formalmente un lenguaje de programación es mediante la aplicación de juicios lógicos para definir propiedades, pertenencia de los elementos de la estructura a un conjunto, relaciones entre diferentes elementos, etc[1].

[1] Para las personas que poseen una madurez matemática superior a aquella que la que un estudiante de cuarto semestre de la Facultad de Ciencias pudiera tener se recomienda encarecidamente iniciar el estudio de este manual en el capítulo 3: Sintaxis. Los temas recomendados para omitir los primeros dos capítulos del manual son: estructuras discretas, juicios lógicos, reglas de inferencia e inducción estructural.

Objetivo

Estudiar las estructuras matemáticas que utilizaremos para familiarizarnos con el concepto de inducción estructural. En particular centraremos nuestra atención a estructuras recursivas como listas de tipo A y árboles binarios balanceados de tipo A[2] junto con las reglas que nos permiten construir estas estructuras.

Este tipo de mecanismos es particularmente útil para ilustrar las propiedades de los lenguajes que definiremos a lo largo de los capítulos de este manual.

Planteamiento

En este capítulo exploraremos diferentes estructuras matemáticas, empezando por los juicios lógicos. Estos constituyen las entidades fundamentales sobre las cuales iniciaremos el estudio de los lenguajes. También estudiaremos brevemente estructuras recursivas como cadenas y listas para enunciar el principio de inducción que nos permite demostrar propiedades sobre dichas estructuras.

Por último estudiaremos un sistema de transición para modelar los estados y reglas de evaluación de la instancia de un juego.

1 Juicios lógicos

Una parte fundamental del razonamiento matemático es la capacidad de determinar si una afirmación es verdadera o falsa basado en la información que poseemos sobre los elementos que son mencionados en dicha afirmación.

Para poder aplicar un juicio sobre algún elemento, objeto o entidad para determinar su veracidad primero debemos definir dichos elementos. A continuación vamos a introducir los conceptos necesarios para poder formalizar esta noción y el cómo aplicarla. Esto será de utilidad para definir características, reglas de construcción, reglas de evaluación en los lenguajes de programación y modelos de computo que discutiremos a lo largo del manual.

Definición 1.1. ¿Qué es una entidad matemática?
En el lenguaje habitual de las matemáticas, una entidad es cualquier cosa que ha sido definida formalmente y con la que se pueden realizar razonamientos deductivos y pruebas matemáticas[a].

[a] Definición formulada de [58], [59] y [60]

Definición 1.2. ¿Qué es un juicio?
En lógica matemática, un juicio es una afirmación o enunciación de una característica o propiedad sobre un objecto o elemento de un dominio[a].

[a] Definición formulada de [61] y [62]

[2] donde A es un conjunto o una colección de elementos.

Los juicios que denotan que un objeto x cumple con la propiedad P usualmente son escritos utilizando notación posfija, como $x\ P$.

Aquellos juicios que se aplican sobre relaciones y operadores binarios, usualmente son escritos con notación infija como en el caso de la relación de igualdad $x\ =\ y$.

Ejercicio 1.1. Proporciona un listado de juicios sobre objetos matemáticos (Puedes suponer definidos con anterioridad elementos como $\mathbb{N}$, $\mathbb{Q}$, *Bool*, *String*, funciones, relaciones de orden, operadores aritméticos, etc).

"Hola Mundo" *String*	La cadena "Hola Mundo" es de tipo *String*.
L NP	El lenguaje L es NP.
$a > b$	el número a es más grande que el número b.
A | B	El evento A es independiente al evento B.
`true` *Bool*	La constante `true` es de tipo *Bool*.
h $= g \circ f$	la función h es la composición de la función g con la función f.
3.1416... $\mathbb{I}$	El número 3.1416... es irracional
0 $\mathbb{N}$	0 es un Natural.
S(0) impar	El sucesor de 0 es impar.
$\frac{p}{q}$ $\mathbb{Q}$	La fracción $\frac{p}{q}$ es un número racional.
...	

2 Reglas de inferencia

Las reglas de inferencia[3] son una representación de la aplicación de un juicio de la forma: "Si las premisas $p_1, p_2,\ ...\ p_n$ son verdaderas, entonces c es verdadero."
Visualmente esto se denota como:

$$\frac{p_1, p_2,\ ...\ p_n}{c}$$

Podemos observar que las reglas de inferencia están compuestas por dos secciones: una sección superior y una inferior. La sección superior es en donde se agrupan las condiciones necesarias para que la regla pueda ser aplicada (a los elementos enlistados en esta parte se les conoce como premisas o hipótesis), cada una de estas están separadas por una coma (',') y todas deben de cumplirse[4].

La sección inferior de una regla de inferencia enlista el resultado de la aplicación de la regla a las premisas. Ocasionalmente la regla tendrá un nombre que ayuda a identificar cuál de ellas es aplicada cuando se trabaja con derivaciones y razonamientos[5]. En esta sección de la regla puede haber uno o más elementos resultados de la aplicación pero no puede ser vacía.

Los axiomas son las reglas de inferencia que carecen de hipótesis dado que siempre serán verdad y no es necesario suponer nada para concluirlos.

[3] En esta manual utilizaremos el término "regla de inferencia " o "juicio lógico " de forma indistinta.

[4] El signo de puntuación (',') se puede interpretar como el operador $\wedge$ donde cada hipótesis es un argumento de dicho operador.

[5] Definición formulada de [1], [5], [9], [63], [64] y [91].

Tomemos como ejemplo la definición de las cadenas binarias de tipo A (denominaremos a este tipo de dato como A*) para ilustrar las reglas de inferencia y sus partes anteriormente discutidas.

Definición 2.1. Sea $\mathrm{A} = \{0, 1\}$ (*def*), definimos el tipo de dato recursivo A^* [a] como sigue:

$$\frac{}{\varepsilon\ \mathrm{A}^*}\ \textit{(eps)} \qquad \frac{a\ \mathrm{A},\ s\ \mathrm{A}^*}{\langle a, s\rangle\ \mathrm{A}^*}\ \textit{(tup)}$$

La regla *eps* se lee como: "la cadena vacía es una cadena binaria de tipo A." Nótese que esta regla es un axioma dado que carece de premisas.

La regla *tup* se lee como: "si a es de tipo A y s es una cadena binaria de tipo A, entonces la construcción de una tupla con cabeza a y cola s es una cadena binaria de tipo A."

Por ejemplo, la cadena 10101 se representa por la 5-tupla $\langle 1, \langle 0, \langle 1, \langle 0, \langle 1,\ \epsilon\rangle\rangle\rangle\rangle\rangle$

[a] Definición formulada a partir de [67], [68] y [69]

Es importante notar que las reglas de inferencia son puramente sintácticas. Por ejemplo, el número "00011"no puede ser generado directamente por las reglas de A*. Su representación como cadena binaria sí: $\langle 0, \langle 0, \langle 0, \langle 1, \langle 1,\ \epsilon\rangle\rangle\rangle\rangle\rangle$. El conjunto de reglas de inferencia de A* no posee alguna que nos ayude a formular los números binarios sin la aplicación de las tuplas.

Ahora que hemos definido una estructura de datos con reglas de inferencia, es natural preguntarse "¿cómo razonar sobre esta?". Por ejemplo como demostrar que[6]:

$$\langle 0, \langle 0, \langle 0, \langle 1, \langle 1,\ \epsilon\rangle\rangle\rangle\rangle\rangle \in \mathrm{A}^*$$

$$\langle 0, \langle 7, \langle 4, \langle 1,\ \epsilon\rangle\rangle\rangle\rangle \notin \mathrm{A}^*$$

Las reglas de inferencia pueden ser utilizadas tanto para definir una propiedad como para demostrar que esta propiedad se cumple.

Si queremos demostrar que un juicio J es válido, tenemos que encontrar una regla que tenga a J como conclusión. Si la regla es un axioma entonces nuestra prueba está concluida. Si la regla contiene premisas $p_1, ..., p_n$ entonces hay que aplicar la misma estrategia de forma recursiva donde la premisa p_x con $x \in \{1, .., n\}$ figure como conclusión de alguna regla.

Si en algún punto de la derivación no podemos aplicar ninguna regla y esa premisa no se corresponde con un axioma entonces no se puede demostrar que la propiedad se cumple.

Ejemplifiquemos esto con el siguiente ejercicio.

[6] Aquí estamos haciendo abuso de notación con la aplicación de $\in$ para hacer explícita la propiedad para el primer caso y denotar que para el segundo no es válida.

Ejercicio 2.1. Utiliza las reglas (*eps*) y (*tup*) para derivar la cadena

$$\langle 1, \langle 0, \langle 0, \varepsilon \rangle \rangle \rangle, \qquad \langle 1, \langle 0, \langle 0, \langle 1, \varepsilon \rangle \rangle \rangle \rangle \qquad y \qquad \langle 1, \langle 1, \langle 1, \langle 0, \langle 1, \langle 0, \varepsilon \rangle \rangle \rangle \rangle \rangle \rangle$$

$$\dfrac{\dfrac{\dfrac{

3 Estructuras recursivas e inducción estructural

Ahora discutamos un tipo de estructuras particulares que se relacionan de forma íntima con los conceptos anteriormente planteados; las estructuras recursivas[7]. Estas se generan a partir de dos casos:

- La estructura está compuesta por alguno de sus elementos primitivos. Muchas veces estos elementos constituyen la clausula de escape para la recursión inducida por la definición y darán lugar a uno o más axiomas en su representación como regla de inferencia.
- La estructura está compuesta por una llamada a alguna de sus funciones constructoras con un elemento nuevo y la estructura misma que se desea definir como argumentos[8].

En el caso del tipo de dato A*, el elemento primitivo que pertenece a esta estructura es la cadena vacía (denotada como ε). Los elementos compuestos se generan por una tupla con cabeza a A y cola de tipo A*.

Este principio es válido tanto para definir la estructura recursiva como para las funciones que se quieran aplicar sobre la misma[9]. Para ilustrar este punto definimos la función para concatenar dos elementos de A* de la siguiente forma:

Definición 3.1 (Concatenación de cadenas de A*)**.** Definimos la concatenación de cadenas A* denotada como: $\bullet :: (\mathrm{A}^* \to \mathrm{A}^*) \to \mathrm{A}^*$ mediante la siguiente especificación[a]:

$$\begin{aligned} \text{Caso base:} \quad & \varepsilon \bullet t = t \\ \text{Constructor:} \quad & \langle a, s \rangle \bullet t = \langle a, \langle s \bullet t \rangle \rangle \end{aligned}$$

[a] Definición formulada a partir de [67], [68]

La característica mas importante que las estructuras recursivas poseen es el principio de inducción. Esta propiedad nos permite demostrar que una proposición o propiedad es válida para cualquier elemento de la estructura mediante su aplicación de la siguiente forma:

- Se demuestra la validez de la proposición para los elementos primitivos.
- Posteriormente se asume que la propiedad es válida para cualquier instancia.
- Por último se demuestra la propiedad aplicada para los constructores que generan instancias más grandes aplicando la hipótesis del inciso anterior.

[7] Definición formulada a partir de [91], [92] y [93]

[8] Los elementos primitivos y las funciones constructoras de las estructuras recursivas pueden ser diversas, en el caso particular de las listas el único elemento primitivo de la estructura es la lista vacía y la única función constructora es la función *append* representada por la regla *tup*.

[9] Información obtenida de [94] y [95]

Ejercicio 3.1. Utiliza la inducción estructural para demostrar que si $t \in \mathrm{A}^*$ entonces:

$$t \bullet \varepsilon = t$$

Caso Base $\mathrm{t} = \varepsilon$
$t \bullet \varepsilon = \varepsilon \bullet \varepsilon$ (Por hipótesis)
$\varepsilon \bullet \varepsilon = \varepsilon$ (Por la definición de $\bullet$)
$\varepsilon = t.$

Hipótesis Inductiva $t \bullet \varepsilon = t$

Paso Inductivo Por demostrar para $\langle a, t\rangle \ \forall \ a \in \mathrm{A}$
$\langle a, t\rangle \bullet \varepsilon = \langle a, \langle t \bullet \varepsilon\rangle\rangle$ (Por la definición de $\bullet$)
$\langle t \bullet \varepsilon\rangle = t$ (Por la hipótesis inductiva)
$\langle a, t\rangle \bullet \varepsilon = \langle a, \langle t \bullet \varepsilon\rangle\rangle = \langle a, t\rangle$

Definición 3.2. Definimos la función para contar ocurrencias de los caracteres 1 ó 0 en una cadena denotada por $numChar$ :: (A $\rightarrow$ A*) $\rightarrow$ Int como sigue[a] :

$$numChar(c,\ \varepsilon) = 0$$

$$numChar(c,\ \langle a, t\rangle) = \begin{cases} 1 + numChar(c,\ t) & \text{Sí } a = \mathrm{c} \\ numChar(c,\ t) & \text{Sí } a \neq c \end{cases}$$

[a] Definición formulada a partir de [67], [68] y [69]

Cuando se quiera aplicar el principio de inducción a una función o propiedad que se enuncia para mas de un argumento, la inducción se tiene que fijar sobre alguno de sus parámetros.

En los siguientes ejercicios, cuando interactúen dos elementos $s,\ t \in \mathrm{A}^*$ como en $(s \bullet t)$, la convención será escoger siempre el parámetro izquierdo (en este caso s) para aplicar el principio de inducción, pues la función de concatenación ($\bullet$), está definida para operar sobre el parámetro izquierdo. Esto simplifica el razonamiento al desarrollar el paso inductivo.

Ejemplifiquemos esto con el siguiente ejercicio.

Ejercicio 3.2. Utiliza la inducción estructural para demostrar que sí $c \in \mathrm{A}$ y $s, t \in \mathrm{A}^*$ entonces:

$$numChar(c, s \bullet t) = numChar(c, s) + numChar(c, t)$$

Demostración por inducción sobre s.

Caso Base $s = \varepsilon$
$numChar(c, s \bullet t) = numChar(c, \varepsilon \bullet t)$ (Por hipótesis)
$numChar(c, \varepsilon \bullet t) = numChar(c, t)$ (Por la definición de $\bullet$)
$0 + numChar(c, t) = numChar(c, \varepsilon) + numChar(c, t)$ (Por la definición de $numChar$)
$= numChar(c, s) + numChar(c, t)$

Hipótesis Inductiva $numChar(c, s \bullet t) = numChar(c, s) + numChar(c, t)$

Paso Inductivo Por demostrar para $\langle a, s\rangle \bullet t \;\; \forall\, t \in \mathrm{A}^*$ y $\forall\, a \in \mathrm{A}$
$numChar(c, \langle a, s\rangle \bullet t) = numChar(c, \langle a, s \bullet t\rangle)$ (Por la definición de $\bullet$)

Caso 1 $a = c$ entonces tenemos
$numChar(c, \langle a, s \bullet t\rangle) = 1 + numChar(c, s \bullet t)$ (Por la definición de $numChar$)
$= 1 + numChar(c, s) + numChar(c, t)$ (Por la hipótesis inductiva)
$= numChar(c, \langle a, s\rangle) + numChar(c, t)$ (Por la definición de $numChar$)

Caso 2 $a \neq c$ entonces tenemos
$numChar(c, \langle a, s \bullet t\rangle) = numChar(c, s \bullet t)$ (Por la definición de $numChar$)
$= numChar(c, s) + numChar(c, t)$ (Por la hipótesis inductiva)
$= numChar(c, \langle a, s\rangle) + numChar(c, t)$ (Por la definición de $numChar$)

Definición 3.3. Definimos las funciones $rev :: \mathrm{A}^* \to \mathrm{A}^*$ y $len :: \mathrm{A}^* \to \mathrm{Int}$ para obtener la reversa y la longitud[a] de los elementos de A^* respectivamente como:

$$rev(\varepsilon) = \varepsilon \qquad rev(\langle a, s\rangle) = rev(s) \bullet \langle a, \varepsilon\rangle$$

$$len(\varepsilon) = 0 \qquad len(\langle a, s\rangle) = 1 + len(s)$$

[a] Definición formulada a partir de [67] y [70]

Ejercicio 3.3. Utiliza inducción estructural para demostrar que si $s, t \in \mathrm{A}^*$, entonces:

$$len(s \bullet t) = len(s) + len(t)$$

Demostración por inducción sobre s.

Caso Base $s = \varepsilon$
$len(s \bullet t) \;=\; len(\varepsilon \bullet t)$ (Por la hipótesis)
$len(\varepsilon \bullet t) \;=\; len(t)$ (Por la definición de $\bullet$)
$= 0 + len(t) = len(\varepsilon) + len(t)$ (Por la definición de len)
$= len(s) + len(t)$

Hipótesis inductiva $len(s \bullet t) = len(s) + len(t)$

Paso Inductivo Por demostrar para $\langle a, s\rangle \bullet t \;\; \forall\, t \in \mathrm{A}^*$ y $\forall\, a \in \mathrm{A}$
$len(\langle a, s\rangle \bullet t) = len(\langle a, \langle s \bullet t\rangle\rangle)$ (Por la definición $\bullet$)
$= \; 1 \; + \; len(s \bullet t) \; = \; 1 \; + \; len(s) \; + \; len(t)$ (Por la hipótesis inductiva)
$= (1 + len(s)) + len(t) = len(\langle a, s\rangle) + len(t)$

Ejercicio 3.4. Utiliza inducción estructural para demostrar

$$len(rev(s)) = len(s)$$

Caso Base $s = \varepsilon$
$len(rev(s)) = len(rev(\varepsilon)) = len(\varepsilon) = len(s).$ (Por la definición de rev)

Hipótesis Inductiva $len(rev(s)) = len(s)$

Paso Inductivo Por demostrar para $\langle a, s\rangle \;\forall\, a \in \mathrm{A}$
$len(rev(\langle a, s\rangle)) \;=\; len(rev(s) \bullet \langle a, \varepsilon\rangle)$ (Por la definición de rev)
$= \;\; len(rev(s)) \;+\; len(\langle a, \varepsilon\rangle)$ (Por la propiedad anterior)
$= len(rev(s)) + 1 + len(\varepsilon)$ (Por la definición de len)
$= len(rev(s)) + 1 + 0$ (Por la definición de len)
$= \; 1 \; + \; len(rev(s)) \;\; = \;\; 1 \; + \; len(s)$ (Por la hipótesis inductiva)
$= len(\langle a, s\rangle)$ (Por definición de len)

Definición 3.4. Definimos la estructura de datos árboles binarios de tipo A denotados como $Tree_A$ con las siguientes reglas[a]:

1. $\emptyset \; Tree_A$.
2. Si $t_1 \; Tree_A$ y $t_2 \; Tree_A$ y $a \in \mathrm{A}$ entonces $node(a, t_1, t_2) \; Tree_A$.

3. Son todos.

[a]Definición formulada a partir de [67], [68] y [69]

Definición 3.5. Definimos la función para obtener la altura de los elementos de $Tree_A$ denotada $height(t) :: Tree_A \to \text{Int}$, como sigue:[a]:

$$\begin{aligned} height(\emptyset) &= 0 \\ height(node(a, t_1, t_2)) &= max(h_1, h_2) + 1 \text{ donde } h_1 = height(t_1) \text{ y } h_2 = height(t_2) \end{aligned}$$

[a]Definición formulada a partir de [67], [68] y [69]

Definición 3.6. Definimos la función que cuenta el número de nodos de los elementos de $Tree_A$ denotada $n_n(t) :: Tree_A \to \text{Int}$, como[a]:

$$\begin{aligned} n_n(\emptyset) &= 0 \\ n_n(node(a, t_1, t_2)) &= 1 + n_n(t_1) + n_n(t_2) \end{aligned}$$

[a]Definición formulada a partir de [67], [68] y [69]

Ejercicio 3.5. Demuestra utilizando inducción estructural que si t $Tree_A$ con altura k entonces $n_n(t) \leq 2^k - 1$

Caso Base $t = \emptyset$, $k = 0$

$n_n(\emptyset) = 0 \leq 2^0 - 1 = 1 - 1 = 0$.

Hipótesis Inductiva $n_n(t) \leq 2^k - 1$, para todo t $Tree_A$, donde $height(t) = k$.

Paso Inductivo Por demostrar $n_n(node(a, t_1, t_2)) \leq 2^{k+1} - 1$ donde $height(node(a, t_1, t_2)) = k + 1$ y $height(t_1) = height(t_2) = k$.

$n_n(node(a, t_1, t_2)) = 1 + n_n(t_1) + n_n(t_2)$ (Por definición de $n_n(t)$)
$\leq 1 + (2^k - 1) + (2^k - 1)$ (Por la hipótesis inductiva)
$= 1 + 2(2^k - 1) = 2^{k+1} - 2 + 1 = 2^{k+1} - 1$

4 Sistemas de transición

En el contexto de las ciencias de la computación los sistemas de transición resultan particularmente útiles para obtener cualquier configuración de un programa en cada paso de su ejecución en forma de estados. En particular tendremos dos clases de estados especiales para denotar la configuración inicial y la configuración final. Adicionalmente nos apoyaremos de un conjunto de reglas que nos permiten aplicar instrucciones y operadores para movernos de una configuración a otra[10].

Obtener todas las configuraciones posibles partiendo de un estado inicial a un estado final

[10]Para el enfoque que seguimos en este manual definiremos las reglas de nuestros sistemas de transición como juicios lógicos.

nos facilitará hacer el análisis de complejidad en tiempo y en espacio de nuestros programas.

Estructuras similares se han estudiado en cursos como autómatas y lenguajes formales[11] donde adicionalmente se define el alfabeto y se considera la lectura de un carácter para obtener una configuración distinta. En este caso omitiremos estos elementos y nos concentraremos únicamente en los estados y las reglas de transición.

[11] Conforme al plan de estudios que se imparte desde el 2013 en la Facultad de Ciencias de la Universidad Nacional Autónoma de México con clave de asignatura 1425.

Ejercicio 4.1. Supón que se quiere modelar una partida de pimpón donde hay 2 jugadores A y B (el jugador A siempre empieza primero), y un marcador que cuenta el número de puntos de A y B representado como una terna:

(jugador en turno, puntuación de A, puntuación de B)

De tal forma que cada estado indica la información del juego hasta ese punto.

Para indicar la transición entre estados utilizamos la notación "$\rightarrow$". Por ejemplo si el jugador B anota, la transición se modela como:

$$(\mathrm{B,X,Y}) \rightarrow (\mathrm{B,X,Y+1})$$

ya que el jugador que anota un punto vuelve a tirar la pelota en el siguiente turno. La partida acaba cuando alguno de los dos logra anotar 10 puntos.

Es importante notar que el estado de un juego puede evolucionar hacia distintos estados dependiendo del desarrollo de la partida. Por ejemplo, el estado (B,X,Y) puede transitar al estado (B,X,Y+1) si el jugador B anota el punto, o al estado (A,X,Y) si no anota y el saque pasa al jugador A. Esto implica que la transición $\rightarrow$ no puede modelarse como una función (ya que un mismo estado puede tener más de una imagen), sino como una relación sobre el conjunto de estados, permitiendo que un estado tenga múltiples estados sucesores.

En base a la especificación dada anteriormente contesta lo siguiente:

1. Define formalmente el conjunto de estados (indica estados iniciales con la letra "I", finales con la letra "F" y los intermedios con "Γ"):

$$\mathrm{I} = \{(\mathrm{A}, 0, 0)\}$$

$$\Gamma = \{(\mathrm{C}, \mathrm{X}, \mathrm{Y})\} \text{ donde } \mathrm{C} \in \{\mathrm{A}, \mathrm{B}\} \text{ y } 0 \leq \mathrm{X}, \mathrm{Y} \leq 10$$

$$\mathrm{F} = \{(\mathrm{C}, \mathrm{X}, \mathrm{Y})\} \text{ donde } \mathrm{C} \in \{\mathrm{A}, \mathrm{B}\} \text{ y X o Y} = 10$$

2. Define la relación de transición (enlista cuales son los casos y como se comporta la relación en cada uno de estos).

   Definimos la relación de transición $f(s) :: \mathrm{I} \cup \Gamma \rightarrow \Gamma \cup \mathrm{F}$ a partir de los siguientes casos:

   Sí el jugador A puntúa:

$$f((\mathrm{A}, \mathrm{X}, \mathrm{Y})) = (\mathrm{A}, \mathrm{X}+1, \mathrm{Y})$$

   Sí el jugador B puntúa:

$$f((\mathrm{B}, \mathrm{X}, \mathrm{Y})) = (\mathrm{B}, \mathrm{X}, \mathrm{Y}+1)$$

   Sí el jugador A no puntúa:

$$f((\mathrm{A}, \mathrm{X}, \mathrm{Y})) = (\mathrm{B}, \mathrm{X}, \mathrm{Y})$$

   Sí el jugador B no puntúa:

$$f((\mathrm{B}, \mathrm{X}, \mathrm{Y})) = (\mathrm{A}, \mathrm{X}, \mathrm{Y})$$

   Sí el jugador $\mathrm{C} \in \{\mathrm{A}, \mathrm{B}\}$ hace un saque:

$$f((\mathrm{C}, \mathrm{X}, \mathrm{Y})) = (\mathrm{C}, \mathrm{X}, \mathrm{Y})$$

3. Define mediante reglas de inferencia la implementación para la relación de transición para cada uno de los casos descritos en el ejercicio anterior.

$$\frac{(\mathrm{A}, \mathrm{X}, \mathrm{Y}) \;\; \mathrm{I} \cup \Gamma, \quad (\mathrm{B}, \mathrm{X}, \mathrm{Y}) \;\; \mathrm{I} \cup \Gamma}{(\mathrm{A}, \mathrm{X}, \mathrm{Y}) \rightarrow (\mathrm{B}, \mathrm{X}, \mathrm{Y}) \;\; \mathrm{I} \cup \Gamma} \text{ (A)} \qquad \frac{(\mathrm{B}, \mathrm{X}, \mathrm{Y}) \;\; \mathrm{I} \cup \Gamma, \quad (\mathrm{A}, \mathrm{X}, \mathrm{Y}) \;\; \mathrm{I} \cup \Gamma}{(\mathrm{B}, \mathrm{X}, \mathrm{Y}) \rightarrow (\mathrm{A}, \mathrm{X}, \mathrm{Y}) \;\; \mathrm{I} \cup \Gamma} \text{ (B)}$$

5 Ejercicios para el lector

Ejercicio 5.1. Escribe el árbol de derivación para la siguiente cadena utilizando la definición 2.1 para el tipo de dato A*:

$$\langle 1, \langle 0, \varepsilon \rangle \rangle$$

Ejercicio 5.2. Escribe el árbol de derivación para la siguiente cadena utilizando la definición 2.1 para el tipo de dato A*:

$$\langle 1, \langle 1, \langle 1 \langle 0, \varepsilon \rangle \rangle \rangle \rangle$$

Ejercicio 5.3. Escribe el árbol de derivación para la siguiente cadena utilizando la definición 2.1 para el tipo de dato A*:

$$\langle 1, \langle 0, \langle 0, \langle 0, \langle 0, \langle 1, \varepsilon \rangle \rangle \rangle \rangle \rangle \rangle$$

Definición 5.1. Definimos la función $map :: (\mathrm{A} \to \mathrm{A}) \to \mathrm{A}^* \to \mathrm{A}^*$ que recibe una función f y una cadena s de la siguiente forma[a]:

$$\begin{aligned} map\ f\ \varepsilon &= \varepsilon \\ map\ f\ \langle a, s \rangle &= \langle f\,\mathrm{a}, map\ f\ s \rangle \end{aligned}$$

[a] Definición formulada a partir de [67], [68] y [69]

Ejercicio 5.4. Demuestra utilizando la inducción estructural sobre el tipo de dato A* que:

$$len(map\ \ f\ \langle a, s \rangle) = len(\langle a, s \rangle)$$

Ejercicio 5.5. Demuestra utilizando la inducción estructural sobre el tipo de dato A* que:

$$map\ f\ \langle a, s \rangle \bullet \langle b, t \rangle = map\ f\ \langle a, s \rangle \bullet map\ f\ \langle b, t \rangle$$

Definición 5.2. Un árbol binario completo es un tipo de árbol binario en el que todos los niveles, excepto posiblemente el último, están completamente llenos, y todos los nodos están lo más a la izquierda posible[a].

[a] Definición extraída de: [68]

Ejercicio 5.6. Demuestra utilizando la inducción estructural sobre el tipo de dato $Tree_A$ que se cumple la siguiente afirmación:

Sea $t\ Tree_A$, tal que t es completo, entonces:

$$height(t) = floor(log_2(n_n(t)))$$

Ejercicio 5.7. Demuestra utilizando la inducción estructural sobre el tipo de dato $Tree_A$ que se cumple la siguiente afirmación:

Sea t $Tree_A$, tal que t es completo, entonces:

$$n_i(t) = \frac{n_n(t) - 1}{2}$$

donde $n_i(t)$ es el número de nodos internos en un árbol y $n_n(t)$ es el número de total de nodos.

Ejercicio 5.8. De acuerdo con el sistema de transición definido en el ejercicio 3.1, contesta lo siguiente:

¿El jugador B puede iniciar una partida?

¿Los jugadores A y B pueden empatar?

¿El marcador puede subir mas allá de los 10 puntos para algún jugador?

¿El marcador puede bajar mas allá de los 0 puntos para algún jugador?

¿Es posible moverse de un estado intermedio a un estado final?

¿Es posible moverse de un estado inicial a un estado final?

Justifica tus respuestas

Ejercicio 5.9. De acuerdo con el sistema de transición definido en el ejercicio 3.1 muestra una ejecución en la que el jugador A gane.

Ejercicio 5.10. Suponga que se necesita definir un lenguaje que permita controlar los movimientos de un robot. El robot se mueve sobre una cuadrícula siguiendo las instrucciones especificadas por el programa.

Al inicio el robot se encuentra en la coordenada (0,0) y viendo hacia el norte. El programa consiste en una secuencia posiblemente vacía de los comandos `move` y `turn` separados por punto y coma (';'), donde cada comando tiene el siguiente funcionamiento[a]:

- `turn` hace que el robot de un giro de 90 grados en el sentido de las manecillas del reloj.
- `move` provoca que el robot avance una casilla en la dirección hacía la que está viendo.

Donde un ejemplo de un programa válido puede ser el siguiente:

$$\texttt{move}; \texttt{turn}; \texttt{move}; \texttt{turn}; \texttt{turn}; \texttt{turn}; \texttt{move}$$

Al final de la ejecución programa el robot termina en la casilla (2,1). La primera entrada de la coordenada indica la posición vertical mientras que la segunda es la posición horizontal.

Con la especificación discutida anteriormente contesta los siguientes puntos:

1. Determina el conjunto de estados.
2. Identifica los estados iniciales y finales del sistema de transición.
3. Define la relación de transición $\rightarrow_R$ que indique como se debe transitar entre los estados del sistema.
4. Muestra paso a paso la ejecución del programa:

   $$\texttt{move}; \texttt{turn}; \texttt{move}; \texttt{turn}; \texttt{turn}; \texttt{turn}; \texttt{move}$$

   Utilizando la relación $\rightarrow_R$ y partiendo del estado inicial.

[a] ejercicio extraído de [107]

Para finalizar esta sección vamos a revisar dos ejercicios prácticos que implementaremos en `Haskell`. Estos ejercicios combinan las definiciones de las funciones que hemos estudiado hasta este punto para listas y que ejemplifican las propiedades recursivas de estas estructuras en un lenguaje de programación funcional[12].

[12] Los ejercicios 5.11 y 5.12 fueron extraídos de [75].

Ejercicio 5.11. La validación del número de una tarjeta de crédito se hace implementando un algoritmo `checkSum` obteniendo información de los dígitos que la componen. Para eso implementaremos nuestro propio validador de tarjetas de crédito como sigue:

Los dígitos que están una posición impar (empezando por el índice 0 de izquierda a derecha) se deben duplicar.

Posteriormente se suman todos los dígitos de los números que componen la tarjeta (aquellos números que fueron duplicados deben de ser sumados junto con el resto que no se modificaron).

Por último se aplica el módulo 10 al resultado de la suma, si el resultado es diferente de 0 entonces la tarjeta es inválida, en caso contrario es válida.

1. Implementa la función `getDigits :: Int -> [Int]`

   Por ejemplo, la tarjeta

   1348 1548 9998 6535

   dará como resultado la lista: `[1,3,4,8,1,5,4,8,9,9,9,8,6,5,3,5]` (asumimos que la entrada son solo los dígitos de la tarjeta sin separación).

2. Implementa la función `duplicateOddPositionDigits :: [Int] -> [Int]`

   Por ejemplo: la lista `[1,3,4,8,1,5,4,8,9,9,9,8,6,5,3,5]` solo duplicará los números que estén en una posición impar obteniendo como resultado: `[1,6,4,16,1,10,4,16,9,18,9,16,6,10,3,10]`.

3. Implementa la función `toDigits :: Int -> [Int]`

   El objetivo de la implementación de esta función es el de obtener los dígitos de los números mayores o iguales a 10, de tal forma que si tenemos como parámetro de entrada para esta función a la lista `[1,6,4,16,1,10,4,16,9,18,9,16,6,10,3,10]` obtengamos la siguiente como resultado `[1,6,4,1,6,1,0,4,1,6,9,1,8,9,1,6,1,0,3,1,0]`

4. Implementa la función `sumDigitsInList :: [Int] -> Int`

   Esta función simplemente suma los dígitos contenidos en la lista (puedes utilizar la función sum provista por `GHCi`).

5. Implementa la función `checkSumCreditCard :: Int -> Bool`

   Recuerda que el criterio de validez es que la suma de los dígitos filtrados de la tarjeta módulo 10 sea igual a 0.

Ejercicio 5.12. Las torres de Hanoi son un rompecabezas clásico cuya solución puede ser escrita de forma recursiva. Los discos de diferentes tamaños se apilan del más grande al más pequeño y el objetivo es moverlos del pivote inicial al pivote final, dónde pivotes = discos = n.

Las reglas son las siguientes:

1. Ningún disco puede estar encima de un disco mas pequeño.
2. Por cada movimiento, solo es válido el desplazamiento de un disco hacía otro pivote.

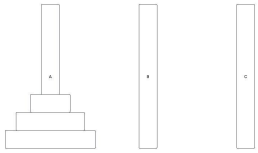
Se definen los siguientes tipos para implementar la solución al problema de la siguiente forma:

```
type Peg  = String
type move = (Peg, Peg)
```

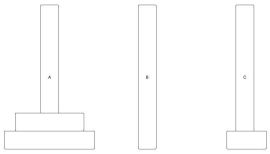
`Peg` designa el nombre del pivote y `move` es una tupla (*origen*, *destino*) que modela el movimiento del disco que está en el tope de la pila del pivote origen al pivote destino.

Implementa la función `Hanoi :: Int` → `[Peg]` → `[move]` Que dada el número de discos y una lista con el nombre de los pivotes regrese una lista con los movimientos para mover los discos desde el pivote de la extrema izquierda hasta el pivote de la extrema derecha, por ejemplo con n = 3:

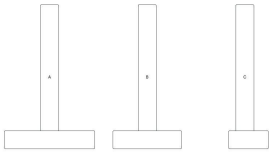
```
Hanoi 3 [''A'' ''B'' ''C''] =
[(''A'',''C''),(''A'',''B''),(''C'',''B''),
(''A'',''C''),(''B'',''A''),(''B'',''C''),(''A'',''C'')]
```



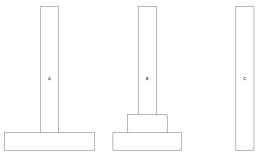
("A","C")



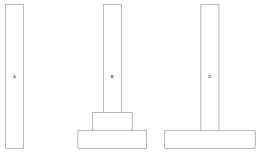
("A","B")



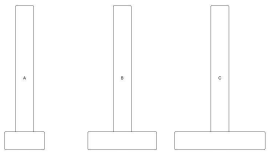
("C","B")



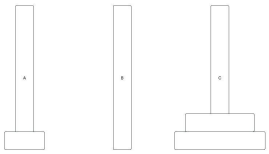
("A","C")



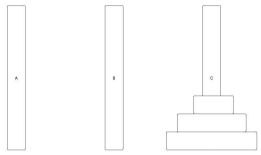
("B","A")



("B","C")

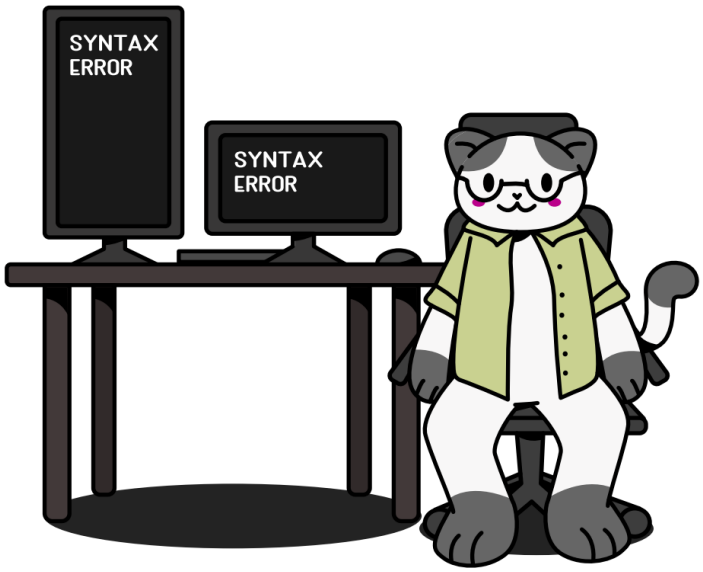


("A","C")



Capítulo 3

Sintaxis



La sintaxis de un lenguaje de programación permite delimitar el conjunto de las cadenas que pertenecen a este. Al mismo tiempo constituye una herramienta de razonamiento para demostrar propiedades utilizando el principio de inducción aplicado a los juicios que la definen.

De forma similar la sintaxis marca la pauta para la definición de funciones, como la función de evaluación para obtener el valor asociado a una expresión[1]. Al mismo tiempo será en esta estructura donde nos apoyaremos para definir los procesos de análisis para determinar la validez de dichas expresiones.

Para poder comenzar a estudiar los niveles sintácticos de un lenguaje de programación, utilizaremos la definición de un nuevo lenguaje el cuál extenderemos con instrucciones a

[1]Este proceso de evaluación no siempre es posible aún cuando se aplica para cadenas correctamente formadas siguiendo las reglas de la sintaxis del lenguaje. Estudiaremos este caso más adelante en el siguiente capítulo.

lo largo del desarrollo del manual y lo denominaremos como EAB (expresiones aritmético-booleanas). A partir de la definición de este lenguaje, mediante la gramática para generar expresiones, abordaremos la construcción y diferencias que existen entre las diferentes categorías de sintaxis que se introducen para EAB.

Objetivo

Definir el lenguaje EAB junto con sus diferentes niveles sintácticos, mostrando las principales diferencias que existen entre la sintaxis concreta y la sintaxis abstracta. Así como estudiar cómo se relacionan, proporcionando las reglas para implementar el analizador sintáctico de este mismo.

Planteamiento

Iniciaremos el estudio de este capítulo brindando las reglas de la sintaxis de EAB[2] junto con la definición de los nuevos niveles sintácticos; la sintaxis concreta y la sintaxis abstracta, misma que introduce una nueva estructura conocida como árbol de sintaxis abstracta.

Esta estructura captura la jerarquía de operación de las cadena válidas del lenguaje, derivadas a partir de las reglas definidas en la sintaxis concreta. En particular estudiaremos el proceso mediante el cual, una expresión de la sintaxis concreta se relaciona con un árbol de sintaxis abstracta conocido como análisis sintáctico.

En este nuevo lenguaje vamos a ejemplificar como estos dos niveles sintácticos se relacionan entre sí extendiendo su definición para soportar a los operadores `if` y `let`. Posteriormente explicaremos la relación que el operador `let` introduce con las variables ligadas, variables libres, el alcance de una variable y estudiaremos los conceptos de sustitución y α-equivalencia.

1 Sintaxis concreta

Esta nivel sintáctico está pensado para el usuario final, es decir una persona. Es por esto que a esta capa se le denomina de ”alto nivel” dado que es un ser humano y no una computadora quien se requiere pueda leer y escribir programas siguiendo las reglas de construcción del lenguaje en cuestión[3].

Definición 1.1 (Definición de sintaxis concreta para expresiones aritméticas de EAB). [a] Representación de EAB como una gramática de libre de contexto[b].

$$
\begin{array}{lcl}
e & ::= & t \mid e + t \mid e - t \\
t & ::= & f \mid t * f \\
f & ::= & n \mid (e) \\
n & ::= & d \mid nd \\
d & ::= & 0 \mid 1 \mid 2 \mid 3 \mid 4 \mid 5 \mid 6 \mid 7 \mid 8 \mid 9
\end{array}
$$

[a] Comenzaremos definiendo las operaciones aritméticas e iremos extendiendo las operaciones boolea-

[2] Este lenguaje es una definición independiente a EAL vista en el capítulo 1: Introducción, dado que iremos extendiendo y agregando nuevas instrucciones durante el desarrollo de los siguientes capítulos.

[3] Definición formulada a partir de [1], [2], [5], [12] y [76]

nas con las constantes `true`, `false` y los operadores `if`, `else` mas adelante en este capítulo.
[b]Definición de [12].

Definición 1.2 (Definición de la sintaxis concreta para expresiones ariméticas de EAB)**.** En su representación como juicios lógicos[a]:

$$\frac{}{0\ \mathrm{D}} \qquad \frac{}{1\ \mathrm{D}} \qquad \cdots \qquad \frac{}{9\ \mathrm{D}}$$

$$\frac{d\ \mathrm{D}}{d\ \mathrm{N}} \qquad \frac{n\ \mathrm{N} \quad d\ \mathrm{D}}{nd\ \mathrm{N}}$$

$$\frac{n\ \mathrm{N}}{n\ \mathrm{F}} \qquad \frac{e\ \mathrm{E}}{(e)\ \mathrm{F}}$$

$$\frac{f\ \mathrm{F}}{f\ \mathrm{T}} \qquad \frac{t\ \mathrm{T} \quad f\ \mathrm{F}}{t * f\ \mathrm{T}}$$

$$\frac{t\ \mathrm{T}}{t\ \mathrm{E}} \qquad \frac{e\ \mathrm{E} \quad t\ \mathrm{T}}{e + t\ \ \mathrm{E}}$$

$$\frac{e\ \mathrm{E} \quad t\ \mathrm{T}}{e - t\ \ \mathrm{E}}$$

[a]Definición extraída de [12].

A las expresiones de EAB formuladas a partir de la sintaxis concreta, las representaremos con el juicio $e\ E$ que se lee como: "e es una expresión de EAB."

2 Sintaxis abstracta

La sintaxis abstracta, como su nombre indica, se relaciona con la abstracción de la estructura del lenguaje y comúnmente es definida mediante reglas de inferencia para construir un árbol sintáctico. Donde cada nodo corresponde a una instrucción ó operador y los hijos a los parámetros que recibe dicha instrucción[4].

Las hojas de los árboles sintácticos corresponden a los valores de los tipos primitivos del lenguaje (enteros, booleanos, caracteres, etc.) En este capítulo vamos a denotar a un árbol de sintaxis abstracta como "ASA".

Los árboles sintácticos son útiles para definir la jerarquía de las operaciones a realizar durante la evaluación de una expresión bien formada y eliminar la ambigüedad del mismo, dado que un árbol de sintaxis abstracta es único sin importar la representación que una expresión tenga en sintaxis concreta.

Definición 2.1 (Sintaxis abstracta de EAB)**.** [a]

$$\frac{n \in \mathbb{N}}{num[n]\ ASA} \qquad \frac{t_1\ ASA \quad t_2\ ASA}{sum(t_1, t_2)\ ASA} \qquad \frac{t_1\ ASA \quad t_2\ ASA}{res(t_1, t_2)\ ASA}$$

[4]Definición formulada a partir de [5], [12], [77], [78] y [79].

$$\frac{t_1\ ASA \qquad t_2\ ASA}{prod(t_1, t_2)\ ASA}$$

En donde el predicado $t_1\ ASA$ se lee como: " t_1 es un árbol de sintaxis abstracta".

Es importante notar que los árboles de sintaxis abstracta aunque similares a la expresión de la sintaxis concreta que representan, corresponden a una categoría distinta de los niveles sintácticos de EAB.

[a] Esta definición fue acuñada a partir de [2], [5] y [12]

2.1 Análisis sintáctico

Al proceso mediante el cual se obtiene el árbol de sintaxis abstracta de una expresión en sintaxis concreta se le conoce como análisis sintáctico. Es importante notar que no siempre será posible encontrar el árbol de sintaxis abstracta.

De tal forma que el objetivo ahora será definir formalmente la relación entre ambos niveles donde el siguiente juicio es válido:[5]

$$e\ E \longleftrightarrow a\ ASA$$

Esta relación se interpreta como válida si y solo sí a es el árbol de sintaxis abstracta asociado a la expresión e. Definimos entonces las reglas para implementar este proceso en EAB como sigue:

Definición 2.2 (Analizador sintáctico para expresiones de lenguaje de EAB). [a]

$$\frac{e\ \mathsf{N} \qquad e \in \mathbb{N}}{e\ \mathsf{F} \longleftrightarrow num[e]\ ASA}\ \textit{(num)} \qquad \frac{e\ \mathsf{E} \longleftrightarrow a\ ASA}{(e)\ \mathsf{F} \longleftrightarrow a\ ASA}\ \textit{(parentE)}$$

$$\frac{e\ \mathsf{F} \longleftrightarrow a\ ASA}{e\ \mathsf{T} \longleftrightarrow a\ ASA}\ \textit{(fact)} \qquad \frac{e_1\ \mathsf{T} \longleftrightarrow a_1\ ASA \qquad e_2\ \mathsf{F} \longleftrightarrow a_2\ ASA}{e_1 * e_2\ \mathsf{T} \longleftrightarrow prod(a_1, a_2)\ ASA}\ \textit{(prod)}$$

$$\frac{e\ \mathsf{T} \longleftrightarrow a\ ASA}{e\ \mathsf{E} \longleftrightarrow a\ ASA}\ \textit{(exprE)} \qquad \frac{e_1\ \mathsf{E} \longleftrightarrow a_1\ ASA \qquad e_2\ \mathsf{T} \longleftrightarrow a_2\ ASA}{e_1 + e_2\ \mathsf{T} \longleftrightarrow sum(a_1, a_2)\ ASA}\ \textit{(sum)}$$

$$\frac{e_1\ \mathsf{E} \longleftrightarrow a_1\ ASA \qquad e_2\ \mathsf{T} \longleftrightarrow a_2\ ASA}{e_1 - e_2\ \mathsf{T} \longleftrightarrow res(a_1, a_2)\ ASA}\ \textit{(res)}$$

[a] Definición formulada a partir de [1], [2], [5] y [12]

En general el analizador sintáctico de un lenguaje de programación debe de proveer con un ASA a todas las expresiones del mismo y este debe de ser libre de ambigüedad. Formalmente, para EAB, esto se representa como:

- $\forall\ e\ E\ \exists\ a\ ASA$.
- Sí $e\ E \longleftrightarrow a_1\ ASA$ y $e\ E \longleftrightarrow a_2\ ASA$ entonces $a_1 = a_2$.

[5] La definición de esta relación fue formulada a partir de [1], [5] y [12], se refiere a las siguientes publicaciones para un estudio en profundidad sobre los niveles sintácticos de los lenguajes de programación: [76], [77], [78] y [79]

2.2 El operador `if`

Ahora queremos extender EAB agregando instrucciones booleanas que permitan tener condicionales en nuestras expresiones de lenguaje.

El objetivo es introducir las constantes `true` y `false` y el operador `if then else`, tomando en cuenta el parentizado de expresiones para evitar condiciones de "else colgante"[6] y la ambigüedad que deriva de no marcar el alcance de las condicionales.

Para este propósito, marcaremos con paréntesis la expresión inmediata a evaluar después del marcador `then`.

Ejercicio 2.1. Define la sintaxis concreta para extender nuestro lenguaje EAB con las instrucciones lógicas previamente discutidas[a].

$$\frac{}{\texttt{true}\ \mathrm{B}} \qquad \frac{}{\texttt{false}\ \mathrm{B}}$$

$$\frac{c\ \mathrm{E}\ t\ \mathrm{E}\ e\ \mathrm{E}}{\texttt{if}\ c\ \texttt{then}\ (t)\ \texttt{else}\ e\ \mathrm{E}} \qquad \frac{x\ \mathrm{B}}{x\ \mathrm{E}}$$

Nótese que el operador if forza el encapsulamiento de la expresión then empleando parentizado explícito. Esta regla podría hacer uso de una estrategia ó heurística para eliminar la ambigüedad del operador y los paréntesis.

Por simplicidad y para facilitar la interpretación de la jerarquía de condicionales, utilizaremos el parentizado explicito.

[a] Definición formulada a partir de [107]

Notemos que con la sintaxis extendida ahora podemos tener expresiones que son sintácticamente correctas, pero que no tienen sentido como:

$$\texttt{true} + 1$$

Por el momento permitiremos este tipo de expresiones y en el siguiente capítulo, introduciremos el concepto de "tipificado" para añadir una capa de verificación antes de saltar directamente a la evaluación de expresiones.

Ejercicio 2.2. Define los juicios que componen la sintaxis abstracta para el operador `if`

$$\frac{\texttt{true}\ \mathrm{B}}{\textit{bool[true] ASA}} \qquad \frac{\texttt{false}\ \mathrm{B}}{\textit{bool[false] ASA}}$$

$$\frac{c\ \textit{ASA}\ t\ \textit{ASA}\ e\ \textit{ASA}}{\textit{if}(c,t,e)\ \textit{ASA}}$$

[6] Este problema es conocido en inglés como "*dangling else*" recurrente en la implementación de *parsers* en el que una cláusula else opcional en una declaración if-then(-else) da como resultado que los condicionales anidados sean ambiguos. [90]

Ejercicio 2.3. Define las reglas del analizador sintáctico para el operador `If`.

$$\frac{}{\texttt{true}\ \mathrm{B} \longleftrightarrow \textit{bool[true] ASA}}\ (\textit{true}) \qquad \frac{}{\texttt{false}\ \mathrm{B} \longleftrightarrow \textit{bool[false] ASA}}\ (\textit{false})$$

$$\frac{e\ \mathrm{B} \longleftrightarrow a\ ASA}{e\ \mathrm{E} \longleftrightarrow a\ ASA}\ (\textit{bool})$$

$$\frac{e_1\ \mathrm{E} \longleftrightarrow a_1\ ASA \quad e_2\ \mathrm{E} \longleftrightarrow a_2\ \mathrm{ASA} \quad e_3\ \mathrm{E} \longleftrightarrow a_3\ ASA}{\texttt{if}\ e_1\ \texttt{then}\ (e_2)\ \texttt{else}\ e_3\ \texttt{E} \longleftrightarrow \textit{if}(a_1, a_2, a_3)\ ASA}\ (if_e)$$

3 El operador let

Ahora necesitamos extender aún mas EAB para incluir expresiones de tipo:

$$\texttt{let}\ \ x = e_1\ \texttt{in}\ e_2\ \texttt{end}$$

La intuición de esta clase de expresiones es que en su proceso de evaluación las apariciones de la variable x en e_2, serán sustituidas por el resultado de la evaluación de e_1.

Para definir al operador `let` en EAB, se tienen que definir los juicios lógicos para la sintaxis concreta y abstracta. Así como una categoría nueva para los identificadores de variables que en conjunto introducen el concepto de ligado de variable.

Definamos entonces la sintaxis concreta de la nueva instrucción como sigue.

Ejercicio 3.1. Proporciona las sintaxis concreta[a] para agregar la instrucción `let` de tal forma que sea congruente con las especificación mencionada anteriormente[b].

$$\frac{e\ \text{identificador}}{e\ \mathrm{V}}\ (\textit{var}) \quad \frac{e\ \mathrm{V}}{e\ \mathrm{B}}\ (\textit{vIn}) \quad \frac{e\ \mathrm{V}}{e\ \mathrm{F}}\ (\textit{vfac}) \quad \frac{x\ \mathrm{V} \quad e_1\ \mathrm{E} \quad e_2\ \mathrm{E}}{\texttt{let}\ x\ \texttt{=}\ e_1\ \texttt{in}\ e_2\ \texttt{end E}}\ (\textit{let})$$

[a]Recordemos que las categorías gramaticales B (definición 3.1) y F (definición 1.2) engloban la definición de los booleanos y los factores en la gramática concreta respectivamente.

[b]Definición formulada a partir de [1], [5], [12] y [81]

Ahora queremos definir la sintaxis abstracta capturando el ligado de variable. Es natural pensar en proponer juicios como:

$$\frac{x\ \ \text{identificador}}{x\ ASA} \qquad \frac{x\ ASA \quad a_1\ ASA \quad a_2\ ASA}{\textit{let}(x, a_1, a_2)\ ASA}$$

3.1 Sintaxis abstracta de orden superior

El problema con esta definición es que no captura el ligado de variable x en a_2, no hay una relación explícita entre ambos elementos. Para ello es necesario utilizar otra categoría conocida como sintaxis abstracta de orden superior.

La sintaxis abstracta de orden superior, abstrae el ligado como parte del metalenguaje con el que se define el ASA, es decir el árbol mismo delimita el alcance de la variable ligada a este.

Extenderemos nuestro lenguaje EAB agregando el árbol de sintaxis abstracta $x.t$ que indica que la variable x está ligada en el árbol t. A este ASA se le llama abstracción[7].

[7]Definición acuñada a partir de [1], [2], [5], [12], [80] y [81]

Ejercicio 3.2. Proporciona las sintaxis abstracta para agregar las instrucción `let`[a] considerando el *ASA x.t.*

$$\frac{x \text{ identificador}}{x\ ASA} \qquad \frac{a_1\ ASA \quad a_2\ ASA}{let(a_1,\ x.a_2)\ ASA}$$

Es importante observar que para el juicio lógico que define al árbol de `let`, no se incluye una premisa para la variable de ligado, esta categoría se considera como primitiva y en esta regla está implícito que x es una variable.

[a]Definición formulada a partir de [1], [5], [12] y [81]

Ejercicio 3.3. Define las reglas del analizador sintáctico para agregar las instrucción `let`[a].

$$\frac{e_1 \text{ E} \longleftrightarrow a_1\ ASA \quad e_2 \text{ E} \longleftrightarrow a_2\ ASA}{\texttt{let}\ x\ \texttt{=}\ e_1\ \texttt{in}\ e_2\ \texttt{end E} \longleftrightarrow let(a_1\ x.a_2)\ ASA}\ leta$$

[a]Definición formulada a partir de [1], [5], [12] y [81]

A continuación tenemos tres ejemplos que nos van a ayudar a ilustrar la aplicación de las reglas que hemos definido hasta el momento para construir los *ASA* de EAB partiendo de expresiones derivadas de las reglas de la sintaxis concreta.

Ejercicio 3.4. Obtén la representación en sintaxis abstracta partiendo de la sintaxis concreta de las siguiente expresión:

`let` $x = 1$ `in` $x + 1$ `end`

let(num[1], x.sum(x, num[1])) ASA

Ejercicio 3.5. Obtén la representación en sintaxis abstracta partiendo de la sintaxis concreta de las siguiente expresión:

`let` $y =$ `if true then` (1) `else` 0 `in if` x `then` (`false`) `else true end`

let(if(bool[true], num[1], num[0]), y.if(x, bool[false], bool[true])) ASA

Ejercicio 3.6. Obtén la representación en sintaxis abstracta partiendo de la sintaxis concreta de las siguiente expresión:

`let` x `= false in if` x `then` (`if` α `then` (`if` α `then` (4) `else` 1) `else` 0) `else false end`

let(bool[false], x.if(x, if(α, if(α, num[4], num[1]), num[0]), bool[false])) ASA

3.2 Clasificación de variables en el operador let

La introducción de nuestro nuevo operador `let` acarrea consigo la definición de ligado de una variable, variable ligada[8], variable libre y la definición de la función de sustitución junto con la función para obtener variables libres de una expresión, mismas que serán mostradas a continuación y serán de utilidad para resolver el resto de los ejercicios presentes en este capítulo.

[8]Aún que los nombres son similares ligado de una variable y variable ligada corresponden a dos definiciones distintas.

Definición 3.1 (Variable de ligado)**.** La instancia de ligado de una variable es aquella que da a esta su valor. Es decir, es la aparición de la variable en donde esta es definida[a].

De tal forma que si tenemos la siguiente expresión:

$$\texttt{let}\ \underline{x} = e_1\ \texttt{in}\ e_2\ \texttt{end}$$

Decimos que x es la variable de ligado en la expresión `let`.

[a] Definición formulada de [2], [5], [12], [80] y [81]

Definición 3.2 (Variable ligada)**.** Una variable está ligada si se encuentra contenida dentro del alcance de una variable de ligado con su nombre[a].

De tal forma que si tenemos la siguiente expresión:

$$\texttt{let}\ x = 5\ \texttt{in}\ \underline{x} + y\ \texttt{end}$$

Decimos que x es la variable ligada en el cuerpo de la expresión `let`.

[a] Definición formulada de [2], [5], [12], [80] y [81]

Definición 3.3 (Alcance de variable)**.** El alcance de una variable es la región de un programa en la que esta obtiene su valor[a].

En el caso del operador `let` de EAB el alcance está definido por las palabras reservadas `in` y `end`. Solo en esta región la definición del valor asociado a la variable ligada de la instrucción puede tomar efecto.

[a] Definición formulada de [5] y [12]

Definición 3.4 (Variable libre)**.** Una variable está libre en una expresión si no se encuentra dentro del alcance de una variable de ligado con su nombre. Es decir, una variable es libre si no está ligada[a].

De tal forma que si tenemos la siguiente expresión:

$$\texttt{let}\ x = e_1\ \texttt{in}\ \underline{y} + 1\ \texttt{end}$$

Decimos que y es una variable libre en nuestra expresión `let`.

[a] Definición formulada de [2], [5], [12], [80] y [81]

Definición 3.5 (Función para obtener las variables libres de una expresión)**.** Dado el árbol de sintaxis abstracta a definimos el conjunto de variables libres de la expresión[a], denotado $FV(a)$ como sigue:

$$\begin{array}{lcl} FV(x) & = & \{x\} \\ FV(O(a_1, \cdots, a_n)) & = & FV(a_1) \cup \cdots \cup FV(a_n) \\ FV(x.a) & = & FV(a) \backslash \{x\} \end{array}$$

Es importante remarcar que la expresión $O(a_1, \cdots, a_n)$, comprende un abuso de la no-

tación para representar a todos los operadores aritméticos, booleanos y expresiones let de EAB.

[a]Definición extraída de [2], [5], [12], [80] y [81]

Ejercicio 3.7. Dada la siguiente expresión `let` definida como:

$$
\begin{aligned}
e =_{\text{def}} \texttt{let}\ x = \texttt{let}\ x = (2+4)\ \texttt{in}\ x+1\ \texttt{end}\ \ \texttt{in}\\
\texttt{let}\ y = (x*2)+x\ \texttt{in}\\
(x*y) + \texttt{let}\ z = 7\ \texttt{in}\ y+z\ \texttt{end}\\
\texttt{end}\\
\texttt{end}
\end{aligned}
$$

Por cada expresión let con un color diferente subraya la variable de ligado y las instancias donde esta aparece ligada.

$$
\begin{aligned}
e =_{\text{def}} \texttt{let}\ \underline{x} = \texttt{let}\ \underline{x} = (2+4)\ \texttt{in}\ \underline{x}+1\ \texttt{end}\ \ \texttt{in}\\
\texttt{let}\ \underline{y} = (\underline{x}*2)+\underline{x}\ \texttt{in}\\
(\underline{x}*\underline{y}) + \texttt{let}\ \underline{z} = 7\ \texttt{in}\ \underline{y}+\underline{z}\ \texttt{end}\\
\texttt{end}\\
\texttt{end}
\end{aligned}
$$

Ejercicio 3.8. Por cada expresión `let` en e escribe la variable de ligado, el alcance y las instancias ligadas.

Expresiones `let` en EAB			
Let	Variable ligada	Expresión a ligar	Alcance del let
$\texttt{let}\ x = (2+4)\ \texttt{in}\ x+1\ \texttt{end}$	$\underline{\texttt{x}}$	$2+4$	$\underline{\texttt{x}}+1$
$\texttt{let z} = 7\ \ \texttt{in}\ \ \texttt{y} + \texttt{z}\ \ \texttt{end}$	$\underline{\texttt{z}}$	7	$\texttt{y} + \underline{\texttt{z}}$
$\texttt{let}\ y = (x*2)+x\ \texttt{in}$ $(x*y) + \texttt{let}\ z = 7\ \texttt{in}\ y+z$ $\texttt{end}$	$\underline{\texttt{y}}$	$(x*2)+x$	$(x*\underline{\texttt{y}}) + \texttt{let}\ z = 7\ \texttt{in}\ \underline{\texttt{y}}+z$
$\texttt{let}\ x = \texttt{let}\ x = (2+4)\ \texttt{in}\ x+1\ \texttt{end in}$ $\texttt{let}\ y = (x*2)+x\ \texttt{in}$ $(x*y) + \texttt{let}\ z = 7\ \texttt{in}\ y+z\ \texttt{end}$ $\texttt{end}$ $\texttt{end}$	$\underline{\texttt{x}}$	$\texttt{let x} = (2+4)\ \ \texttt{in}\ \ \texttt{x+1}\ \ \texttt{end}$	$\texttt{let}\ y = (\underline{\texttt{x}}*2)+\underline{\texttt{x}}\ \texttt{in}$ $(\underline{\texttt{x}}*y) + \texttt{let}\ z = 7\ \texttt{in}\ y+z\ \texttt{end}$ $\texttt{end}$

Ejercicio 3.9. Al evaluar la expresión e se puede obtener un valor? si sí muestra el procedimiento para la evaluación informal (aún no definimos la semántica operacional para `let` en EAB, pero se puede esbozar el método de evaluación partiendo de la descripción de instrucción.)

$$
\begin{aligned}
e =_{\text{def}} & \texttt{let}\ x = \texttt{let}\ x = (2+4)\ \texttt{in}\ x+1\ \texttt{end}\ \ \texttt{in} \\
& \quad \texttt{let}\ y = (x*2)+x\ \texttt{in} \\
& \qquad (x*y) + \texttt{let}\ z = 7\ \texttt{in}\ y+z\ \texttt{end} \\
& \quad \texttt{end} \\
& \texttt{end}
\end{aligned}
$$

Primero se resuelve la expresión let que la da valor a la variable x en la expresión e:

$$\texttt{let}\ x = (2+4)\ \texttt{in}\ x+1\ \texttt{end}\ = (2+4)+1 = (6)+1 = 7$$

Se sustituye este valor en el cuerpo de e resultando en la siguiente expresión:

$$
\begin{aligned}
& \texttt{let}\ y = (7*2)+7\ \texttt{in} \\
& \quad (7*y) + \texttt{let}\ z = 7\ \texttt{in}\ y+z\ \texttt{end} \\
& \texttt{end}
\end{aligned}
$$

Resolviendo la operación para asignar un valor a la variable y obtenemos el siguiente resultado:

$$y = (7*2)+7 = (14)+7 = 21$$

Sustituyendo el nuevo valor en el cuerpo de la expresión anterior se obtiene:

$$= (7*21) + \texttt{let}\ z = 7\ \texttt{in}\ 21+z\ \texttt{end}$$

Resolviendo las operaciones y la sustitución de la variable z en el cuerpo de la expresión let se obtiene:

$$= 147+21+7 = 168+7 = 175$$

Ejercicio 3.10. En la expresión

$$
\begin{aligned}
e =_{\text{def}} & \texttt{let}\ x = \texttt{let}\ x = (2+4)\ \texttt{in}\ x+1\ \texttt{end}\ \ \texttt{in} \\
& \quad \texttt{let}\ y = (x*2)+x\ \texttt{in} \\
& \qquad (x*y) + \texttt{let}\ z = 7\ \texttt{in}\ y+z\ \texttt{end} \\
& \quad \texttt{end} \\
& \texttt{end}
\end{aligned}
$$

¿Hay variables libres? argumenta por qué si o por qué no.

No, todas las variables definidas en la expresión e están asociadas a un operador `let` marcando su alcance en las expresiones anidadas presentes por lo que ninguna variable aparece sin su respectiva asociación.

4 Sustitución y α-equivalencias

Definición 4.1 (α-equivalencia)**.** Se dice que dos expresiones e_1 y e_2 son α-equivalentes si y sólo si difieren únicamente en el nombre de las variables ligadas[a].

Esta relación será denotada como:

$$e_1 \equiv_\alpha e_2$$

[a]La notación de la relación de α-equivalencia fue extraída de [1], [5] y [12], la definición formulada a partir de [81] y [89]

Esta definición resulta de utilidad para trabajar con expresiones que necesiten ser renombradas a lo más en sus variables ligadas para poder aplicar la sustitución sobre dicha expresión cuando haya un conflicto ocasionado por los identificadores.

A continuación definimos la función de sustitución aplicada a expresiones pertenecientes a nuestro lenguaje EAB.

Definición 4.2 (Función de sustitución para EAB)**.** Se define la función de sustitución[a] sobre los árboles de sintaxis abstracta de EAB:

$$\begin{array}{lcl}
x[x := e] & = & e \\
z[x := e] & = & z \\
\textit{bool[true]}\,[z := e] & = & \textit{bool[true]} \\
\textit{bool[false]}\,[z := e] & = & \textit{bool[false]} \\
num[n][x := e] & = & num[n] \\
sum(a_1, a_2)[x := e] & = & sum(a_1[x := e], a_2[x := e]) \\
res(a_1, a_2)[x := e] & = & res(a_1[x := e], a_2[x := e]) \\
prod(a_1, a_2)[x := e] & = & prod(a_1[x := e], a_2[x := e]) \\
let(a_1, a_2)[x := e] & = & let(a_1[x := e], a_2[x := e]) \\
if(a_1, a_2, a_3)[x := e] & = & if(a_1[x := e], a_2[x := e], a_3[x := e]) \\
(z.a)[x := e] & = & z.(a[x := e]) \text{ Si } x \neq z \text{ y } z \notin FV(e) \\
(z.a)[x := e] & = & \texttt{indefinido} \text{ Si } z \in FV(e)^b
\end{array}$$

[a]Esta definición está formulada a partir de [1], [2], [5] y [12] adaptada para las instrucciones que hemos revisado hasta este momento para EAB.

[b]Cuando nos encontramos en este caso, podemos aplicar la α-equivalencia para obtener otra expresión equivalente y aplicar la sustitución sobre esta.

Ejercicio 4.1. Proporciona una expresión α-equivalente a la siguiente expresión:

$$
\begin{array}{l}
\texttt{let}\ x = \texttt{let}\ y = \texttt{false in} \\
\qquad\qquad \texttt{if}\ y\ \texttt{then}\ (1)\ \texttt{else}\ 0 \\
\qquad\quad \texttt{end} \\
\quad \texttt{in} \\
\qquad (x+2) * w \\
\texttt{end}
\end{array}
$$

La α-equivalencia se aplica únicamente sobre las variables ligadas, resultando en la siguiente expresión:

$$
\begin{array}{l}
\texttt{let}\ k = \texttt{let}\ j = \texttt{false in} \\
\qquad\qquad \texttt{if}\ j\ \texttt{then}\ (1)\ \texttt{else}\ 0 \\
\qquad\quad \texttt{end} \\
\texttt{in} \\
\qquad (k+2) * w \\
\texttt{end}
\end{array}
$$

Ejercicio 4.2. Proporciona una expresión α-equivalente a la expresión e definida en el ejercicio 3.7.

$$
\begin{array}{l}
e_\alpha = \ \texttt{let}\ a = \texttt{let}\ d = (2+4)\ \texttt{in}\ d+1\ \texttt{end}\ \ \texttt{in} \\
\qquad\qquad \texttt{let}\ b = (a*2) + a\ \texttt{in} \\
\qquad\qquad\qquad (a*b) + \texttt{let}\ c = 7\ \texttt{in}\ b+c\ \texttt{end} \\
\qquad\qquad \texttt{end} \\
\quad \texttt{end}
\end{array}
$$

Ejercicio 4.3. Desarrolla la sustitución aplicada a la siguiente expresión de acuerdo con la función de sustitución previamente definida.

$$(\texttt{let}\ x = (w+1) * 2\ \texttt{in if true}\ \ \texttt{then}\ w\ \texttt{else}\ (2 * x)\ \texttt{end}\ )[w := 2]$$

$$let(prod(sum(w,\ num[1]), num[2]), x.if(bool[true],\ w,\ \ prod(2,\ x)))\ [w := num[2]]$$

$$let(prod(sum(w,\ num[1]), num[2]), x.if(bool[true],\ w,\ \ prod(2,\ x))[w := num[2]])$$

$$let(prod(sum(w,\ num[1])[w\ :=\ num[2]], num[2]\ [w\ :=\ num[2]]), x.if(bool[true][w\ :=\ num[2]], w[w\ :=\ num[2]],\ prod(num[2],\ x)[w\ :=\ num[2]]))$$

$$let(prod(sum(w[w\ :=\ num[2]], num[1][w\ :=\ num[2]]), num[2]), x.if(bool[true],\ num[2],\ prod(num[2][w := num[2]],\ x[w\ :=\ num[2]])))$$

$$let(prod(sum(num[2], num[1]), num[2]), x.if(bool[true],\ num[2],\ prod(num[2],\ x)))$$

5 Ejercicios para el lector

Ejercicio 5.1. Explica la diferencia entre la sintaxis abstracta y la sintaxis concreta.

Ejercicio 5.2. Explica por que es necesaria la sintaxis abstracta de orden superior que se introduce con el árbol de sintaxis abstracta $x.t$

Ejercicio 5.3. Proporciona el árbol de sintaxis abstracta para la siguiente expresión `let`

$$\texttt{let}\ z = 777\ \texttt{in}\ (w+h) * (z) + \texttt{let}\ v = -33\ \texttt{in}\ (z * z) + v\ \texttt{end end}$$

Ejercicio 5.4. Proporciona el árbol de sintaxis abstracta para la siguiente expresión `if`

$$\texttt{if true then}\ (\texttt{if false then}\ (0)\ \texttt{else}\ 1)\ \texttt{else}\ (2 * 7) - 1$$

Ejercicio 5.5. La siguiente expresión ¿es una expresión correcta de nuestro lenguaje? Explica por que si o porque no de acuerdo a las reglas de sintaxis concreta definidas.

$$\texttt{if } \top \texttt{ then } \texttt{ let } x = \text{true} \texttt{ in if } x \texttt{ then } \bot \texttt{ end}$$

Ejercicio 5.6. Dada la siguiente expresión `let` definida como:

$$\begin{aligned} e_1 =_{\text{def}} & \texttt{let } x = \texttt{let } x = 2 \texttt{ in let } y = 3 \texttt{ in } y + x \texttt{ end end in} \\ & \quad \texttt{let } y = (x * 2) * 19 \texttt{ in} \\ & \qquad (x + y) * \texttt{let } z = \texttt{ let } v = 8 \texttt{ in } 1 \texttt{ end in } (y + z) * v \texttt{ end} \\ & \quad \texttt{end} \\ & \texttt{end} \end{aligned}$$

Proporciona lo siguiente por cada subexpresión `let` definida en e:

- La variable de ligado
- El alcance de cada variable
- Las variables ligadas
- Las variables libres
- El valor que se obtiene al desarrollar la expresión.

Ejercicio 5.7. Proporciona una expresión α-equivalente a e_1 del ejercicio anterior

Ejercicio 5.8. Desarrolla la siguiente sustitución, en caso de ser inválida explica por qué.

$$(\texttt{let } z = 3 \texttt{ in } (w + z) + \texttt{ if true then } (45) \texttt{ else } y \texttt{ end})[y := 7]$$

Ejercicio 5.9. Desarrolla la siguiente sustitución, en caso de ser inválida explica por qué.

$$(\texttt{let } z = 3 \texttt{ in } (w + z) + \texttt{ if true then } (45) \texttt{ else } y \texttt{ end})[y := z + 7]$$

Ejercicio 5.10. A continuación se define la sintaxis concreta para un lenguaje funcional simple:

$$e ::= \ x \mid n \mid e_1 \ e_2 \mid fun(x) \to e$$

En donde el primer constructor representa las variables del lenguaje, el segundo números naturales, el tercero aplicación de función y el último la definición de funciones.

De acuerdo a la especificación discutida anteriormente, contesta los siguiente incisos:

- Define las reglas de la sintaxis concreta de la gramática.
- Define las reglas de sintaxis abstracta para la gramática (es necesario especificar el alcance de ligado de las variables de la función con un ASA similar a $x.e$).
- Escribe las reglas del análisis sintáctico para la relación $e \longleftrightarrow ASA$ del lenguaje.
- Define la función de sustitución para este lenguaje.

  La relación de α-equivalencia en este lenguaje se define con respecto al operador fun:

  $$fun(x) \to e_1 \equiv_\alpha fun(y) \to e_2$$

  sí y solo sí

  e_1, e_2 difieren únicamente en el nombre de sus variables ligadas

- Demuestra que $\equiv_\alpha$ es una relación de equivalencia.

Capítulo 4

Semántica

En el capítulo anterior, se estudió la composición sintáctica de los lenguajes de programación mediante la definición de juicios lógicos que nos permiten construir expresiones válidas de EAB, definiendo así la sintaxis concreta de este lenguaje. También se estudió la estructura jerárquica de bajo nivel con la que se obtiene la representación única de las expresiones válidas de EAB, conocida como árboles de sintaxis abstracta. Este primer nivel de los lenguajes de programación está enfocado a la representación del lenguaje pero no a darle un significado.

En el presente capítulo nos ocuparemos de la semántica del lenguaje, que tiene como propósito responder a la pregunta ¿qué significa una expresión del lenguaje? y como podemos interpretar aquellas expresiones correctamente formuladas a partir de la sintaxis.

En el contexto de los lenguajes de programación podemos pensar de forma intuitiva que la respuesta al cómo interpretamos las instrucciones del lenguaje puede derivarse de su comportamiento en tiempo de ejecución. En particular nos interesa obtener el valor que la expresión retorna al concluir su evaluación (semántica dinámica) y el tipo de esta (semántica estática).[1].

[1] Definición acuñada de [98], [99] y [100]

Muchas veces en la documentación oficial de los lenguajes de programación la semántica está escrita en un manual el cual contiene una descripción de alto nivel acerca de como una determinada instrucción, método o expresión se comporta al momento de ejecutarse. Si bien esto resulta útil al escribir un programa, nuestro interés sera definir formalmente la semántica de ejecución mediante juicios y reglas para el lenguaje EAB.

Objetivo

Definir los diferentes niveles semánticos de EAB mediante reglas de inferencia que dictaminen el proceso que se debe seguir al evaluar una expresión bien formada de este lenguaje (cuando esto sea posible).

Planteamiento

Iniciaremos el estudio de este capítulo presentando las reglas para dictaminar si una expresión de EAB contiene variables libres. Este nivel corresponde a la semántica estática y será de utilidad para fundamentar la seguridad del lenguaje[2] y garantizar que se pueda regresar un valor al final de la evaluación de la expresión.

Adicionalmente estudiaremos los estados y transiciones de la máquina abstracta para definir la semántica operacional del mismo. Este mecanismo se conoce como semántica dinámica y se estudiará en dos paradigmas: de paso grande y de paso pequeño.

Por último definiremos la función `eval` para EAB.

1 Semántica estática

La semántica estática extrae información del programa en tiempo de compilación, la cantidad y calidad de la información obtenida depende de la implementación de cada compilador[3].

La información que se extrae del programa provisto por el usuario puede ser: el alcance de una variable, el tipo de una expresión, saber si una expresión tiene variables libres, etc. Saber esto es indispensable para poder evaluar las expresiones de EAB (este último punto es importante ya que necesitamos que todas las variables estén ligadas y no haya presencia de variables libres en nuestras expresiones, de lo contrario de la evaluación no se obtendrá valor alguno).

En el lenguaje de expresiones aritméticas con el que hemos estado trabajando, las expresiones de la forma:

$$\texttt{let}\ x = y\ \texttt{in}\ x + x\ \texttt{end}$$

son sintácticamente correctas pero semánticamente incorrectas, pues la variable y está libre

[2] Vamos a estudiar a profundidad esta y otras propiedades relacionadas con la ejecución de los programas en los siguientes capítulos.

[3] Definición acuñada de [96] y [97]

en la expresión por lo que no podría evaluarse.

Esta es una de las propiedades de las que se encarga la semántica estática, así que daremos un conjunto de reglas para definir una que se encargue de evitar estos errores[4]. Para esto se define el siguiente juicio:

Definición 1.1 (Semántica estática para capturar expresiones con variables libres en EAB)**.** Sea Δ un conjunto de variables y a un ASA, decimos que a no tiene variables libres bajo el conjunto Δ denotado como[a]:

$$\Delta \sim a \ ASA$$

En particular nos interesa que todas las variables ligadas de a estén presentes en Δ. Para construir este conjunto definimos las siguientes reglas que nos permitirán acrecentar Δ cada que encontremos una variable ligada en el árbol $x.t$:

$$\frac{x \in \Delta}{\Delta \sim \ x}\ \textit{(fvv)} \qquad \frac{}{\Delta \sim num[n]}\ \textit{(fvn)} \qquad \frac{\Delta, x \sim e}{\Delta \sim \ x.e}\ \textit{(fva)}$$

$$\frac{}{\Delta \sim \textit{bool[b]}}\ \textit{(fvb)} \qquad \frac{\Delta \sim e_1 \quad \cdots \quad \Delta \sim e_n}{\Delta \sim O(e_1, \cdots, e_n)}\ \textit{(fvo)}$$

Para garantizar que una expresión e no tiene variables libres se inicia con el conjunto vacío y se debe probar $\emptyset \sim e$ usando las reglas anteriores.

Nuevamente hacemos abuso de la notación para denotar los operadores de EAB, representados como $O(e_1, \cdots, e_n)$.

[a] Definición acuñada de [5] y [12]

Ejercicio 1.1. Para la siguiente expresión, aplica las reglas de la semántica estática definida para la relación $\sim$ y determina la presencia de variables libres.

$$\texttt{let}\ x = 1\ \texttt{in}\ (y * x) + z\ \texttt{end}$$

$$\frac{\frac{}{\emptyset \sim 1}\ \textit{(fvn)} \qquad \frac{\frac{\frac{\frac{Error}{\{x\} \sim y} \quad \frac{x \in \{x\}}{\{x\} \sim x}\ \textit{(fvv)}}{\{x\} \sim \textit{prod(x,y)}}\ \textit{(fvo)} \quad \frac{Error}{\{x\} \sim z}}{\{x\} \sim \textit{sum(prod(y, x), z)}}\ \textit{(fvo)}}{\emptyset \sim \textit{x.sum(prod(y, x), z))}}\ \textit{(fva)}}{\emptyset \sim \textit{let(1,x.sum(prod(y,x),z))}}\ \textit{(fvo)}$$

En este caso dos de las ramas en el árbol de derivación concluyeron en un error, pues si una variable es libre el juicio $\Delta \ \sim \ e$ no es válido. Concluimos entonces que la expresión no es correcta.

[4] Fragmento extraído de: [12]

Ejercicio 1.2. Para la siguiente expresión realiza el análisis estático para encontrar variables libres mediante la derivación aplicando las reglas del juicio $\sim$

$$\texttt{if true then } ((1+1)*2) \texttt{ else } 7$$

$$\dfrac{\dfrac{}{\emptyset \sim \textit{bool[true]}}\,(fvb) \quad \dfrac{\dfrac{\dfrac{}{\emptyset \sim 1}\,(fvn) \quad \dfrac{}{\emptyset \sim 1}\,(fvn)}{\emptyset \sim \textit{sum(1, 1)}}\,(fvo) \quad \dfrac{}{\emptyset \sim 2}\,(fvn)}{\emptyset \sim \textit{prod(sum(1, 1), 2)}}\,(fvo) \quad \dfrac{}{\emptyset \sim 7}\,(fvn)}{\emptyset \sim \textit{if(bool[true], prod(sum(1, 1), 2), 7)}}\,(fvo)$$

Ejercicio 1.3. Para la siguiente expresión realiza el análisis estático para encontrar variables libres mediante la derivación aplicando las reglas del juicio $\sim$

$$1 + \texttt{if let } x = \texttt{true in } x \texttt{ end then } (2) \texttt{ else } 1$$

$$\dfrac{\dfrac{}{\emptyset \sim 1}\,(fvn) \quad \dfrac{\dfrac{}{\emptyset \sim \textit{bool}[false]}\,(fvb) \quad \dfrac{\dfrac{}{\{x\} \sim x}\,(fvv)}{\emptyset \sim x.x}\,(fva)}{\emptyset \sim \textit{let(bool[false],x.x)}}\,(fvo) \quad \dfrac{}{\emptyset \sim 2}\,(fvn) \quad \dfrac{}{\emptyset \sim 1}\,(fvn)}{\emptyset \sim \textit{if(let(bool[false],x.x)},2,1)}\,(fvo)}{\emptyset \sim \textit{sum(1,if(let(bool[false],x.x)},2,1))}\,(fvo)$$

Definición 1.2 (Expresión cerrada de EAB)**.** Sea a un ASA decimos que a es cerrada si y sólo si

$$FV(a) \;=\; \emptyset$$

Es decir, a es una expresión del lenguaje cerrada si no tiene apariciones de variables libres[a].

[a] Definición acuñada de [5] y [12]

De los ejercicios anteriores podemos notar que el análisis sintáctico para evaluar expresiones con variables libres fallará en una o más ramas, mientras que los árboles de derivación generados de expresiones cerradas concluirán todas sus ramas con algún axioma (*fvn, fvb, o fvv*).

2 Semántica dinámica

La semántica dinámica constituye el siguiente eslabón del proceso de ejecución de un programa. Una vez verificada que la expresión en sintaxis concreta es una expresión válida y no posee variables libres, entonces se puede comenzar a discutir el como se ejecutará dicha expresión.

Existen diferentes paradigmas de ejecución que pueden ser aplicados para evaluar las expresiones de los lenguajes de programación según el interés de estudio (por ejemplo cambios en la memoria, el valor que contienen las variables del programa, el valor final al que evalúa una expresión, etc.)

En este capítulo nos centraremos en la semántica operacional que estudia el proceso de ejecución modelando cada configuración del programa como un estado y las transiciones entre estos como los pasos en la evaluación.

2.1 Semántica operacional

En esta categoría se hace la distinción entre los dos paradigmas mas importantes para estudiar la semántica operacional de los lenguajes de programación[5]:

- **Semántica de paso pequeño:** la cuál modela la ejecución de un programa describiendo las transiciones una a una mostrando los cómputos generados de forma individual.
- **Semántica de paso grande:** que contrasta con la de paso pequeño porque aquí no nos importa que estados sucedieron en la ejecución del programa, únicamente nos interesa el valor que regresa.

Definición 2.1 (Sistema de transición para la semántica operacional de EAB)**.** [a]:

Conjunto de estados $S = \{a \mid a\ ASA\}$
Los estados del sistema son las expresiones del lenguaje en sintaxis abstracta. Esta definición corresponde a la regla de inferencia:

$$\frac{a\ ASA}{a\ estado}\ \textit{(state)}$$

Estados Iniciales $I = \{a \mid a\ ASA,\ FV(a) = \emptyset\}$
Los estados iniciales son todas las expresiones cerradas del lenguaje y corresponde a la regla:

$$\frac{a\ ASA \quad \emptyset \sim a}{a\ inicial}\ \textit{(init)}$$

Estados Finales Este conjunto de estados finales está compuesto únicamente por los valores de EAB que formalizamos a continuación[b].

Definimos una categoría de valores los cuales son un subconjunto de expresiones de EAB. Esta nueva categoría se representa con el juicio $v\ valor$.

Para el caso de EAB se denota con las reglas:

$$\frac{}{num[n]\ valor}\ \textit{(vnum)} \qquad \frac{}{bool[b]\ \ valor}\ \textit{(vbool)}$$

Entonces se define el conjunto de estados finales $F = \{a \mid a\ valor\}$, correspondiente a la regla:

$$\frac{a\ valor}{a\ final}\ \textit{(fin)}$$

Transiciones Para definir la semántica operacional es necesario puntualizar la constitución de un sistema de transición. La semántica de paso pequeño y de paso grande comparten la definición de los estados difiriendo únicamente en la definición de la función de transición, esta será enunciada a continuación para cada enfoque.

[a] Definición acuñada de [5] y [12] y [105]
[b] Los estados que no se pueden reducir más pero que no corresponden a un valor (es decir expresiones mal formadas) se les conoce como estados bloqueados.

Definición 2.2. Estado bloqueado[a]: Un estado s está bloqueado si no existe otro estado s' tal que $s \to s'$ y lo denotamos como $s \nrightarrow$

[5] Definición acuñada de y [2], [103] y [104]

[a]Definición acuñada de [5] y [12]

2.2 Semántica de paso pequeño

Para esta semántica las transiciones se modelarán paso a paso mediante la función de transición, denotada como:

$$e_1 \to e_2$$

Donde e_1 es llamado "*redex*" y e_2 es llamado "*reducto*". Esta relación se interpreta cómo la transición entre e_1 y e_2 que existe si y solo sí en un paso de evaluación se puede reducir la expresión e_1 a la expresión e_2[6].

Para definir las reglas de transición para los cómputos siguiendo el paradigma de semántica de paso pequeño, primero vamos a evaluar el argumento más a la izquierda dentro de las expresiones que denotan un operador hasta obtener un valor. Posteriormente evaluaremos el argumento a la derecha

Definición 2.3 (Función de transición para semántica de paso pequeño)**.** [a]

Suma

$$\frac{}{sum(num[n], num[m]) \to num[n +_N m]} \; (sum_N)$$

$$\frac{e_1 \to e_1'}{sum(e_1, e_2) \to sum(e_1', e_2)} \; (sum_L) \qquad \frac{e_2 \to e_2'}{sum(num[n], e_2) \to sum(num[n], e_2')} \; (sum_R)$$

Resta

$$\frac{}{res(num[n], num[m]) \to num[n -_N m]} \; (res_N)$$

$$\frac{e_1 \to e_1'}{res(e_1, e_2) \to res(e_1', e_2)} \; (res_L) \qquad \frac{e_2 \to e_2'}{res(num[n], e_2) \to res(num[n], e_2')} \; (res_R)$$

Producto

$$\frac{}{prod(num[n], num[m]) \to num[n \times_N m]} \; (prod_N)$$

$$\frac{e_1 \to e_1'}{prod(e_1, e_2) \to prod(e_1', e_2)} \; (prod_L) \qquad \frac{e_2 \to e_2'}{prod(num[n], e_2) \to prod(num[n], e_2')} \; (prod_R)$$

Expresiones lógicas

$$\frac{}{if(bool[true], e_1, e_2) \to e_1} \; (if_T) \qquad \frac{}{if(bool[false], e_1, e_2) \to e_2} \; (if_F)$$

$$\frac{e_1 \to e_1'}{if(e_1, e_1, e_2) \to if(e_1', e_1, e_2)} \; (if_B)$$

[6]Definición acuñada de [5] y [12]

Asignaciones locales

$$\frac{v\ valor}{let(v, x.e_2) \to e_2[x := v]}\ (let_V) \qquad \frac{e_1 \to e_1'}{let(e_1, x.e_2) \to let(e_1', x.e_2)}\ (let_L)$$

La evaluación del operador `if` es una evaluación "perezosa", en donde no se evaluará el cuerpo hasta antes haber determinado el valor de la sentencia de control (`true` ó `false`)

[a]Definición acuñada de [2], [5], [12] y [105].

Ejercicio 2.1. Dada la siguiente expresión de nuestro lenguaje EAB utiliza las reglas de transición de la semántica de paso pequeño para evaluarla.

$$\texttt{let}\ k = (3+1)\ \texttt{in}\ (7 * k) + 1\ \texttt{end}$$

$$\begin{array}{ll} & let(sum(3,1), k.sum(prod(7,k),1)) \\ \to & let(4, k.sum(prod(7,k),1)) \\ \to & (sum(prod(7,k),1))[k := 4] \\ \to & sum(prod(7,k)[k := 4], 1[k := 4]) \\ \to & sum(prod(7[\mathrm{k} := 4], k[k := 4]), 1) \\ \to & sum(prod(7,4),1) \\ \to & sum(28,1) \\ \to & 29 \end{array}$$

Ejercicio 2.2. Dada la siguiente expresión de nuestro lenguaje EAB utiliza las reglas de transición de la semántica de paso pequeño para evaluarla.

$$\texttt{if false then}\ (4 * \texttt{let}\ x = 99 * 99\ \texttt{in}\ x + x\ \texttt{end})\ \texttt{else}\ 0$$

$$\begin{array}{ll} & if(bool[false],\ let(prod(99,99),\ x.sum(x,x)),\ 0) \\ \to & 0 \end{array}$$

Del ejercicio anterior podemos notar la ventaja de la evaluación perezosa. Dado que nuestro valor de control es `false` no es necesario evaluar la expresión e_1 y directamente nos saltaremos a evaluar e_2 por la regla if_F que en este caso es el valor 0.

Ejercicio 2.3. Dada la siguiente expresión de nuestro lenguaje EAB utiliza las reglas de transición de la semántica de paso pequeño para evaluarla.

$$\texttt{let}\ x = \ \texttt{if true then}\ (42)\ \texttt{else}\ 0\ \texttt{in if false then}\ (41)\ \texttt{else}\ x$$

$$\begin{array}{ll} & let(if(bool[true], 42, 0), x.if(F, 41, x)) \\ \to & let(42, x.if(F, 41, x)) \\ \to & if(bool[false], 41, x)[x := 42] \\ \to & if(bool[false][x := 42], 41[x := 42], x[x := 42]) \\ \to & if(bool[false], 41, 42) \\ \to & 42 \end{array}$$

La relación de transición ($\to$) define tres categorías importantes de aplicación las cuales se conocen como "cerraduras" y pueden ser:

- **reflexiva:** un estado puede llegar a si mismo en 0 aplicaciones de pasos.
- **transitiva:** si un estado e_1 puede alcanzar un estado e_2 y e_2 puede alcanzar a e_n en un número finito de pasos entonces e_1 puede llegar a e_n en un número finito de pasos.
- **positiva:** la aplicación de las reglas de transición n veces con n $\geq$ 1.

A continuación enunciamos las caracterización de cada cerradura mediante la aplicación de la relación de transición y la iteración en n pasos.

Definición 2.4 (Definición de cerradura transitiva y reflexiva)**.** Denotamos la aplicación de la función de transición n-veces como: $\rightarrow^*$ que define las siguientes relaciones[a]:

$$\frac{}{s \rightarrow^* s}\ (refl) \qquad \frac{s_1 \rightarrow s_2 \quad s_2 \rightarrow^* s_3}{s_1 \rightarrow^* s_3}\ (trans)$$

La relación $s_1 \rightarrow^* s_2$ modela que es posible llegar desde s_1 hasta s_2 en un número finito de aplicaciones (posiblemente 0) de la relación de transición $\rightarrow$.

[a]Definición acuñada de [2], [5] y [12].

Definición 2.5 (Cerradura positiva)**.** Esta cerradura se denota como $\rightarrow^+$ y se define con las siguientes reglas[a]:

$$\frac{s_1 \rightarrow s_2}{s_1 \rightarrow^+ s_2}\ (one+) \qquad \frac{s_1 \rightarrow s_2 \quad s_2 \rightarrow^+ s_3}{s_1 \rightarrow^+ s_3}\ (trans+)$$

La relación $s_1 \rightarrow^+ s_2$ modela que es posible llegar desde s_1 hasta s_2 en un número finito de aplicaciones estrictamente mayor a cero de la relación de transición $\rightarrow$. Es decir, se llega de s_1 a s_2 en al menos un paso.

[a]Definición acuñada de [2], [5] y [12]

Definición 2.6 (Iteración en n pasos)**.** Se denota como $\rightarrow^n$ con $n \in N$ y se define con las siguientes reglas[a]:

$$\frac{}{s \rightarrow^0 s}\ (iter_z) \qquad \frac{s_1 \rightarrow s_2 \quad s_2 \rightarrow^n s_3}{s_1 \rightarrow^{n+1} s_3}\ (iter_n)$$

La relación $s_1 \rightarrow^n s_2$ modela que es posible llegar desde s_1 hasta s_2 en exactamente n aplicaciones de la relación de transición $\rightarrow$.

[a]Definición acuñada de [2], [5] y [12]

2.3 Semántica de paso grande

Este paradigma para definir la semántica dinámica encapsula de forma general el proceso de evaluación sin mostrar de forma explícita paso a paso como es que una expresión es evaluada (contrario a la semántica de paso pequeño).

La relación de transición en este caso se denota como $e \Downarrow v$ donde e es una expresión válida de nuestro lenguaje EAB y v es un valor. Esta se lee como "la expresión e se evalúa a v".

Definición 2.7 (Definición de semántica de paso grande para EAB)**.** Se define la relación de transición $\Downarrow$ mediante las siguientes reglas de inferencia[a]:

Valores

$$\frac{}{num[n] \Downarrow num[n]}\ (bsnum) \qquad \frac{}{bool[b] \Downarrow bool[b]}\ (bsbool)$$

Suma

$$\frac{e_1 \Downarrow num[n] \qquad e_2 \Downarrow num[m]}{sum(e_1, e_2) \Downarrow num[n +_N m]}\ (bssum)$$

Resta

$$\frac{e_1 \Downarrow num[n] \qquad e_2 \Downarrow num[m]}{res(e_1, e_2) \Downarrow num[n -_N m]}\ (bssum)$$

Producto

$$\frac{e_1 \Downarrow num[n] \qquad e_2 \Downarrow num[m]}{prod(e_1, e_2) \Downarrow num[n \times_N m]}\ (bsprod)$$

Asignaciones locales

$$\frac{e_1 \Downarrow v_1 \qquad e_2[x := v_1] \Downarrow v_2}{let(e_1, x.e_2) \Downarrow v_2}\ (bslet)$$

Sentencias de control

$$\frac{e_0 \Downarrow T \qquad e_1 \Downarrow v_1}{if(e_0, e_1, e_2) \Downarrow v_1}\ (bsift) \frac{e_0 \Downarrow F \qquad e_2 \Downarrow v_2}{if(e_0, e_1, e_2) \Downarrow v_2}\ (bsift)$$

Es importante destacar que los valores de tipo $num[n]$. T, F si tienen reglas definidas en esta semántica de paso grande.

[a] Definición formulada a partir de [2], [5] y [12]

Es importante notar que en estos ejercicios estamos haciendo abuso de la notación dado que las reglas para la función de transición de paso grande $\Downarrow$ están definidas para la sintaxis abstracta pero permitimos el uso de la sintaxis concreta para simplificar la notación.

Ejercicio 2.4. Dada la siguiente expresión de EAB evalúa utilizando la semántica de paso grande. Adicionalmente proporciona la representación en sintaxis abstracta.

$$\texttt{if false then } ((3 * 7) + 1) \texttt{ else } (2 * 7) + 1$$

Sintaxis abstracta:

$$\textit{if(bool[false]}, sum(prod(3, 7), 1), sum(prod(2, 7), 1))$$

Evaluación paso grande:

$$\frac{\frac{}{bool[false] \Downarrow bool[false]}\ (bsbool) \qquad \frac{\frac{\frac{}{2 \Downarrow 2}\ (bsnum) \qquad \frac{}{7 \Downarrow 7}\ (bsnum)}{prod(2, 7) \Downarrow 14}\ (bsprod) \qquad \frac{}{1 \Downarrow 1}\ (bsnum)}{sum(prod(2, 7), 1)\ ) \Downarrow 15}\ (bssum)}{\textit{if(bool[false]}, sum(prod(3, 7), 1), sum(prod(2, 7), 1)\ ) \Downarrow 15}\ (bsiff)$$

Ejercicio 2.5. Dada la siguiente expresión de EAB evalúa utilizando semántica de paso grande. Adicionalmente proporciona la representación en sintaxis abstracta.

$$\texttt{let } x = \texttt{let } y \texttt{ = false in if } y \texttt{ then } (0) \texttt{ else } 1 \texttt{ end in } x + 1 \texttt{ end}$$

Sintaxis abstracta:

$$let(let(bool[false], y.if(y, 0, 1)), x.sum(x, 1))$$

Evaluación paso grande:

$$\dfrac{\dfrac{}{bool[false] \Downarrow bool[false]}\,(bsbool) \qquad \dfrac{\dfrac{\dfrac{}{bool[false] \Downarrow bool[false]}\,(bsbool) \quad \dfrac{}{1 \Downarrow 1}\,(bsnum)}{if(bool[false], 0, 1)) \Downarrow 1}\,(bsift)}{if(y, 0, 1))[y := bool[false]] \Downarrow 1}\,(bslet) \qquad \dfrac{\dfrac{\dfrac{}{1 \Downarrow 1}\,(bsnum) \quad \dfrac{}{1 \Downarrow 1}\,(bsnum)}{sum(1, 1) \Downarrow 2}\,(bssum)}{sum(x, 1)[x := 1] \Downarrow 2}\,(bslet)}{let(bool[false], y.if(y, 0, 1)), x.sum(x, 1)) \Downarrow 2}$$

Ejercicio 2.6. Dada la siguiente expresión de EAB evalúa utilizando la semántica de paso grande. Adicionalmente proporciona la representación como árbol de sintaxis abstracta.

```
((7 + 4) * 4) + ((8 + 3) * 2)
```

Sintaxis abstracta:

$$sum(prod(sum(7,4),4), prod(sum(8,3),2))$$

Evaluación paso grande:

$$\dfrac{\dfrac{\dfrac{\dfrac{}{7 \Downarrow 7}\,(bsnum) \quad \dfrac{}{4 \Downarrow 4}\,\text{bsnum}}{sum(7,4) \Downarrow 11}\,(bsum) \quad \dfrac{}{4 \Downarrow 4}\,\text{bsnum}}{prod(sum(7,4),4) \Downarrow 44}\,(bsprod) \quad \dfrac{\dfrac{\dfrac{}{8 \Downarrow 8}\,(bsnum) \quad \dfrac{}{3 \Downarrow 3}\,\text{bsnum}}{sum(8,3) \Downarrow 11}\,(bsum) \quad \dfrac{}{2 \Downarrow 2}\,(bsnum)}{prod(sum(8,3),2) \Downarrow 22}\,(bsprod)}{sum(prod(sum(7,4),4), prod(sum(8,3),2)) \Downarrow 66}\,(bssum)$$

Teorema 2.8 (Equivalencia entre semántica de paso pequeño y paso grande)**.** Decimos que para cualquier expresión e del lenguaje EAB se cumple:

$$e \to^* v \longleftrightarrow e \Downarrow v$$

Es decir, las semánticas que hemos definido son equivalentes[a].

[a] La demostración se puede hacer por inducción estructural sobre cada instrucción de EAB, esta queda fuera del alcance del presente manual de carácter práctico pero si se desea consultarla esta está disponible en [106]

3 La función eval

Concluimos este capítulo con la definición de la función de evaluación para EAB, la cuál nos ayuda a vincular ambos paradigmas tratados para la semántica dinámica mediante la siguiente especificación.

Definición 3.1 (Función de evaluación para EAB)**.** Se define la función `eval` en términos de la semántica dinámica del lenguaje como sigue[a]:

$$\texttt{eval}(e) = e_f \text{ si y sólo si } e \to^* e_f \text{ y } e_f \nrightarrow$$

La función de evaluación aplica de forma exhaustiva la semántica dinámica hasta llegar a un estado final (en el caso de EAB, un valor).

[a] Definición formulada a partir de [2], [5] y [12]

Ejercicio 3.1. Dada la siguiente expresión de nuestro lenguaje EAB utiliza las reglas de transición de la semántica de paso pequeño para evaluarla.

$$\texttt{let } y = (5-2) \texttt{ in if true then } (y * 3) \texttt{ else } 0 \texttt{ end}$$

$$\begin{aligned}
& \mathit{let}(\mathit{res}(5,2),\ y.\mathit{if}(\mathit{bool}[\mathit{true}],\ \mathit{prod}(y,3),\ 0)) \\
\to\ & \mathit{let}(3,\ y.\mathit{if}(\mathit{bool}[\mathit{true}],\ \mathit{prod}(y,3),\ 0)) \\
\to\ & \mathit{if}(\mathit{bool}[\mathit{true}],\ \mathit{prod}(y,3),\ 0)[y := 3] \\
\to\ & \mathit{if}(\mathit{bool}[\mathit{true}][y := 3],\ \mathit{prod}(y,3)[y := 3],\ 0[y := 3]) \\
\to\ & \mathit{if}(\mathit{bool}[\mathit{true}],\ \mathit{prod}(3,3),\ 0) \\
\to\ & \mathit{prod}(3,3) \\
\to\ & 9
\end{aligned}$$

Por lo tanto,

$$\texttt{eval}(\texttt{let } y = (5-2) \texttt{ in if true then } (y * 3) \texttt{ else } 0) = 9$$

Ejercicio 3.2. Dada la siguiente expresión de nuestro lenguaje EAB utiliza las reglas de transición de la semántica de paso pequeño para evaluarla.

$$\texttt{if } ((2+3)-5) \texttt{ then } 1 \texttt{ else let } z = 4+1 \texttt{ in } z * z \texttt{ end}$$

$$\begin{array}{rl} & \mathit{if}(\mathit{res}(\mathit{sum}(2,3),5),\ 1,\ \mathit{let}(\mathit{sum}(4,1),\ z.\mathit{prod}(z,z))) \\ \rightarrow & \mathit{if}(\mathit{res}(5,5),\ 1,\ \mathit{let}(\mathit{sum}(4,1),\ z.\mathit{prod}(z,z))) \\ \rightarrow & \mathit{if}(0,\ 1,\ \mathit{let}(\mathit{sum}(4,1),\ z.\mathit{prod}(z,z))) \end{array}$$

En este punto la evaluación se bloquea, ya que 0 no es un valor booleano (T o F), por lo que no es posible aplicar ni la regla (if_T) ni (if_F).
Por lo tanto,

$$\texttt{eval}(e) \text{ no está definida}$$

4 Ejercicios para el lector

Ejercicio 4.1. Dada la siguiente expresión de EAB:

```
let x = 5 * 2 in x - 5 end
```

Contesta lo siguiente:

- Proporciona la representación en sintaxis abstracta.
- Aplica la semántica estática para inferir la presencia de variables libres con el juicio $\sim$.
- Aplica la semántica dinámica de paso pequeño para evaluar la expresión.
- Aplica la semántica dinámica de paso grande para evaluar la expresión.

Ejercicio 4.2. Dada la siguiente expresión de EAB:

```
let y = false in if y then (let z = 1 in 11 - z end) else x end
```

Contesta lo siguiente:

- Proporciona la representación en sintaxis abstracta.
- Aplica la semántica estática para inferir la presencia de variables libres con el juicio $\sim$.
- Aplica la semántica dinámica de paso pequeño para evaluar la expresión.
- Aplica la semántica dinámica de paso grande para evaluar la expresión.

Ejercicio 4.3. Considera la siguiente sintaxis concreta para un lenguaje proposicional simple `Prop`[a] donde solo se utiliza el conector `AND` ($\wedge$) y el operador `NOT` ($\neg$) definida como:

$$\frac{x \ \texttt{Prop} \quad y \ \texttt{Prop}}{x \wedge y \ \texttt{Prop}} \qquad \frac{x \ \texttt{Prop}}{\neg x \ \texttt{Prop}} \qquad \frac{}{\top \ \texttt{Prop}} \qquad \frac{}{\bot \ \texttt{Prop}}$$

- Proporciona una semántica de paso pequeño para evaluar las expresiones en `Prop`.
- Proporciona una semántica de paso grande para evaluar las expresiones `Prop`.

[a] Ejercicio extraído de [107].

Ejercicio 4.4. Ahora supón que se quieren añadir cuantificadores y variables a nuestro lenguaje `Prop`[a] de la siguiente forma:

$$\frac{x\ \texttt{Prop}}{\exists v.\ x\ \texttt{Prop}} \qquad \frac{x\ \texttt{Prop}}{\forall v.\ x\ \texttt{Prop}} \qquad \frac{v\ variable}{v\ \texttt{Prop}}$$

- Proporciona un conjunto de reglas que definan la semántica estática que nos permite decidir cuando una expresión de `Prop` no contiene variables libres bajo un contexto Γ. Denotado de la siguiente forma:
  $\Gamma \vdash e\ Ok$

[a]Ejercicio extraído de [107].

Ejercicio 4.5. Una calculadora de Notación Polaca Reversa (`NPR`)[a] es una calculadora que no requiere de parentizado para evaluar las expresiones que son pasadas como argumento.
Esta se apoya de una pila para "empujar" los operandos y los operadores así como de la notación pos-fija, es decir el operador se escribe después de los operandos, por ejemplo:

$$7\ +\ 1 \text{ se representa como: } 7\ 1\ +$$

$$7\ -\ (3\ +\ 2) \text{ se representa como: } 7\ 3\ 2\ +\ -$$

Esta calculadora empuja símbolos a la pila hasta encontrar un operador, en tal caso, los dos símbolos más próximos al tope son extraídos de la pila y el resultado de la operación es empujado nuevamente.

La sintaxis concreta de la calculadora está definida por las siguientes reglas:

$$\frac{x \in N}{x\ Symbol} \qquad \frac{x \in \{+, -, *, /\}}{x\ Symbol} \qquad \frac{}{\epsilon\ \texttt{NPR}} \qquad \frac{x\ Symbol \quad xs\ \texttt{NPR}}{x\ xs\ \texttt{NPR}}$$

Esta gramática tiene el problema de poder formar expresiones que no pertenecen necesariamente a `NPR` como:

$$1 + 2$$

$$+ * /$$

- Proporciona un conjunto de reglas para definir la semántica estática que pueda analizar una expresión e y nos diga si una expresión perteneciente a `NPR` está bien formada, denotando el juicio como: $\vdash e\ Ok$.
- Proporciona un conjunto de reglas para definir la semántica dinámica de paso pequeño que evalúen las expresiones de `NPR`.
- Proporciona un conjunto de reglas para definir la semántica dinámica de paso grande que evalúen las expresiones de `NPR`.

[a]Ejercicio extraído de [107].

Ejercicio 4.6. Utilizando el analizador sintáctico para `NPR` y las reglas de semántica dinámica de paso pequeño y de paso grande definidas para `NPR` contesta lo siguiente:

Dada la expresión

$$7\ 1\ 9\ -\ -$$

- Aplica la semántica estática definida con el juicio $\vdash e\ Ok$
- Evalúa la expresión utilizando la semántica de paso pequeño.
- Evalúa la expresión utilizando la semántica de paso grande.

Ejercicio 4.7. Utilizando el analizador sintáctico para `NPR` y las reglas de semántica dinámica de paso pequeño y de paso grande definidas para `NPR` contesta lo siguiente:

Dada la expresión

$$3\ 2\ 4\ 1\ *+-$$

- Aplica la semántica estática definida con el juicio $\vdash e\ Ok$
- Evalúa la expresión utilizando la semántica de paso pequeño.
- Evalúa la expresión utilizando la semántica de paso grande.

Capítulo 5

Cálculo Lambda

Cuando pensamos en las bases que sentaron los modelos de cómputo modernos podemos inmediatamente asociarlo con las máquinas de Turing, un sistema de transición que se ha estudiado con anterioridad en materias como autómatas y lenguajes formales[1] siendo el más reconocido cuando se habla de conceptos como: "computabilidad", "algoritmo" o "complejidad", términos apenas formalizados en 1936.

En este curso estudiaremos un modelo equivalente de cómputo, es aquí donde Alonzo Church nos introduce en 1928 a su aproximación de un sistema formal basado en el concepto de "función" con las primitivas de "abstracción" y "aplicación" que fungiría como fundamento lógico para sustituir la teoría de conjuntos de Zermelo-Fraenkel y la teoría de tipos de Russell. Este

[1] Conforme al plan de estudios que se imparte desde el 2013 en la Facultad de Ciencias de la Universidad Nacional Autónoma de México con clave de asignatura 1425.

sistema fue demostrado como inconsistente por sus dos alumnos Stephen Kleene y Barkley Rosser pero no todo sería desechado, conservando la parte del manejo de funciones que se demostró equivalente al modelo de Alan Turing. A este sistema se le conoce como Cálculo Lambda[2].

Objetivo

Revisar la composición del Cálculo Lambda mediante el estudio de la sintaxis, semántica operacional y la representación de tipos primitivos en este modelo de cómputo, así como las propiedades inherentes a la semántica del cálculo junto con los sistemas de recursión para la evaluación de λ-expresiones.

Planteamiento

En este capítulo se revisará la sintaxis concreta del Cálculo Lambda para generar λ-expresiones junto con la semántica operacional del cálculo, basándonos en la operación de sustitución sintáctica ([variable := valor]) conocida como β-reducción.

También se abordará la definición de los booleanos junto con los operadores lógicos, estructuras de datos (tuplas, proyecciones, listas) y los numerales de Church con operadores aritméticos, mismos que se espera el lector pueda manipular, razonar y resolver en la sección de ejercicios.

Finalmente se introducirá el sistema de recursión para el Cálculo Lambda con los operadores de punto fijo.

1 Sintaxis del Cálculo Lambda

El Cálculo Lambda es un modelo simple, su sintaxis comprende sólo tres categorías de términos[3]:

- **Variables:** los elementos de esta categoría pertenecen a un conjunto infinito de letras minúsculas, generalmente las últimas del alfabeto (y en ocasiones pueden estar numeradas, algunos ejemplos pueden ser: x, y, z, x_1, z_2, etc.) y son las lambda-expresiones más simples.

- **Abstracciones:** esta categoría engloba los términos que definen a las funciones anónimas, compuestas por tres elementos: el primero es la letra griega "λ", el segundo es la variable que estará ligada en el cuerpo de la expresión y por último el cuerpo mismo de la función denotado como e, estas se representan de la forma "$\lambda x.e$".

- **Aplicación:** Esta operación se representa como: "$e_1\ e_2$" y corresponde a la aplicación de la función e1 a la función e2. La aplicación es asociativa a la izquierda de modo que una expresión como $e_1\ e_2\ e_3$ se interpreta como $(e_1 e_2)e_3$.

[2]Extraído de [108]
[3]Definición extraída de [108], [109] y [110]

Definición 1.1 (Sintaxis concreta del Cálculo Lambda)**.** Definimos el juicio $e\ \lambda term$ que indica que e es una expresión válida de acuerdo a la sintaxis del Cálculo Lambda. A este tipo de términos los denominaremos $\lambda-$expresiones[a].

$$\frac{x\ \mathsf{var}}{x\ \lambda term}\ var \qquad \frac{x\ \mathsf{var} \quad e\ \lambda term}{\lambda x.e\ \lambda term}\ abs \qquad \frac{e_1\ \lambda term \quad e_2\ \lambda term}{e_1\ e_2\ \lambda term}\ app$$

[a]Definición extraída de [5], [12], [108], [109]

Ejercicio 1.1. Para las siguientes $\lambda-$expresiones anota a la derecha a que categoría de la sintaxis del Cálculo Lambda corresponde dicho término.

x	variable
y	variable
$\lambda x.x$	lambda abstracción
$\lambda x.y$	lambda abstracción
$(\lambda x.x)v$	aplicación
$(\lambda x.x)v'$	aplicación
$\lambda x.\lambda y.xy$	lambda abstracción
$x(\lambda x.y)$	aplicación
$(\lambda x.x)(\lambda y.y)$	aplicación

En el ejemplo anterior, el tercer caso corresponde a la función identidad que regresa el mismo argumento que recibe. El cuarto corresponde a la función constante que siempre regresa el mismo valor y el séptimo es un ejemplo de como se puede anidar lambda abstracciones mono-parametrizadas para incrementar el número de parámetros en el cuerpo de la expresión[4].

Currificación en el Cálculo Lambda

Dado que la definición sintáctica del Cálculo Lambda sólo permite abstracciones con un único parámetro, las funciones de múltiples argumentos no son primitivas, sino que se representan mediante abstracciones anidadas.

La *currificación* es el principio según el cual una función de n parámetros puede expresarse como una sucesión de n funciones unarias. Formalmente, si M es un término y $x_1, \ldots, x_n$ son variables, entonces una función de n parámetros se escribe como:

$$\lambda x_1.\lambda x_2. \ldots \lambda x_n.\, M.$$

Cada abstracción introduce una variable ligada, y la aplicación de varios argumentos se entiende como aplicación iterada asociativa por la izquierda. De esta manera, un término como:

$$\lambda x.\lambda y.\, xy$$

[4]A este proceso se le conoce como "*currificación*" en honor a Haskell Curry y nos permite anidar tantas funciones como parámetros necesitemos.

representa una función de dos parámetros construida exclusivamente a partir de abstracciones unarias anidadas. En general, este mecanismo permite representar funciones de cualquier aridad sin extender la sintaxis básica del lenguaje.

2 α-equivalencia en el Cálculo Lambda

En el Cálculo Lambda se tiene un constructor similar al operador let que tiene una variable ligada y un alcance asociado a ella. Esto nos permite definir la α-equivalencia para expresiones que difieren a lo más en el nombrado de sus variables ligadas.

Definición 2.1 (α-equivalencia en el Cálculo Lambda)**.** Dos $\lambda-$expresiones e_1 y e_2 son α-equivalentes si y sólo si solo difieren a lo mas en el nombre de las variables ligadas[a]. Esto es denotado como:

$$e_1 \equiv_\alpha e_2$$

Por ejemplo:

$$\lambda x.x \quad y \quad \lambda z.z$$

son α-equivalentes y se representa como:

$$\lambda x.x \equiv_\alpha \lambda z.z$$

[a]Definición extraída de [5] y [12]

3 Semántica operacional del Cálculo Lambda

Cómo se discutió brevemente en la introducción del capítulo, la semántica operacional del Cálculo Lambda está definida por la sustitución sintáctica. A la operación de sustituir los términos que concuerden con la variable del operador en la $\lambda-$expresión se le conoce como β-reducción y se representa con el símbolo $\rightarrow_\beta$.

Definición 3.1 (Semántica operacional del Cálculo Lambda)**.** La semántica operacional está definida por la siguiente regla[a]:

$$(\lambda x.t)\, s \rightarrow_\beta t[x := s] \quad (\beta\text{-reducción})$$

Donde se tienen los siguientes casos según la composición de la $\lambda-$expresión:

- $x[x := r] = r$.
- $y[x := r] = y$ si $x \neq y$.
- $(ts)[x := r] = t[x := r]s[x := r]$.
- $(\lambda y.t)[x := r] = \lambda y.t[x := r]$ donde $y \notin FV(r)$ y $x \neq y$.

[a]Definición formulada de [5], [12], [108] y [109]

Definición 3.2 (Forma Normal del Cálculo Lambda)**.** Se dice que un λ término está en forma normal si no existe otro término e' tal que $e \rightarrow_\beta e'$[a].

De esta forma, si $e \to_\beta^* e_n$ y e_n está en forma normal, decimos que e_n es la forma normal de e.

[a]Ejercicio extraído de [5].

Ejercicio 3.1. Utiliza la definición de β-reducción para evaluar la siguiente $\lambda-$expresión.

$$e = (\lambda x.z)y$$
$$(\lambda x.z)y \to_\beta z[x := y]$$
$$= z$$

Ejercicio 3.2. Utiliza la definición de β-reducción para evaluar la siguiente $\lambda-$expresión.

$$e = (\lambda x.x)(\lambda y.yy)z$$
$$(\lambda x.x)(\lambda y.yy)z \to_\beta (x[x := \lambda y.yy])(z)$$
$$= (\lambda y.yy)(z) \to_\beta yy[y := z]$$
$$= y[y := z]y[y := z] = zz$$
$$= zz$$

Ejercicio 3.3. Utiliza la definición de β-reducción para evaluar la siguiente $\lambda-$expresión[a].

$$e = \big(\lambda x.\lambda y.xy\big)(\lambda z.z)(w)$$
$$\big(\lambda x.\lambda y.xy\big)(\lambda z.z)(w) \to_\beta (\lambda y.xy[x := \lambda z.z])(w)$$
$$= (\lambda y.x[x := \lambda z.z]y[x := \lambda z.z])(w)$$
$$= (\lambda y.(\lambda z.z)y)(w) \to_\beta (\lambda y.z[z := y])(w)$$
$$= \big(\lambda y.y\big)(w) \to_\beta y[y := w]$$
$$= w$$

[a]Ejercicio extraído de [5].

4 Definibilidad Lambda

En el Cálculo Lambda las funciones son los únicos objetos primitivos del sistema, no existen constantes, estructuras de datos ni tipos básicos incorporados en la sintaxis. Esto significa que las funciones:

1. Constituyen las entidades fundamentales a partir de las cuales se construyen todos los términos del lenguaje.
2. Pueden recibir otras funciones como argumentos y devolver funciones como resultado.

Esta propiedad caracteriza al Cálculo Lambda como un sistema de orden superior, pues las funciones no sólo actúan sobre valores, sino también sobre otras funciones.

La definibilidad lambda entonces es consecuencia directa del sistema, por ejemplo, cualquier objeto que deseemos introducir en el sistema, como: números, valores booleanos o estructuras más complejas. En el Cálculo Lambda, estos tipos primitivos serán definidos mediante funciones anónimas a las que convendremos un nombre para facilitar su represen-

tación

En contraste, en los lenguajes de programación convencionales las entidades o tipos primitivos son proporcionados directamente por la biblioteca estándar del lenguaje (int, char, bool, etc.).[5].

4.1 Booleanos y operadores lógicos

Las constantes booleanas True y False son entidades opuestas, una es el valor contrario de la otra. Si tratemos de trasladar esta idea a un par de funciones podemos pensar en algo como: la función que toma dos argumentos y regresa el primero es el opuesto de la función que toma dos argumentos y regresa el segundo[6]:

$$\lambda x.\lambda y.x$$

$$\lambda x.\lambda y.y$$

Estas entidades están haciendo exactamente lo contrario que hace la otra, como las funciones son objetos primitivos en el Cálculo Lambda entonces definiremos este par de elementos como nuestro True y nuestro False respectivamente.

De esta forma podemos definir funciones que a su vez, construyan la lógica booleana y nos permitan operar instrucciones de control.

Definición 4.1 (Operadores lógicos para el Cálculo Lambda)**.** Definimos las constantes booleanas True y False junto con los operadores NOT, AND e IF con las siguientes funciones[a]:

- $\mathsf{True} =_{def} \lambda x.\lambda y.x$
- $\mathsf{False} =_{def} \lambda x.\lambda y.y$
- $\mathsf{NOT} =_{def} \lambda z.z(\mathsf{False})(\mathsf{True})$
- $\mathsf{AND} =_{def} \lambda x.\lambda y.xy(\mathsf{False})$
- $\mathsf{If} =_{def} \lambda f.\lambda a.\lambda b.fab$

Es importante mencionar que para estas definiciones estamos abusando de la notación al no escribir la definición de las constantes y usar su nombre directamente. Esto es para ahorrar espacio en los desarrollos y estará permitido su uso en los ejercicios posteriores.

[a]Definición formulada de [108], [109] y [110]

Para el operador NOT la lógica de ejecución es aplicar la función identidad al parámetro de entrada, si este es True entonces se regresará el primer argumento, en este caso la constante False:

$$(\lambda x.x(\mathsf{False})(\mathsf{True}))\mathsf{True} \rightarrow_\beta^* \mathsf{True}(\mathsf{False})(\mathsf{True}) \rightarrow_\beta^* \mathsf{False}$$

Sí al contrario el parámetro de entrada es False este regresará el segundo argumento, en este caso la constante True:

$$(\lambda x.x(\mathsf{False})(\mathsf{True}))\mathsf{False} \rightarrow_\beta^* \mathsf{False}(\mathsf{False})(\mathsf{True}) \rightarrow_\beta^* \mathsf{True}$$

[5]Información consultada de [108] y [109]
[6]Definición formulada de [108], [110] y [111]

En el caso del operador AND la lógica subyacentes es que si el primer parámetro es un False no importa el segundo parámetro que reciba, siempre regresaremos la constante False:

$$(\lambda x.\lambda y.xy(\mathsf{False}))\ \mathsf{False}\ w \rightarrow_\beta^* \mathsf{False}\ w\ (\mathsf{False}) \rightarrow_\beta^* \mathsf{False}$$

Si recibe como primer parámetro la constante True entonces el resultado será el segundo parámetro:

$$(\lambda x.\lambda y.xy(\mathsf{False}))\ \mathsf{True}\ w \rightarrow_\beta^* \mathsf{True}\ w\ (\mathsf{False}) \rightarrow_\beta^* w$$

Notemos que la única forma de evaluar la instrucción AND como True es si se recibe esta constante dos veces.

En el Cálculo Lambda se espera que los operadores anteriormente definidos satisfagan las siguientes propiedades de comportamiento, en el sentido de que, tras aplicar un número finito de β-reducciones ($\rightarrow_\beta$), los términos correspondientes se reduzcan a las expresiones indicadas a la derecha:

- $\mathsf{If\ True}\, e_1\, e_2 \rightarrow_\beta^* e_1$
- $\mathsf{If\ False}\, e_1\, e_2 \rightarrow_\beta^* e_2$
- $\mathsf{NOT\ True} \rightarrow_\beta^* \mathsf{False}$
- $\mathsf{NOT\ False} \rightarrow_\beta^* \mathsf{True}$
- $\mathsf{AND\ False}\, b \rightarrow_\beta^* \mathsf{False}$
- $\mathsf{AND\ True}\, b \rightarrow_\beta^* b$

5 Aritmética del Cálculo Lambda

En los lenguajes de programación uno de los tipos de de datos primitivos que es de vital importancia y que se nos es proporcionado por la biblioteca estándar son los números enteros. Estas entidades tienen una representación en el Cálculo Lambda como funciones.

La construcción de los números naturales se obtiene al aplicar la función sucesor al cero, de tal forma que no tenemos el número dos, tenemos dos veces la aplicación de la función sucesor al cero: *s(s(0))*, en Cálculo Lambda esta idea se mantiene vigente, los números serán la aplicación anidada de una función a la constante cero.

5.1 Numerales de Church

Church introdujo los numerales como la abstracción de dos parámetros "s" y "z" en una función anónima, de tal forma que si se desea representar al n-ésimo número, este será formado por la aplicación de s a z, n veces[7]:

- $\bar{0} =_{def} \lambda s.\lambda z.z$
- $\bar{1} =_{def} \lambda s.\lambda z.s(z)$
- $\bar{2} =_{def} \lambda s.\lambda z.s(s(z))$
- $\bar{n} =_{def} \lambda s.\lambda z.\underbrace{s(\ldots(s}_{n\ veces}(z))\ldots)$

[7]Definición formulada de [108], [109] y [111]

5.2 Funciones aritméticas

Para los numerales de Church podemos difinir operadores aritméticos similares a los que se utilizan cuando trabajamos con los números enteros, estos operadores nos permitirán computar valores y estarán definidos como una función donde los operandos serán los argumentos de entrada.

Función sucesor

Comenzamos el estudio de las funciones aritméticas para el Cálculo Lambda definiendo la función suc para los numerales de Church, esta función recibe un numeral y construye el sucesor añadiendo un símbolo "s" al inicio.

Definición 5.1 (Función sucesor para los numerales de Church)**.** [a]

$$\mathsf{suc} =_{def} \lambda n.\lambda s.\lambda z.s(n\ s\ z)$$

[a] Definición extraída de [109] y [111]

Ejercicio 5.1. Utiliza la definición de la función suc para evaluar la siguiente λ-expresión

$$\mathsf{suc}\ \overline{2}$$
$$(\lambda n.\lambda s.\lambda z.s(n\ s\ z))\ \lambda s.\lambda z.s(s(z)) \rightarrow_\beta$$
$$\lambda s.\lambda z.s((\lambda s.\lambda z.s(s(z)))\ s\ z) \rightarrow_\beta$$
$$\lambda s.\lambda z.s(\lambda z.s(s(z))\ z) \rightarrow_\beta$$
$$\lambda s.\lambda z.s(s(s(z)))$$
$$= \overline{3}$$

Ejercicio 5.2. Utiliza la definición de la función suc para evaluar la siguiente λ-expresión

$$\mathsf{suc}\ \overline{5}$$
$$(\lambda n.\lambda s.\lambda z.s(n\ s\ z))\ \lambda z.\lambda s.s(s(s(s(s(z))))) \rightarrow_\beta$$
$$\lambda s.\lambda z.s((\lambda s.\lambda z.s(s(s(s(s(z))))))\ s\ z) \rightarrow_\beta$$
$$\lambda s.\lambda z.s(\lambda z.s(s(s(s(s(z)))))\ z) \rightarrow_\beta$$
$$\lambda s.\lambda z.s(s(s(s(s(s(z))))))$$
$$= \overline{6}$$

Suma

Esta función se define de forma similar a la función suma para los números naturales donde se reciben dos argumentos m y n y se aplica n veces la función suc al numeral m.

Definición 5.2 (Definición de la suma para los numerales de Church)**.** [a]

$$\mathsf{sum} =_{def} \lambda m.\lambda n.\lambda s.\lambda z.n(s\ (m\ s\ z))$$

[a] Definición extraída de [140], [109] y [111]

Ejercicio 5.3. Utiliza la definición de la función sum para evaluar la siguiente λ-expresión

$$\mathsf{sum}\ \overline{2}\ \overline{3}$$

$$= (\lambda m.\lambda n.\lambda s.\lambda z.n(s\ (m\ s\ z)))\ \lambda s.\lambda z.s(s(z))\ \ \lambda s.\lambda z.s(s(s(z))) \to_\beta$$

$$(\lambda n.\lambda s.\lambda z.n(s\ ((\lambda s.\lambda z.s(s(z)))\ s\ z)))\ \lambda s.\lambda z.s(s(s(z))) \to_\beta$$

$$(\lambda n.\lambda s.\lambda z.n(s\ ((\lambda z.s(s(z)))\ z)))\ \lambda s.\lambda z.s(s(s(z))) \to_\beta$$

$$(\lambda n.\lambda s.\lambda z.n(s\ s(s(z))))\ \lambda s.\lambda z.s(s(s(z))) \to_\beta$$

$$\lambda s.\lambda z.((\lambda s.\lambda z.s(s(s(z))))\ s\ s(s(z)))\ \to_\beta$$

$$\lambda s.\lambda z.((\lambda z.s(s(s(z))))\ s(s(z)))\ \to_\beta$$

$$\lambda s.\lambda z.s(s(s(s(s(z)))))\ \to_\beta$$

$$\overline{5}$$

Ejercicio 5.4. Utiliza la definición de la función sum para evaluar la siguiente λ-expresión

$$\mathsf{sum}\ \overline{4}\ \overline{5}$$

$$= (\lambda m.\lambda n.\lambda s.\lambda z.n(s\ (m\ s\ z)))\ \lambda s.\lambda z.s(s(s(s(z))))\ \lambda s.\lambda z.s(s(s(s(z)))))\ \to_\beta$$

$$(\lambda n.\lambda s.\lambda z.n(s\ ((\lambda s.\lambda z.s(s(s(s(z)))))\ s\ z)))\ \lambda s.\lambda z.s(s(s(s(z)))))\ \to_\beta$$

$$(\lambda n.\lambda s.\lambda z.n(s\ ((\lambda z.s(s(s(s(z)))))\ z)))\ \lambda s.\lambda z.s(s(s(s(z)))))\ \to_\beta$$

$$(\lambda n.\lambda s.\lambda z.n(s\ s(s(s(s(z)))))\ \lambda s.\lambda z.s(s(s(s(z)))))\ \to_\beta$$

$$(\lambda s.\lambda z.((\lambda s.\lambda z.s(s(s(s(s(z))))))\ s\ (s(s(s(s(z))))))\ \to_\beta$$

$$(\lambda s.\lambda z.((\lambda z.s(s(s(s(s(z))))))\ (s(s(s(s(z))))))\ \to_\beta$$

$$\lambda s.\lambda z.s(s(s(s(s(s(s(s(z)))))))))\ =$$

$$\overline{9}$$

Producto

Para el producto de m y n la idea es anidar la operación: sum $\overline{n}\,\overline{n}$, m veces reemplazando las apariciones de de la variable s en el numeral m por sum $\overline{n}$. Esta operación está definida de la siguiente manera:

Definición 5.3 (Definición del producto para los numerales de Church.)**.** [a]

$$\mathsf{prod} =_{def}\ \lambda m.\lambda n.m\ (\mathsf{sum}\ n)\ \bar{0}$$

[a] Definición extraída de [109] y [111]

Ejercicio 5.5. Utiliza la definición de la función prod para evaluar la siguiente λ-expresión

$$\mathsf{prod}\ \overline{2}\ \overline{3}$$

$$\mathsf{prod}\ \overline{2}\ \overline{3} = (\lambda m.\lambda n.m\ (\mathsf{sum}\ n)\ \overline{0})\ \ \lambda s.\lambda z.s(s(z))\ \ \lambda s.\lambda z.s(s(s(z))) \rightarrow_\beta$$

$$(\lambda n.(\lambda s.\lambda z.s(s(z)))\ (\mathsf{sum}\ n)\ \overline{0})\ \ \overline{3} \rightarrow_\beta$$

$$(\lambda n.(\lambda z.\mathsf{sum}\ n\ (\mathsf{sum}\ n\ (z)))\ \overline{0})\ \overline{3} \rightarrow_\beta$$

$$(\lambda n.\mathsf{sum}\ n\ (\mathsf{sum}\ n\ \overline{0}))\ \overline{3} \rightarrow_\beta$$

$$\mathsf{sum}\ \overline{3}\ (\mathsf{sum}\ \overline{3}\ \overline{0}) \rightarrow_\beta *$$

$$\mathsf{sum}\ \overline{3}\ \overline{3} \rightarrow_\beta *$$

$$\overline{6}$$

isZero

Este operador está diseñado para regresar True cuando el numeral que recibe como parámetro corresponde al numeral $\overline{0}$. En caso contrario la función regresa False y está definida de la siguiente forma.

Definición 5.4 (Función isZero para numerales de Church)**.** [a]

$$\mathsf{isZero} =_{def}\ \lambda n.n\ (\lambda x.\mathsf{False})\ \mathsf{True}$$

[a] Definición extraída de [2], [5] y [12]

Ejercicio 5.6. Utiliza las definición de la función isZero para evaluar la siguiente λ-expresión

$$\mathsf{isZero}\ \overline{2}$$

$$(\lambda n.n\ (\lambda x.\mathsf{False})\ \mathsf{True})\ \lambda s.\lambda z.s(s(z)) \rightarrow_\beta$$

$$(\lambda s.\lambda z.s(s(z)))\ (\lambda x.\mathsf{False})\ \mathsf{True} \rightarrow_\beta$$

$$(\lambda z.(\lambda x.\mathsf{False})((\lambda x.\mathsf{False})\ z))\ \mathsf{True} \rightarrow_\beta$$

$$(\lambda x.\mathsf{False})((\lambda x.\mathsf{False})\ \mathsf{True}) \rightarrow_\beta$$

$$\mathsf{False}$$

Ejercicio 5.7. Utiliza las definición de la función isZero para evaluar la siguiente λ-expresión

$$\mathsf{isZero}\ \overline{0}$$

$$(\lambda n.n\ (\lambda x.\mathsf{False})\ \mathsf{True})\ \lambda s.\lambda z.z \rightarrow_\beta$$

$$(\lambda s.\lambda z.z)\ (\lambda x.\mathsf{False})\ \mathsf{True} \rightarrow_\beta$$

$$(\lambda z.z)\ \mathsf{True} \rightarrow_\beta$$

$$\mathsf{True}$$

Definición 5.5 (Otros operadores aritméticos del Cálculo Lambda)**.** Ahora que nos hemos familiarizado con las ideas que fundamentan a la definición de los operadores aritméticos básicos en el Cálculo Lambda, podemos definir operadores más complejos que nos permitirán componer funciones elaboradas aplicando los operadores que hemos revisado

hasta este punto[a].

$$
\begin{aligned}
\mathsf{pred}\ n =_{def} & \quad \lambda n.\lambda f.\lambda x.n(\lambda g.\lambda h.h(gf))(\lambda u.x)(\lambda u.u) \\
\mathsf{sub}\ m\ n =_{def} & \quad \lambda m.\lambda n.n\ \mathsf{pred}\ m \\
\mathsf{absDiff}\ m\ n =_{def} & \quad \mathsf{add}(\mathsf{sub}\ n\ m)\ (\mathsf{sub}\ m\ n) \\
\mathsf{equal}\ m\ n =_{def} & \quad \mathsf{isZero}(\mathsf{absDiff}\ m\ n) \\
>\ m\ n =_{def} & \quad \mathsf{not}(\mathsf{isZero}(\mathsf{sub}\ m\ n)) \\
<\ m\ n =_{def} & \quad >\ n\ m
\end{aligned}
$$

Para el caso de la resta hay que tener en cuenta que los números negativos no están definidos en los numerales de Church, si el segundo argumento es mayor que el primero entonces se regresa el numeral $\overline{0}$, en caso contrario se aplica la operación pred n veces al numeral m.

[a]Definiciones tomadas de: [139] y [141]

6 Datos estructurados en el Cálculo Lambda

En el Cálculo Lambda es posible definir estructuras de datos para almacenar información junto con las funciones para recuperar los elementos guardados en estas.
La estructura más simple que revisaremos serán las tuplas binarias (pares) junto con sus proyecciones first y second que ayudarán a sentar la base para definir a las listas con sus respectivas funciones para obtener la cabeza y la cola.

6.1 Pares

Los pares es la estructura que representa a un par compuesto por un elemento izquierdo y un elemento derecho. En el Cálculo Lambda se define como la función que tiene tiene tres argumentos; el elemento izquierdo (en inglés se le conoce como $first$), el elemento derecho (o *second*) y una función b.

Definición 6.1 (Definición de pares para el Cálculo Lambda). [a]
Constructor

$$\mathsf{pair} =_{def} \lambda f \lambda s \lambda b.bfs$$

Proyección del primer elemento

$$\mathsf{fst} =_{def} \lambda p.p\ \mathsf{True}$$

Proyección del segundo elemento

$$\mathsf{snd} =_{def} \lambda p.p\ \mathsf{False}$$

[a]Definición extraída de [12], [110] y [113]

En general se tienen las siguientes reducciones cuando se aplican las proyecciones a los pares, donde a y b son expresiones arbitrarias del Cálculo Lambda[8]:

[8]Extraído de [12]

$\mathsf{fst}\ (\mathsf{pair\ a\ b}) \to_\beta^* \mathsf{a}$

$$\begin{array}{rl} & \mathsf{fst}\ (\mathsf{pair\ a\ b}) \\ = & \mathsf{fst}\ ((\lambda f.\lambda s.\lambda x.x f s)\ \mathsf{a\ b}) \\ \to_\beta & \mathsf{fst}\ ((\lambda s.\lambda x.x\ \mathsf{a}\ s)\ \mathsf{b}) \\ \to_\beta & \mathsf{fst}\ (\lambda x.x\ \mathsf{a\ b}) \\ = & (\lambda p.p\ \mathsf{True})(\lambda x.x\ \mathsf{a\ b}) \\ \to_\beta & (\lambda x.x\ \mathsf{a\ b})\ \mathsf{True} \\ \to_\beta & \mathsf{True\ a\ b} \end{array}$$

$$\begin{array}{rl} = & (\lambda x.\lambda y.x)\ \mathsf{a\ b} \\ \to_\beta & (\lambda y.\mathsf{a})\ \mathsf{b} \\ \to_\beta & \mathsf{a} \end{array}$$

$\mathsf{snd}\ (\mathsf{pair\ a\ b}) \to_\beta^* \mathsf{b}$

$$\begin{array}{rl} & \mathsf{snd}\ (\mathsf{pair\ a\ b}) \\ = & \mathsf{snd}\ ((\lambda f.\lambda s.\lambda x.x f s)\ \mathsf{a\ b}) \\ \to_\beta & \mathsf{snd}\ ((\lambda s.\lambda x.x\ \mathsf{a}\ s)\ \mathsf{b}) \\ \to_\beta & \mathsf{snd}\ (\lambda x.x\ \mathsf{a\ b}) \\ = & (\lambda p.p\ \mathsf{False})(\lambda x.x\ \mathsf{a\ b}) \\ \to_\beta & (\lambda x.x\ \mathsf{a\ b})\ \mathsf{False} \\ \to_\beta & \mathsf{False\ a\ b} \\ = & (\lambda x.\lambda y.y)\ \mathsf{a\ b} \\ \to_\beta & (\lambda y.y)\ \mathsf{b} \\ \to_\beta & \mathsf{b} \end{array}$$

6.2 Listas

Definición 6.2 (Definición de listas en Cálculo Lambda)**.** Las listas en el Cálculo Lambda se apoyan del constructor de los pares para ir añadiendo elementos anidando esta operación agregando una unidad por aplicación.

Para esta estructura la lista vacía será representada por una λ-expresión que sea α-equivalente a la constante False.[a]

Lista vacía

$$\mathsf{nil} = \mathsf{False}$$

Constructor para listas

$$\mathsf{cons} = \mathsf{pair}$$

Cabeza de una lista

$$\mathsf{head} = \mathsf{fst}$$

Cola de la lista

$$\mathsf{tail} = \mathsf{snd}$$

Test de la lista vacía

$$\mathsf{isNil} = \lambda l.l(\lambda h.\lambda t.\lambda d.\mathsf{False})\mathsf{True}$$

[a]Definición extraída de [12], [110] y [113]

7 Propiedades semánticas del Cálculo Lambda

7.1 No terminación

Con anterioridad estudiamos la propiedad de terminación para las expresiones correctamente formadas de EAB, en donde estas se evaluarán a un valor en un número finito de pasos. Cuando la evaluación ha llegado a este punto se le conoce como forma normal. No obstante esta propiedad no se cumple para el Cálculo Lambda como se puede ver a continuación:

Ejercicio 7.1. Demuestra o da un contra ejemplo de por qué la propiedad de terminación para el Cálculo Lambda es válida[a].

Consideremos el siguiente par de λ-expresiones:

$$\omega = \lambda x.xx$$

$$\Omega = \omega\omega$$

$$\Omega = \omega\omega = (\lambda x.xx)\omega \rightarrow_\beta \omega\omega$$

En general esta expresión cumple que en cada β-reducción se tiene la misma expresión Ω. Por lo tanto una expresión bien formada del Cálculo Lambda no necesariamente tiene una forma normal.

[a]Ejemplo extraído de [120]

Esto se deriva de la propiedad del Cálculo Lambda para auto-aplicar un término a sí mismo y resulta en expresiones que se ciclan al aplicar la β-reducción.

7.2 No determinismo

En los lenguajes de programación una propiedad deseable es el determinismo al evaluar las expresiones de los programas. Es decir, que los pasos en la evaluación que se aplique para una expresión sean siempre los mismos.

El Cálculo Lambda carece de esta propiedad, dependiendo del segmento que escogemos para aplicar la β-reducción el siguiente paso de la evaluación puede diferir[9].

Tomemos como ejemplo la siguiente λ-expresión[10]:

$$(\lambda z.(\lambda y.z)a)b$$

Si consideramos la aplicación para la expresión completa, se obtiene la siguiente evaluación:

$$\underline{(\lambda z.(\lambda y.z)a)b} \rightarrow_\beta (\lambda y.b)a$$

Si por el contrario consideramos la aplicación para la λ-expresión interna se obtiene:

$$(\lambda z.\underline{(\lambda y.z)a})b \rightarrow_\beta (\lambda z.z)b$$

[9]Definición formulada de [119]
[10]Ejemplo extraído de [5] y [12]

En ambos ejemplos, el resultado al que se llega al terminar de evaluar la expresión es la variable b sin importar cuál segmento se haya escogido.

7.3 Confluencia

La propiedad de confluencia nos asegura que dadas dos evaluaciones distintas para la misma λ-expresión, estas convergen en un término común en un punto de su evaluación:

Teorema 7.1 (Propiedad de Church-Rosser). [a]

Si $e \rightarrow_\beta^* e_1$ y $e \rightarrow_\beta^* e_2$ entonces existe un término t tal que $e_1 \rightarrow_\beta^* t$ y $e_2 \rightarrow_\beta^* t$.

[a] La demostración queda fuera del alcance de este manual pero se puede consultar en [114]

Corolario 7.2 (Unicidad de formas normales). Para cualquier λ-expresión e si $e \rightarrow_\beta^* e_f$ y $e \rightarrow_\beta^* e'_f$ tal que $\neg\exists\ e_{final}\ y\ e'_{final}$ donde $e_f \rightarrow_\beta e_{final}\ \ y\ \ e'_f \rightarrow_\beta\ \ e'_{final}$ entonces $e_f = e'_f$ salvo α-equivalencias.

8 Combinadores de punto fijo

El Cálculo Lambda por si mismo no posee un mecanismo iterativo que nos permita formar alguna secuencia de control como los ciclos for o while como en la mayoría de los lenguajes de programación modernos. Su naturaleza funcional nos hace preguntarnos si podemos emplear un mecanismo mas afín como la recursión para definir funciones que iteren sobre algún parámetro.

Para este fin se precisa de la introducción de los combinadores de punto fijo que capturan la esencia del principio de recursión general en computación:

$$rec\ f\ =\ f\ (rec\ f)$$

De esta forma, si definimos una λ-expresión rec que permita la auto-aplicación de si misma a una función que tome como argumento, entonces podemos definir cualquier función recursiva aplicando n veces el mismo procedimiento.

$$rec\ f = f(rec\ f) = f(\ f(rec\ f)) =\ f(\ f(\ f(\ ...\ )))$$

Definición 8.1 (Combinador de punto fijo). Un λ-expresión cerrada C es un combinador de punto fijo sí y sólo si cumple alguna de las siguientes condiciones[a]:

1. $C\,f \rightarrow_\beta^* f\,(C\,f)$
2. $C\,f \equiv_\beta f\,(C\,f)$ es decir, existe un término t tal que $C\,f \rightarrow_\beta^* t$ y $f\,(C\,f) \rightarrow_\beta^* t$

[a] Definición formulada de [5], [12] y [113]

Definición 8.2 (Combinador Y). Existen diferentes λ-expresiones que cumplen con la definición provista anteriormente. Uno de los combinadores más sencillos y populares es el Combinador Y que se define de la siguiente forma[a]:

$$Y =_{def} \lambda f.(\lambda x.f(xx))(\lambda x.f(xx))$$

[a]Definición extraída de [113]

Ejercicio 8.1. Demuestra que el combinador Y es un combinador de punto fijo[a].

$$\begin{array}{rl} & Y\,g \\ & (\lambda f.(\lambda x.f(xx))(\lambda x.f(xx)))\,g \\ \to_\beta & (\lambda x.g(xx))(\lambda x.g(xx)) \\ \to_\beta & g((\lambda x.g(xx))(\lambda x.g(xx))) \end{array}$$

Tomando la segunda parte de la igualdad y desarrollando obtenemos

$$\begin{array}{rl} & g\,(Y\,g) \\ & g\,((\lambda f.(\lambda x.f(xx))(\lambda x.f(xx)))\,g) \\ \to_\beta & g\,((\lambda x.g(xx))(\lambda x.g(xx))) \end{array}$$

de esta forma podemos concluir que $Y\,g \equiv_\beta g\,(Y\,g)$, entonces Y es un combinador de punto fijo.

[a]Extraído de [12]

Una vez definido el principio de recursión para el Cálculo Lambda y los combinadores de punto fijo, podemos definir funciones recursivas en este modelo. Por ejemplo, fibonacci.

$$\mathsf{fib} =_{def} Y\ \mathsf{fib'}$$
$$\mathsf{fib'} =_{def} \lambda f.\lambda x.\mathsf{if}\ (x < \overline{2})\ x\ \mathsf{else\ sum}\ f\ (\mathsf{pred}\ x)\ f\ (\mathsf{pred\ pred}\ x)$$

De esta forma la evaluación de la función fib, puede ser desarrollada como sigue:

$$\begin{array}{rl} & \mathsf{fib}\ \overline{2} \\ =_{def} & Y\ \mathsf{fib'}\ \overline{2} \\ \to_\beta * & \mathsf{fib'}\ (Y\ \mathsf{fib'})\ \overline{2} \\ =_{def} & \lambda f.\lambda x.\mathsf{if}\ (x < \overline{2})\ x\ \mathsf{else\ sum}\ f\ (\mathsf{pred}\ x)\ f\ (\mathsf{pred\ pred}\ x)\ (Y\ \mathsf{fib'})\ \overline{2} \\ \to_\beta & \lambda x.\mathsf{if}\ (x < \overline{2})\ x\ \mathsf{else\ sum}\ \ (Y\ \mathsf{fib'})\ (\mathsf{pred}\ x)\ \ (Y\ \mathsf{fib'})\ (\mathsf{pred\ pred}\ x)\ \overline{2} \\ \to_\beta & \mathsf{if}\ (\overline{2} < \overline{2})\ \overline{2}\ \mathsf{else\ sum}\ \ (Y\ \mathsf{fib'})\ (\mathsf{pred}\ \overline{2})\ \ (Y\ \mathsf{fib'})\ (\mathsf{pred\ pred}\ \overline{2}) \\ \to_\beta * & \mathsf{sum}\ \ (Y\ \mathsf{fib'}\ \overline{1})\ \ (Y\ \mathsf{fib'}\ \overline{0}) \\ \to_\beta * & \mathsf{sum}\ \ (Y\ \mathsf{fib'}\ \overline{1})\ \ (\mathsf{fib'}\ (Y\ \mathsf{fib'})\ \overline{0}) \\ =_{def} & \mathsf{sum}\ \ (Y\ \mathsf{fib'}\ \overline{1})\ \ (\lambda f.\lambda x.\mathsf{if}\ (x < 2)\ x\ \mathsf{else\ sum}\ f\ (\mathsf{pred}\ x)\ f\ (\mathsf{pred\ pred}\ x)\ (Y\ \mathsf{fib'})\ \overline{0}) \\ \to_\beta * & \mathsf{sum}\ \ (Y\ \mathsf{fib'}\ \overline{1})\ \ (\mathsf{if}\ (\overline{0} < \overline{2})\ \overline{0}\ \mathsf{else\ sum}\ ((Y\ \mathsf{fib'})\ (\mathsf{pred}\ \overline{0}))\ ((Y\ \mathsf{fib'})\ (\mathsf{pred\ pred}\ \overline{0})) \end{array}$$

$$\begin{array}{rl} \to_\beta * & \mathsf{sum}\ \ (Y\ \mathsf{fib'}\ \overline{1})\ \ \overline{0} \\ \to_\beta * & \mathsf{sum}\ \ (\mathsf{fib'}\ (Y\ \mathsf{fib'})\ \overline{1})\ \ \overline{0} \\ =_{def} & \mathsf{sum}\ \ (\lambda f.\lambda x.\mathsf{if}\ (x < 2)\ x\ \mathsf{else\ sum}\ f\ (\mathsf{pred}\ x)\ f\ (\mathsf{pred\ pred}\ x)\ (Y\ \mathsf{fib'})\ \overline{1})\ \ \overline{0} \\ \to_\beta * & \mathsf{sum}\ \ (\mathsf{if}\ (\overline{1} < 2)\ \overline{1}\ \mathsf{else\ sum}\ (Y\ \mathsf{fib'})\ (\mathsf{pred}\ \overline{1})\ (Y\ \mathsf{fib'})\ (\mathsf{pred\ pred}\ \overline{1}))\ \ \overline{0} \\ \to_\beta * & \mathsf{sum}\ \overline{1}\ \overline{0} \\ \to_\beta * & \overline{1} \end{array}$$

9 Ejercicios para el lector

Ejercicio 9.1. Supón que se requiere extender la definición 5.6) para booleanos y sus operadores en el Cálculo Lambda implementando el siguiente operador:

$$\mathsf{OR}\ a\ b$$

Donde el operador se comporta de la siguiente forma:

$$OR\ \mathsf{True}\ b \rightarrow_\beta^* \ \mathsf{True}$$

$$OR\ \mathsf{False}\ b \rightarrow_\beta^* \ b$$

Proporciona la λ-expresión que implemente dicha instrucción.

Ejercicio 9.2. Verifica las propiedades de la semántica operacional para cada operador booleano según la definición 5.6)

$$\mathsf{IF}\ \mathsf{True}\ e_1\ e_2 \rightarrow_\beta^* e_1$$

$$\mathsf{IF}\ \mathsf{False}\ e_1\ e_2 \rightarrow_\beta^* e_2$$

$$\mathsf{NOT}\ \mathsf{True} \rightarrow_\beta^* \mathsf{False}$$

$$\mathsf{NOT}\ \mathsf{False} \rightarrow_\beta^* \mathsf{True}$$

$$\mathsf{AND}\ \mathsf{False}\ b \rightarrow_\beta^* \mathsf{False}$$

$$\mathsf{AND}\ \mathsf{True}\ b \rightarrow_\beta^* b$$

$$\mathsf{OR}\ \mathsf{True}\ b \rightarrow_\beta^* \mathsf{True}$$

$$\mathsf{OR}\ \mathsf{False}\ b \rightarrow_\beta^* b$$

Ejercicio 9.3. Resuelve las siguientes operaciones de los numerales de Church (el abuso de notación para simplificar λ-expresiones está permitido con el fin de tener una representación más compacta cuando sea posible)

$$\mathsf{prod}\ (\mathsf{sum}\ \overline{7}\ \overline{4})\ \overline{2}$$

$$\mathsf{isZero}(\mathsf{prod}\ \overline{1}\ \overline{0})$$

$$\mathsf{isZero}(\mathsf{sum}\ \overline{3}\ (\mathsf{prod}\ \overline{5}\ \overline{4}))$$

$$\mathsf{isZero}(\mathsf{sub}\ \overline{4}\ \overline{5})$$

$$>\ \overline{4}\ \overline{5}$$

Ejercicio 9.4. Siguiendo la definición 6.2 de listas en el Cálculo Lambda define la función isNil que se comporta de la siguiente forma

$$\mathsf{isNil}\ [\ ]\ =\ \mathsf{True}$$

$$\mathsf{isNil}\ (x : xs)\ =\ \mathsf{False}$$

Ejercicio 9.5. Encuentra la forma normal de las siguientes λ-expresiones, si no es posible explica por qué.

$$(\lambda y.yy)(\lambda z.zz)(\lambda\alpha.\lambda\beta.\alpha\beta\alpha)\ \mathsf{False}\ \overline{0}$$

$$((\lambda z.\lambda y.\lambda u.uzy)(\lambda x.x)(\lambda wv.wv)\ \overline{0}\ \overline{1})\ \mathsf{True}$$

Ejercicio 9.6. Para cada una de las siguientes λ-expresiones demuestra que son combinadores de punto fijo.

$$\text{Turing } V \ = \ UU \text{ en donde } U \ = \ \lambda f.\lambda x.x(ffx)$$

$$\text{Estricto } Z \ = \ \lambda f.(\lambda x.f(\lambda v.xxv))(\lambda x.f(\lambda v.xxv))$$

Ejercicio 9.7. Utilizando cualquiera de los combinadores de punto fijo implementa la función recursiva factorial para los numerales de Church en el Cálculo Lambda.

$$\mathsf{factorial}\ 0 \ = \ 1$$

$$\mathsf{factorial}\ n \ = \ n \ * \ \mathsf{factorial}\ n-1$$

Adicionalmente muestra la ejecución para n = $\overline{3}$

Ejercicio 9.8. Utilizando el combinador Y, define la función concat para la representación de listas en el Cálculo Lambda.

$$\mathsf{concat}\ [\ ]\ (y:ys) \ = \ (y:ys)$$

$$\mathsf{concat}\ (x:xs)\ (y:ys) \ = x : \mathsf{concat}(xs \ : \ (y:ys))$$

Ejercicio 9.9. Utilizando cualquier combinador de punto fijo resuelve lo siguiente: Define la función reversa para las listas del Cálculo Lambda.

$$\mathsf{rev}\ [\ ] \ = \ [\ ]$$

$$\mathsf{rev}\ (x:xs) \ = \ \mathsf{rev}\ xs \ ++ \ [x]$$

Aplica la función para obtener la reversa de la lista: pair $\overline{1}$ (pair $\overline{2}$ (pair $\overline{3}$ False))

Ejercicio 9.10. Utilizando el combinador Y contesta lo siguiente:

- Da una definición de la función exponencial para los numerales de Church.
- Resuelve la función para exp $\overline{4}\ \overline{2}$.

Ejercicio 9.11. Supón que se requieren extender las estructuras de datos para el Cálculo Lambda con la introducción de árboles binarios, en donde cada nodo puede almacenar un valor (numerales de Church o booleanos). Contesta lo que se te pide a continuación:

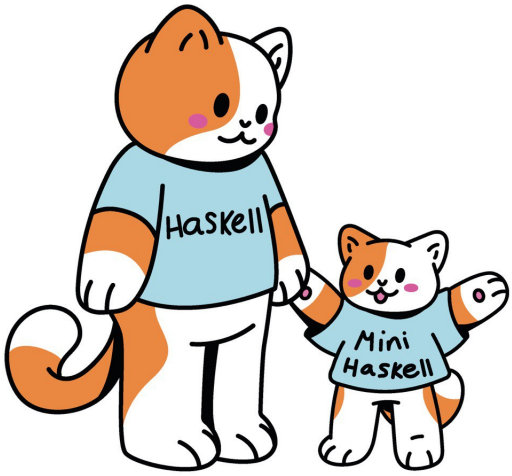
- Proporciona la λ-expresión para definir el árbol binario vacío.
- Proporciona la λ-expresión para definir el constructor de un nodo que contiene un elemento, un árbol binario izquierdo y un árbol binario derecho.
- Define la función fold que nos permita iterar el árbol y computar alguna operación sobre sus elementos (deja indicado el operador con el símbolo ”op”).

Ejercicio 9.12. Para cada una de los siguientes λ-expresiones menciona a cuál definición pertenece.

- $\lambda s \lambda z.s(s(s(s(s(sz)))))$
- $\lambda f.\lambda a.\lambda b.fab$
- $\lambda f \lambda s \lambda b.bfs$
- $\lambda m \lambda n \lambda s \lambda z.n\ s\ (m\ s\ z)$

Capítulo 6

MinHS



Con la teoría que hemos revisado hasta el momento en nuestro lenguaje EAB cuya definición engloba a los números naturales, booleanos, instrucciones lógicas y el operador let a la par del Cálculo Lambda, nos es natural preguntarnos ¿Cómo es la implementación de un lenguaje de programación de características similares para una computadora moderna? Para poder contestar a esta pregunta tomaremos como caso de estudio una versión simplificada del lenguaje de programación Haskell. Este posee características importantes para el enfoque de este curso que serán de interés trasladar a EAB.

Hasta el momento se ha trabajado con tipos de datos primitivos, pero no hemos hablado de las reglas de la semántica estática que asignan el tipo a las expresiones. Esto constituye un problema al poder escribir expresiones de EAB sintácticamente correctas pero cuya evaluación no termine en un valor, puesto que la ejecución se detendría en algún momento al no hallar una regla de la semántica dinámica que nos permita continuar.

Con esto en mente el objetivo de este capítulo será definir un lenguaje similar que nos permita tener todos los tipos de datos de EAB, el sistema de tipos de Haskell y las características más importante del mismo: evaluación perezosa, pureza funcional y asignación de tipos explícita y estática. A este lenguaje lo llamaremos MinHS al ser una versión acotada de Haskell pero servirá para ilustrar los conceptos que introduciremos en este capítulo.

Objetivo

Proveer la definición del lenguaje funcional MinHS que preserve las características principales del lenguaje de programación Haskell a manera de ilustrar una implementación concreta de los conceptos que se han revisado hasta este momento: sintaxis, semántica, Cálculo Lambda y recursión, haciendo especial énfasis en el sistema de tipos de este lenguaje y sus propiedades.

Planteamiento

El desarrollo del capítulo se plantea de forma similar a como lo hemos hecho con el lenguaje EAB dividiendo su definición en sintaxis concreta, sintaxis abstracta, semántica dinámica y es aquí en donde se introducirá una nueva capa para estudiar la asignación de tipos de las expresiones. Finalmente concluiremos mencionando brevemente las propiedades que este pequeño lenguaje posee.

1 Sintaxis de MinHS

1.1 Sintaxis concreta

El lenguaje de programación Haskell emplea la declaración explícita de tipos en su sintaxis concreta, este patrón se traslada a MinHS mediante la introducción de las anotaciones de tipos para *Int*, *Bool* y en el caso de las funciones, su tipo será representado por la expresión $T_1 \to T_2$ en la sintaxis concreta donde T_1 representa el tipo de la variable ligada en el cuerpo de la expresión y T_2 es el tipo que regresa la evaluación de la expresión.

Esta modificación a la sintaxis concreta solo aplicará para aquellas expresiones donde sea estrictamente necesario anotar su tipo para verificar la correctitud de tipificado. El resto de las expresiones se definen de manera similar a como lo hicimos para EAB limitando la aparición de típo en la medida de lo posible.

Definición 1.1 (Sintaxis Concreta de MinHS)**.** A continuación definimos los elementos que componen a las expresiones válidas de MinHS.[a]

El tipo de las expresiones ahora figura como parte de la sintaxis concreta.

Expresiones	e	$::=$	$var \mid n \mid b \mid (e) \mid e_1 \otimes e_2 \mid e_1\ e_2$
		$\mid$	$\mathsf{if}\ e_1\ \mathsf{then}\ e_2\ \mathsf{else}\ e_3$
		$\mid$	$\mathsf{let}\ x = e_1\ \mathsf{in}\ e_2\ \mathsf{end}$
		$\mid$	$\mathsf{fun}\ x :: T \Rightarrow e$
		$\mid$	$\mathsf{recfun}\ f :: (T_1 \to T_2)\ x \Rightarrow e$
Tipos	T	$::=$	$Bool \mid Nat \mid T_1 \to T_2$
Variables	var	$::=$	$x \mid y \mid \ldots$
Números	n	$::=$	$0 \mid 1 \mid \ldots$
Booleanos	b	$::=$	$\mathsf{true} \mid \mathsf{false}$
Operadores Infijos	$\otimes$	$::=$	$+ \mid * \mid - \mid = \mid < \mid >$

[a] Definición extraída de [5], [12], [115] y [116]

Ejercicio 1.1. Escribe la definición de los siguientes instrucciones utilizando la sintaxis concreta de MinHS

- Proporciona la definición de la función sucesor que recibe un número y regresa el sucesor:
$$\mathsf{suc} =_{def} \mathsf{fun}\ x\ ::\ Nat \Rightarrow x + 1$$
- Proporciona la definición de la función que recibe un número y decide si es cero:
$$\mathsf{IsZero} =_{def} \mathsf{fun}\ x\ ::\ Nat \Rightarrow\ \mathsf{if}\ (x\ =\ 0)\ \mathsf{then\ true\ else\ false}$$
- Proporciona la definición de una función que recibe un número y regresa el factorial:
$$\mathsf{fact} =_{def} \mathsf{recfun\ fact}\ ::\ (Nat \to Nat)\ x \Rightarrow\ \mathsf{if\ IsZero}(x)\ \mathsf{then}\ 1\ \mathsf{else}\ n * \mathsf{fact}\ (n-1)$$
- Proporciona la definición de una función que recibe dos números y regresa su producto:
$$\mathsf{prod} =_{def} \mathsf{fun}\ x\ ::\ Nat \Rightarrow\ \mathsf{fun}\ y\ ::\ Nat \Rightarrow\ x * y$$

1.2 Sintaxis abstracta

Definición 1.2 (Sintaxis abstracta de MinHS)**.** Una vez definidas las reglas para generar programas en MinHS empleando la sintaxis concreta del lenguaje, podemos definir su representación intermedia como un árbol de sintaxis abstracta[a].

Valores y variables

$$\frac{n \in \mathbb{N}}{num[n]\ ASA} \qquad \frac{}{bool[true]\ ASA} \qquad \frac{}{bool[false]\ ASA}$$

$$\frac{x \in\ Variables}{x\ ASA}$$

Operadores

$$\frac{t_1\ ASA \quad \cdots \quad t_n\ ASA}{O(t_1, \ldots, t_n)\ ASA}$$

Condicional

$$\frac{t_1\ ASA \quad t_2\ ASA \quad t_3\ ASA}{if(t_1, t_2, t_3)\ ASA}$$

Asignaciones locales

$$\frac{t_1\ ASA \quad t_2\ ASA}{let(t_1, x.t_2)\ ASA}$$

Definición de funciones

$$\frac{t\ ASA}{fun(T, x.t)\ ASA} \qquad \frac{t\ ASA}{recfun(T, f.x.t)\ ASA}$$

Aplicación de función

$$\frac{t_1\ ASA \quad t_2\ ASA}{app(t_1, t_2)\ ASA}$$

Operador de punto fijo

$$\frac{t\ ASA}{fix(T, f.t)\ ASA}$$

El operador de punto fijo fix es una implementación interna para evaluar expresiones recursivas. Como no está asociado a ninguna expresión de la sintaxis concreta es imposible que un usuario lo pueda instanciar directamente.

El operador $recfun$ tiene dos variables ligadas en el cuerpo de la definición: el nombre de la función y la variable que recibe como argumento.

[a] Definición extraída de [5], [12], [115] y [116]

2 Semántica de MinHs

2.1 Sistemas de tipos

Los tipos en el contexto de los lenguajes de programación son la descripción abstracta de una colección de valores que nos permite agruparlos y emplearlos de manera similar aún sin saber el contenido específico que estos pudieran tener.

En los lenguajes de programación los sistemas de tipos brindan información adicional sobre la evaluación de las expresiones. Por ejemplo, que valores debemos esperar recibir y regresar al concluir la ejecución de nuestro programa para definir restricciones que nos permitan tener seguridad y congruencia en los datos[1].

Este sistema estará definido en la siguiente capa de MinHS de forma similar como fue trabajado con anterioridad con el lenguaje EAB, dicho nivel es la semántica estática que define un conjunto de juicios para brindar la seguridad de evaluación a las expresiones.

2.2 Semántica estática

Haskell es un lenguaje de programación que implementa un sistema de verificación de tipos estático, esto quiere decir que la evaluación de las expresiones solo es posible cuando

[1] definición formulada a partir de [96]

el programa es congruente con las reglas definidas por su sistema de tipos, descartando la evaluación de todas aquellas expresiones que estén bien formadas pero que no respeten las restricciones de este sistema[2].

Definición 2.1 (Semántica estática)**.** La semántica estática nos ayuda a definir criterios para evaluar los programas de MinHS con la información que se pueda inferir acerca de los parámetros que una expresión toma como argumento o el tipo que esta regresa al concluir su evaluación. Para este propósito definimos el siguiente juicio[a]:

$$\Gamma \vdash t : T$$

El cual se lee como: "la expresión t tiene tipo T bajo el contexto Γ". En donde Γ es un conjunto de asignaciones de tipos a variables de la forma $\{x_1 : T_1 \ldots x_n : T_n\}$. Adicionalmente se utiliza la notación $\Gamma, x : T$ para indicar el conjunto $\Gamma \cup \{x : T\}$.

Variables

$$\frac{}{\Gamma, x : T \vdash x : T}$$

Valores numéricos

$$\frac{}{\Gamma \vdash num[n] : Nat}$$

Valores Booleanos

$$\frac{}{\Gamma \vdash bool[false] : Bool} \qquad \frac{}{\Gamma \vdash bool[true] : Bool}$$

Operadores

$$\frac{\Gamma \vdash t_1 : Nat \quad \Gamma \vdash t_2 : Nat}{\Gamma \vdash sum(t_1, t_2) : Nat} \qquad \frac{\Gamma \vdash t_1 : Nat \quad \Gamma \vdash t_2 : Nat}{\Gamma \vdash prod(t_1, t_2) : Nat}$$

$$\frac{\Gamma \vdash t_1 : Nat \quad \Gamma \vdash t_2 : Nat}{\Gamma \vdash res(t_1, t_2) : Nat} \qquad \frac{\Gamma \vdash t_1 : Nat \quad \Gamma \vdash t_2 : Nat}{\Gamma \vdash eq(t_1, t_2) : Bool}$$

$$\frac{\Gamma \vdash t_1 : Nat \quad \Gamma \vdash t_2 : Nat}{\Gamma \vdash gt(t_1, t_2) : Bool} \qquad \frac{\Gamma \vdash t_1 : Nat \quad \Gamma \vdash t_2 : Nat}{\Gamma \vdash lt(t_1, t_2) : Bool}$$

Condicional

$$\frac{\Gamma \vdash t_c : Bool \quad \Gamma \vdash t_t : T \quad \Gamma \vdash t_e : T}{\Gamma \vdash if(t_c, t_t, t_e) : T}$$

Asignaciones Locales

$$\frac{\Gamma \vdash t_v : T \quad \Gamma, x : T \vdash t_b : S}{\Gamma \vdash let(t_v, x.t_b) : S}$$

Funciones

$$\frac{\Gamma, x : T \vdash t : S}{\Gamma \vdash fun(T, x.t) : T \to S} \qquad \frac{\Gamma \vdash f : T \to S, x : T \vdash t : S}{\Gamma \vdash recfun(T \to S, f.x.t) : T \to S}$$

Aplicación de función

$$\frac{\Gamma \vdash t_f : T \to S \quad \Gamma \vdash t_p : T}{\Gamma \vdash app(t_f, t_p) : S}$$

[2]definición formulada a partir de [75]

Operador de punto fijo

$$\frac{\Gamma, x : T \vdash t : T}{\Gamma \vdash fix(T, x.t) : T}$$

Observe que en el caso de fix se está asumiendo el mismo tipo que se debe concluir.

[a] Definición extraída de [5], [12], [115] y [116]

Ejercicio 2.1. Para cada expresión de MinHS enlistada a continuación obtén su representación en sintaxis abstracta y aplica las reglas de la semántica estática para hacer el análisis de tipos de la expresión.

- 1 + (7 - 1)

  Representación en sintaxis abstracta:

$$sum(num[1], res(num[7], num[1]))$$

  Análisis de tipo aplicando la semántica estática:

$$\dfrac{\dfrac{}{\vdash num[1] \ : Nat} \qquad \dfrac{\dfrac{}{\vdash num[7] \ : \ Nat} \qquad \dfrac{}{\vdash num[1] \ : \ Nat}}{\vdash res(num[7], num[1]) \ : \ Nat}}{\vdash sum(num[1], res(num[7], num[1])) : Nat}$$

- fun $x \ :: Nat \Rightarrow \ x > 1$

  Representación en sintaxis abstracta:

$$fun(Nat, x.gt(x, num[1]))$$

  Análisis de tipo aplicando la semántica estática:

$$\dfrac{\dfrac{\dfrac{}{x \ : \ Nat \vdash x \ : \ Nat} \qquad \dfrac{}{x \ : \ Nat \vdash num[1] \ : \ Nat}}{x \ : \ Nat \vdash gt(x, num[1]) : Bool}}{\vdash fun(Nat, x.gt(x, num[1])) \ : \ Nat \rightarrow Bool}$$

- let $x \ = \ 3$ in if $x < 7$ then 7 else $x \ + \ (7 \ - \ x)$ end

  Representación en sintaxis abstracta:

$$let(num[3], x.if(lt(x, num[7]), num[7], sum(x, res(num[7], x))))$$

  Análisis de tipo aplicando la semántica estática:

$$\dfrac{\dfrac{}{\vdash num[3] \ : \ Nat} \qquad \dfrac{\dfrac{(A)\cdots}{x : Nat \vdash lt(x, num[7]) : Bool} \qquad \dfrac{}{x : Nat \vdash num[7] : Nat} \qquad \dfrac{(B)\cdots}{x : Nat \vdash sum(x, res(num[7], x)) : Nat}}{x : Nat \vdash if(lt(x, num[7]), num[7], sum(x, res(num[7], x)) \ : \ Nat}}{\vdash let(num[3], x.if(lt(x, num[7]), num[7], sum(x, res(num[7], x)))) \ : \ Nat}$$

  (A) Análisis semántico para lt

$$\dfrac{\dfrac{}{x : Nat \vdash x : Nat} \qquad \dfrac{}{x : Nat \vdash num[7] : Nat}}{x : Nat \vdash lt(x, num[7]) : Bool}$$

  (B) Análisis semántico para sum

$$\dfrac{\dfrac{}{x : Nat \vdash x : Nat} \qquad \dfrac{\dfrac{}{x : Nat \vdash num[7] : Nat} \qquad \dfrac{}{x : Nat \vdash x : Nat}}{x : Nat \vdash res(num[7], x) : Nat}}{x : Nat \vdash sum(x, res(num[7], x)) : Nat}$$

2.3 Semántica dinámica

Definición 2.2 (Semántica operacional de paso pequeño)**.** Para concluir con la definición de MinHS enunciaremos las reglas de la semántica operacional.

Es importante observar que la evaluación perezosa característica de Haskell se debe particularmente a los operadores app y let dado que la sustitución se hace sin evaluar la expresión que dará el valor en la variable ligada[a].

- Conjunto de estados $S = \{a \mid a\ ASA\}$
- Estados Iniciales $I = \{a \mid a\ ASA,\ \varnothing \sim a\}$
- Estados Finales $F = \{num[n], bool[true], bool[false], fun(T, x.t)\}$
- Transiciones, dadas por las siguientes reglas:

Condicional

$$\frac{}{if(bool[true], a_t, a_e) \to a_t}\ ifT \qquad \frac{}{if(bool[false], a_t, a_e) \to a_e}\ ifF$$

$$\frac{a_c \to a'_c}{if(a_c, a_t, a_e) \to if(a'_c, a_t, a_e)}\ if$$

Asignaciones locales

$$\frac{}{let(a_1, x.a_2) \to a_2[x := a_1]}\ let$$

Aplicación de función

$$\frac{a_f \to a'_f}{app(a_f, a_p) \to app(a'_f, a_p)}\ appL$$

$$\frac{}{app(fun(T, x.a_b), a_p) \to a_b[x := a_p]}\ app$$

$$\frac{}{app(recfun(T, x.f.a_b), a_p) \to a_b[f := fix(T, f.x.a_b),\ x :=\ a_p])}\ appR$$

$$\frac{}{app(fix(T, f.x.a_b), a_p) \to a_b[f := fix(T, f.x.a_b), x := a_p]}\ appF$$

Operador de punto fijo

$$\frac{}{fix(T, f.a) \to a[f := fix(T, f.a)]}\ fix$$

Operadores

Para acotar el listado de reglas se omite la representación para los operadores cuya definición es la misma a la que se dió para EAB en el capítulo 4: Semántica. Las reglas para operadores booleanos de comparación $<, >, =$ se definen de manera análoga.

Es importante notar que el operador *fix* puede aplicarse a sí mismo de manera indefinida, reemplazando las apariciones del nombre ligado en el cuerpo por la misma definición *fix*.

Esto supone un problema que revisaremos en la siguiente sección ya que deriva en estados ciclados donde el programa puede aplicar esta regla sin llegar a ningún valor si no se define con cuidado la función que estará en el cuerpo de la expresión.

[a] Definición extraída de [5], [12], [115] y [116]

3 Propiedades de MinHS

3.1 Seguridad del lenguaje

Esta propiedad proporciona un mecanismo para corroborar que derivado de la evaluación de una expresión que cumple las restricciones dictadas por el sistema de tipos del lenguaje, se obtendrá un valor. Vinculando las definiciones para la semántica estática concerniente al tipo de las expresiones y la semántica dinámica concerniente a la evaluación de las mismas[3].

En el contexto de MinHS nos interesa que a una expresión correctamente construida y correctamente tipificada se le pueda aplicar alguna de las reglas definidas por la semántica dinámica (progreso). Adicionalmente nos interesa que el tipo de las expresiones sea único y que a cada paso en la evaluación este se conserve (preservación). A continuación daremos una definición formal para estas propiedades.

- **Progreso:** un programa que cumple con las restricciones del sistema de tipos no se bloquea.

  Definición 3.1 (Propiedad de progreso)**.** Si $\Gamma \vdash t : T$ para algún tipo T entonces se cumple una de los siguientes dos casos[a]:
  - t es un valor
  - Existe una expresión t' tal que $t \to t'$

  [a] Definición extraída de [5], [12], [117] y [118]

- **Preservación:** un programa que cumple con las restricciones del sistema de tipos dará como resultado de la evaluación una expresión correctamente tipificada. Más aún, el tipo resultante será el mismo que aquél con el que se inició la evaluación.

  Definición 3.2 (Preservación de tipos)**.** Sí $\Gamma \vdash t : T$ y $t \to t'$ entonces $\Gamma \vdash t' : T$[a].

  [a] Definición extraída de [5], [12] y [118]

  Definición 3.3 (Unicidad de tipo)**.** Para cualesquiera Γ y expresión t de MinHS existe a lo más un tipo T de tal forma que se cumple: $\Gamma \vdash t : T$[a].

  [a] Definición extraída de [5], [12]

[3] Definición formulada de [117] y [118]

3.2 No terminación

MinHS hereda un problema similar a lo que sucedió en el Cálculo Lambda con la introducción de los combinadores de punto fijo para implementar la recursión general. Este problema es la existencia de expresiones del lenguaje sintácticamente y semánticamente correctas que producen un estado conocido como estado ciclado.

Ejercicio 3.1. Demuestra o da un contra ejemplo de la propiedad de terminación para MinHS[a]

Considera la función identidad que recibe un parámetro y regresa el mismo representada como *f.f*

Sí utilizamos esta expresión como el cuerpo de nuestro operador fix obtenemos la expresión:

$$fix(Nat,\ f.f) \rightarrow f[f := fix(Nat,\ f.f)] = fix(Nat,\ f.f) \rightarrow ...$$

Por lo tanto MinHS no posee la propiedad de terminación.

[a] Ejemplo extraído de [120]

4 Ejercicios para el lector

Ejercicio 4.1. Queremos extender la definición de MinHS para agregar el tipo de dato algebraico tupla representado como pair(x,y), donde ambos elementos no necesariamente tienen que tener el mismo tipo, así como las proyecciones fst y snd.

Con la descripción anterior responde los siguientes incisos:

- Define la sintaxis concreta para las tuplas.
- Define la regla de sintaxis abstracta para las tuplas.
- Define la regla de la semántica estática para verificación de tipos para las tuplas.
- Define la(s) reglas de evaluación para las tuplas.
- ¿Cuál es la diferencia entre una expresión que construye la tupla y la proyección que obtiene el elemento?.

Ejercicio 4.2. Para las siguientes expresiones de MinHS determina el tipo y verifica tu respuesta utilizando las reglas de la semántica estática de este lenguaje.

- pair(3, true)
- snd pair(3, true)
- fun x :: $Nat \ \Rightarrow\ x + 1$
- recfun $fact$ :: $(Nat \rightarrow Nat)\ x \Rightarrow$ if IsZero(x) then 1 else $n\ *\ fact\,(n-1)$
- let neg = (fun x : $Bool \Rightarrow$ if x then false else true) in neg($3 < 2$) end

Ejercicio 4.3. Queremos definir el operador Case con la siguiente sintaxis[a]:

$$\mathsf{Case}\ x\ \mathsf{of}\ x.g_1 \rightarrow\ c1\ ;\ x.g_2 \rightarrow\ c2\ ;\ ...\ \mathsf{end}$$

Donde Case es la generalización de múltiples expresiones if else para evaluar un argumento de entrada con una condicional o "guardia" (g_n) para determinar el caso al que este es aplicado (c_n). De tal forma que se toma como resultado de la expresión la primera guardia que se evalúe a verdadero de izquierda a derecha (o de arriba hacia abajo como generalmente se escribe el operador Case).

Adicionalmente queremos definir una expresión que por omisión sea el resultado de la evaluación de Case en caso de que ninguna de las guardias se evalúe a verdadero (en este caso se escribe como el último elemento representado como c_d):

$$\mathsf{Case}\ x\ \mathsf{of}\ x.g_1 \rightarrow\ c_1\ ;\ x.g_2 \rightarrow\ c_2\ ;\ ...\ ;\ c_d\ \mathsf{end}$$

Con la especificación anteriormente dada contesta los siguientes incisos:

- Extiende la sintaxis concreta del lenguaje para el operador Case.
- Extiende la sintaxis abstracta del lenguaje para el operador Case.
- Extiende la semántica estática del lenguaje para el operador Case.
- Extiende la semántica dinámica del lenguaje para el operador Case.

[a]Ejercicio extraído de [121]

Ejercicio 4.4. Dada la siguiente expresión Case:

$$\mathsf{Case}\ n\ \mathsf{of}\ n < 0 \rightarrow\ \mathsf{true}\ ; n = 0 \rightarrow\ \mathsf{true}\ ;\ \mathsf{false}$$

- Evalúa utilizando las reglas de semántica dinámica definidas en el inciso anterior.
- Sí $\Gamma = \{n : Nat\}$ utiliza las reglas de semántica estática definidas en el inciso anterior para obtener el tipo de la expresión.

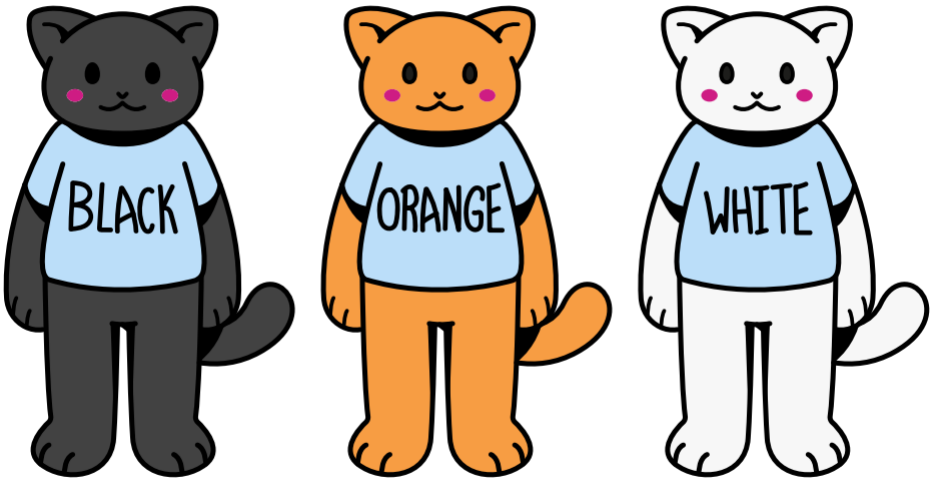
Ejercicio 4.5. Define las siguientes funciones utilizando la sintaxis concreta de MinHS y aplica las reglas de la semántica estática para verificar el tipo de las mismas[a]:

- Define la función exp(n_1, n_2) que eleva el primer argumento a la potencia del segundo.
- Define la función fibonacci(n) que obtiene el n-ésimo elemento de la sucesión de Fibonacci.
- Evalúa la expresión exp(2,2).
- Evalúa la expresión fibonacci(3).

[a]Ejercicio extraído de [121]

Capítulo 7

Inferencia de tipos



Una propiedad importante de Haskell que queremos trasladar a MinHS es el mecanismo de inferencia de tipos. Este mecanismo provee una capa extra que permite trabajar con expresiones aún si estas no poseen un tipo explícito en cada uno de sus parámetros de entrada y valores de retorno.

Anteriormente se definieron mediante la semántica estática las reglas para obtener el tipo de una expresión, no obstante este proceso es dependiente de las anotaciones explícitas de tipo que se encuentran definidas en la sintaxis del lenguaje. Con el mecanismo de inferencia de tipos se pretende desacoplar este conjunto de anotaciones de la sintaxis de MinHS.

Con esto en mente, vamos a formular un sistema de restricciones derivadas de la estructura de la expresión que se desea tipificar, junto con el proceso de estandarización de variables para poder hacer uso de la inferencia de tipos mediante la aplicación del algoritmo para

obtener el unificador general μ para expresiones de MinHS.

Objetivo

Definir el sistema de inferencia de tipos para MinHS que nos permita tipificar las expresiones garantizando la seguridad del lenguaje. Para esto, el objetivo principal será proporcionar una definición de las restricciones que cada estructura particular de las expresiones sintácticas de MinHS generan. A estas se les conoce como restricciones de tipo.

Adicionalmente se proporcionará la definición del algoritmo de unificación para aplicar la inferencia de tipos conocido como algoritmo para obtener el unificador general μ.

Planteamiento

En este capítulo revisaremos diferentes procedimientos que se aplican a las expresiones de MinHS para poder garantizar la correcta asignación de un tipo. Estudiaremos la estandarización de variables para obtener expresiones α-equivalentes cuando existan variables ligadas repetidas dentro de una expresión.

Posteriormente, vamos a definir los juicios para generar las restricciones de tipo que cada expresión de MinHS introduce, para finalmente definir el algoritmo unificador μ.

1 Estandarización de variables

La semántica estática de MinHS y la semántica estática de EAB se implementan de forma similar para ir agregando a un contexto (que denotamos con la letra Γ) aquellas variables definidas en las expresiones junto con el tipo que se espera estas tengan.

En el contexto Γ solo puede existir una sola aparición por variable, es decir, nunca podremos tener $\Gamma = \{x : Nat,\ x : Bool\}$ dado que Γ es un conjunto de variables y tipos. Tener la misma variable con dos tipos distintos es una inconsistencia dentro de nuestro programa y constituye un error en el tipificado de la expresión[1].

Para evitar esta situación se define el proceso conocido como estandarización de variables que consiste en brindar una α-equivalencia cuando dos variables ligadas tengan el mismo nombre.

Tomemos como ejemplo la siguiente expresión:

$$\mathsf{let}\ x = \mathsf{true}\ \mathsf{in}\ \mathsf{if}\ x\ \mathsf{then}\ \ \mathsf{let}\ x = 5\ \mathsf{in}\ x < x\ \mathsf{end}\ \ \mathsf{end}$$

En este caso la expresión está bien formada, no hay ninguna regla de la sintaxis concreta que esté mal aplicada para obtener nuestra expresión. La evaluación tampoco comprende un problema dado que la expresión let mas interna se evalúa a false y luego se continua con la evaluación del la instrucción if. Sin embargo, esta expresión nos genera el contexto $\Gamma = \{x : Bool,\ x : Nat\}$ el cual es inconsistente.

El problema puede ser corregido utilizando una α-equivalencia y renombrando las varia-

[1] Definición de semántica estática para MinHS formulada de [5], [12], [115] y [116]

bles x por x_0 y x_1 respectivamente:

$$\mathsf{let}\ x_0 = \mathsf{true}\ \mathsf{in}\ \mathsf{if}\ x_0\ \ \mathsf{then}\ \ \mathsf{let}\ x_1 = 5\ \mathsf{in}\ x_1 < x_1\ \mathsf{end}\ \ \mathsf{end}$$

Donde el contexto obtenido será: $\Gamma = \{x_0 : Bool,\ x_1 : Nat\}$

Ejercicio 1.1. Para la siguiente expresión de MinHS expresando el contexto, aplica la estandarización de variables:

$$\mathsf{let}\ x = \mathsf{false}\ \mathsf{in}\ (\mathsf{let}\ x = 6\ \mathsf{in}\ (x + x + x) > 10\ \mathsf{end})\ \mathsf{end}$$

$$\Gamma = \{x : Bool, x : Nat\}$$

$$\mathsf{let}\ x_0 = \mathsf{false}\ \mathsf{in}\ (\mathsf{let}\ x_1 = 6\ \mathsf{in}\ (x_1 + x_1 + x_1) > 10\ \mathsf{end})\ \mathsf{end}$$

$$\Gamma = \{x_0 : Bool, x_1 : Nat\}$$

Ejercicio 1.2. Para la siguiente expresión de MinHS expresando el contexto, aplica la estandarización de variables:

$$\mathsf{let}\ x = \mathsf{true}\ \mathsf{in}\ (\mathsf{let}\ x = 5\ \mathsf{in}\ x < 5\ \mathsf{end})\ \mathsf{end}$$

$$\Gamma = \{x : Bool,\ x : Nat\}$$

$$\mathsf{let}\ x_0 = \mathsf{true}\ \mathsf{in}\ (\mathsf{let}\ x_1 = 5\ \mathsf{in}\ x_1 < 5\ \mathsf{end})\ \mathsf{end}$$

$$\Gamma = \{x_0 : Bool,\ x_1 : Nat\}$$

Ejercicio 1.3. Para la siguiente expresión de MinHS, expresando el contexto, aplica la estandarización de variables:

$$\mathsf{let}\ x = 1\ \mathsf{in}\ (\mathsf{let}\ x = \mathsf{false}\ \mathsf{in}\ (\mathsf{let}\ x = fun\ y :: Bool \Rightarrow y\ \mathsf{in}\ x\ \mathsf{end})\ \mathsf{end})\ \mathsf{end}$$

$$\Gamma = \{x : Nat, x : Bool,\ x : Bool \to Bool\}$$

$$\mathsf{let}\ x_0 = 1\ \mathsf{in}\ (\ \mathsf{let}\ x_1 = \mathsf{false}\ \mathsf{in}\ (\mathsf{let}\ x_2 = fun\ y :: Bool \Rightarrow y\ \mathsf{in}\ x_2\ \mathsf{end})\ \mathsf{end})\ \mathsf{end}$$

$$\Gamma = \{x_0 : Nat,\ x_1 : Bool,\ x_2 : Bool \to Bool\}$$

2 Generación de restricciones

Para poder tipificar una expresión bien formada de MinHS sin utilizar las anotaciones de tipo explícitamente en los argumentos y valores de retorno, es necesario definir un proceso que nos ayude a obtener información sobre la estructura que esta posee.

A esta información se le conoce como restricciones y nos ayudan a ligar las condiciones que se deben cumplir para que una expresión esté bien tipificada.

Definición 2.1 (Conjunto de restricciones)**.** Definimos el juicio para generar restricciones partiendo de una expresión válida e representado como[a]:

$$e \mapsto R$$

Que se lee como: "La expresión e genera el conjunto R de restricciones de tipo".

[a] Juicio para representar restricciones [123], [124] y [125]

Definición 2.2 (Extensión de tipos para la generación de restricciones)**.** Para definir el algoritmo se extiende la categoría de tipos como sigue[a]:

$$T \ ::= \ X \mid Nat \mid Bool \mid T_1 \ \rightarrow T_2 \mid [\![e]\!]$$

En donde X es una variable de tipo. Estas variables nos ayudan en la definición de programas polimorfos, por ejemplo la función identidad:

$$fun\ x \Rightarrow x :: X \rightarrow X$$

La variable de tipo X va a tomar su valor hasta que la función sea evaluada, por ejemplo en la aplicación:

$$(fun\ x \Rightarrow x)\ 5 :: Nat \rightarrow Nat$$

La expresión $[\![e]\!]$ es una construcción sintáctica para definir el tipo de una expresión del lenguaje y se lee como: "el tipo de e".

Es importante aclarar que esta construcción no puede figurar como el tipo final asociado a la expresión y la encontraremos usualmente aplicada de la siguiente forma:

$$[\![e]\!] = E$$

[a]Definición formulada de [5], [12], [125] y [124]

Definición 2.3 (Algoritmo de generación de restricciones)**.** Una restricción es una ecuación de la forma $T_1 = T_2$ en donde T_1 y T_2 son tipos. La ecuación indica que T_1 debe ser igual a T_2 bajo unificación[a].

Variables

$$\frac{}{x_i \mapsto [\![x_i]\!] = X_i}$$

Para el tipo de las variables se usará la misma variable pero en mayúsculas. Todas las apariciones de la variable generarán la misma restricción y como los nombres de variables son únicos no habrá dos variables distintas con el mismo tipo.

Valores numéricos

$$\frac{}{num[n] \mapsto [\![num[n]]\!] = Nat}$$

Valores Booleanos

$$\frac{}{bool[b] \mapsto [\![bool[b]]\!] = Bool}$$

Condicional

$$\frac{c \mapsto R_1 \qquad t \mapsto R_2 \qquad e \mapsto R_3}{if(c,t,e) \mapsto R_1, R_2, R_3, [\![c]\!] = Bool, [\![t]\!] = [\![e]\!], [\![if(c,t,e)]\!] = [\![e]\!], [\![if(c,t,e)]\!] = [\![t]\!]}$$

Asignaciones Locales

$$\frac{v \mapsto R_1 \qquad b \mapsto R_2}{let(v, x_i.b) \mapsto R_1, R_2, X_i = [\![v]\!], [\![let(v, x_i.b)]\!] = [\![b]\!]}$$

Funciones

$$\frac{t \mapsto R}{fun(x_i.t) \mapsto R, [\![fun(x_i.t)]\!] = X_i \rightarrow [\![t]\!]}$$

$$\frac{b \mapsto R_1}{recfun(f.x_i.b) \mapsto R_1, [\![recfun(f.x_i.b)]\!] = X_i \to [\![b]\!]}$$

Aplicación de función

$$\frac{f \mapsto R_1 \qquad p \mapsto R_2}{app(f,p) \mapsto R_1, R_2, [\![f]\!] = [\![p]\!] \to [\![app(f,p)]\!]}$$

Operadores

$$\frac{e_1 \mapsto R_1 \qquad e_2 \mapsto R_2}{sum(e_1,e_2) \mapsto R_1, R_2, [\![e_1]\!] = Nat, [\![e_2]\!] = Nat, [\![sum(e_1,e_2)]\!] = Nat}$$

$$\frac{e_1 \mapsto R_1 \qquad e_2 \mapsto R_2}{prod(e_1,e_2) \mapsto R_1, R_2, [\![e_1]\!] = Nat, [\![e_2]\!] = Nat, [\![prod(e_1,e_2)]\!] = Nat}$$

$$\frac{e_1 \mapsto R_1 \qquad e_2 \mapsto R_2}{res(e_1,e_2) \mapsto R_1, R_2, [\![e_1]\!] = Nat, [\![e_2]\!] = Nat, [\![res(e_1,e_2)]\!] = Nat}$$

$$\frac{e_1 \mapsto R_1 \qquad e_2 \mapsto R_2}{eq(e_1,e_2) \mapsto R_1, R_2, [\![e_1]\!] = Nat, [\![e_2]\!] = Nat, [\![eq(e_1,e_2)]\!] = Bool}$$

$$\frac{e_1 \mapsto R_1 \qquad e_2 \mapsto R_2}{gt(e_1,e_2) \mapsto R_1, R_2, [\![e_1]\!] = Nat, [\![e_2]\!] = Nat, [\![gt(e_1,e_2)]\!] = Bool}$$

$$\frac{e_1 \mapsto R_1 \qquad e_2 \mapsto R_2}{lt(e_1,e_2) \mapsto R_1, R_2, [\![e_1]\!] = Nat, [\![e_2]\!] = Nat, [\![lt(e_1,e_2)]\!] = Bool}$$

[a] Definición formulada de [5], [12], [125] y [124]

3 Algoritmo de unificación

Una vez obtenida la lista de restricciones asociadas a una expresión e tenemos toda la información que necesitamos acerca de su estructura para comenzar a unificar los tipos aplicando las restricciones hasta encontrar el tipo más general de la expresión, o hasta encontrar un error de tipificado al asignar dos tipos distintos a un mismo elemento.

Definición 3.1 (Algoritmo de unificación)**.** La entrada del algoritmo es una lista de restricciones y la salida es una composición de sustituciones cuando la unificación sea posible. Fail en caso contrario.

A esta composición de sustituciones la denotamos con la letra μ y se le conoce como: "unificador general"[a].

$$
\begin{array}{rcll}
U([\,]) & = & [\,] & \\
U(T = T : R) & = & U(R) & \\
U(X = T : R) & = & U(R[X := T]) \circ [X := T] & \textit{si } X \notin var(T) \\
U(X = T : R) & = & \mathsf{Fail} & \text{si } X \in var(T) \\
U([\![e]\!] = T : R) & = & U(R[[\![e]\!] := T]) \circ [[\![e]\!] := T] & \\
U(T = X : R) & = & U(X = T : R) & \\
U(T = [\![e]\!] : R) & = & U([\![e]\!] = T : R) & \\
U(S_1 \to S_2 = T_1 \to T_2 : R) & = & U(S_1 = T_1 : S_2 = T_2 : R) & \\
U(R) & = & \mathsf{Fail} &
\end{array}
$$

Este algoritmo compone el unificador general convirtiendo la restricción de la cabeza de la lista en una sustitución (cuando sea posible), aplicándola al resto de las restricciones y concatenándola para generar el unificador general.

[a] Definición formulada de [5], [12], [123], [124] y [125]

4 Algoritmo de inferencia de tipos

Definición 4.1 (Algoritmo de Inferencia de tipos)**.** Se define el algoritmo $\mathcal{T}(e)$ de inferencia de tipos que recibe una expresión e de MinHS y regresa su tipo como sigue:[a]:

- Se encuentra la expresión e' con nombres de variables únicas.
- Se encuentra el conjunto de restricciones R tal que $e' \mapsto R$.
- Utilizando la función U se calcula el unificador general μ del conjunto de restricciones R, tal que $U(R) = \mu$.
- Se busca en μ la ecuación $[\![e']\!] := T$.
- T es el tipo general de e, es decir, $\mathcal{T}(e) = T$.

[a] Definición formulada de [5], [12], [123], [124] y [125]

5 Ejercicios para el lector

Ejercicio 5.1. Para la siguiente expresión de MinHS contesta los incisos enumerados a continuación:

$$\mathsf{let}\ z = 0\ \mathsf{in}\ z < z\ \mathsf{end}$$

- obtén la representación en sintaxis abstracta.
- Aplica la estandarización de variables para obtener una expresión α-equivalente.
- Aplica el algoritmo para obtener la lista de restricciones.

Ejercicio 5.2. Para la siguiente expresión de MinHS contesta los incisos enumerados a continuación:

$$\mathsf{fun}\ x \Rightarrow x\ \mathsf{*}\ (x\ \mathsf{-}\ 1) + (x\ \mathsf{*}\ x)$$

- obtén la representación en sintaxis abstracta.
- Aplica la estandarización de variables para obtener una expresión α-equivalente.
- Aplica el algoritmo para obtener la lista de restricciones.

Ejercicio 5.3. Para la siguiente expresión de MinHS contesta los incisos enumerados a continuación:

$$\mathsf{let}\ x = \mathsf{true}\ \mathsf{in}\ \mathsf{let}\ x = \mathsf{false}\ \mathsf{in}\ \mathsf{if}\ x\ \mathsf{then}\ \mathsf{let}\ y = 1\ \mathsf{in}\ y\ \mathsf{end}\ \mathsf{else}\ \mathsf{let}\ y = 2\ \mathsf{in}\ y\ \mathsf{end}\ \mathsf{end}\ \mathsf{end}$$

- obtén la representación en sintaxis abstracta.
- Aplica la estandarización de variables para obtener una expresión α-equivalente.
- Aplica el algoritmo para obtener la lista de restricciones.

Ejercicio 5.4. Dada la siguiente expresión de MinHS, obtén el tipo más general del unificador μ utilizando el algoritmo de inferencia[a]

$$\mathsf{fun}\ x \Rightarrow (\mathsf{if}\ x\ \mathsf{then}\ 1\ \mathsf{else}\ 0)\ \mathsf{true}$$

[a] Ejercicio extraído de [12]

Ejercicio 5.5. Dada la siguiente expresión de MinHS, obtén el tipo más general del unificador μ utilizando el algoritmo de inferencia[a]

$$\mathsf{fun}\ \ x \Rightarrow\ \mathsf{app}(x\,,x)$$

[a]Ejercicio extraído de [12]

Ejercicio 5.6. Dada la siguiente expresión de MinHS, obtén el tipo más general del unificador μ utilizando el algoritmo de inferencia.

$$\mathsf{let}\ x = 0\ \mathsf{in}\ \mathsf{fun}\ y \Rightarrow y + x\ \mathsf{end}$$

Ejercicio 5.7. Dada la siguiente expresión de MinHS, obtén el tipo más general del unificador μ utilizando el algoritmo de inferencia.

```
(recfun factorial n ⇒
    if (n < 2)
       then 1
    else factorial (n-1) * n) 9
```

Ejercicio 5.8. Contesta las siguientes preguntas.

- ¿Cuál es la complejidad de cada fase del algoritmo para encontrar el unificador general μ?
- ¿Cuál es la complejidad total del algoritmo para encontrar el unificador general μ?

Capítulo 8

Máquinas abstractas

Una máquina abstracta define una colección de estados y reglas de transición que nos permiten obtener información acerca de la ejecución de un programa por cada cómputo que este realiza. Este tipo de mecanismos de evaluación ya han sido revisados en cursos previos, específicamente en autómatas y lenguajes formales[1], donde se estudian sistemas de transición como autómatas y las máquinas de Turing.

En este capítulo introduciremos un sistema de cómputo mezclando las ideas de los sistemas de transición para definir una máquina de ejecución para MinHS, conocidas como las máquinas $\mathcal{H}$ y $\mathcal{J}$.

[1]Conforme al plan de estudios que se imparte desde el 2013 en la Facultad de Ciencias de la Universidad Nacional Autónoma de México con clave de asignatura 1425.

Objetivo

En este capítulo exploraremos la definición de las máquinas $\mathcal{H}$ y $\mathcal{J}$ implementando la colección de estados, cómputos y los axiomas de transición que nos permitan visualizar el proceso de evaluación de un expresión cerrada de MinHS, ejemplos de programas válidos serán revisados para ilustrar los cómputos.

Planteamiento

El capítulo está estructurado para definir los estados, marcos y reglas de transición de las máquinas $\mathcal{H}$ y $\mathcal{J}$ seguido de la ejecución detallada paso a paso de programas correctamente formados de MinHS para ilustrar la ejecución de ambas máquinas.

De forma similar, la pila de ejecución será introducida para controlar aquellos cómputos que se quedan pendientes de evaluación. Estos elementos los conocemos como ”marcos” y se bloquean hasta obtener uno o mas valores que puedan ser utilizados como argumentos. La pila entonces es un mecanismo de control que retiene el orden (LIFO: Last In First Out) de las llamadas en nuestro programa.

Posteriormente extenderemos la definición de la máquina $\mathcal{H}$ para abstraer el manejo de variables definiendo así la máquina $\mathcal{J}$.

Finalmente se estudian los conceptos de alcance estático y dinámico que cada una de estas máquinas implementa junto con el concepto de cerradura.

1 La máquina $\mathcal{H}$

Para la implementación de este máquina de estados es necesaria la introducción de marcos. Los marcos representan un programa con marcadores de los cómputos pendientes que necesitamos para continuar con la evaluación de un determinado estado en la expresión de MinHS que queremos evaluar. Adicionalmente se presenta la pila de ejecución para llevar un control de dichos cómputos (en inglés se conocen como *frames* pero en esta nota les llamaremos ”marcos”[2]).

Esta implementación resulta particularmente cómoda para seguir la ejecución de un programa de una manera simple y legible. Sin embargo, posteriores optimizaciones serán propuestas con la introducción de la máquina $\mathcal{J}$

1.1 Marcos y la pila de control

Los marcos (o *frames* en inglés) son estructuras sintácticas que denotan cómputos pendientes en la evaluación de un programa. Por ejemplo, para el caso de los operadores, si ambos operandos no han sido evaluados exhaustivamente hasta ser reducidos a un valor, la ejecución del operador no puede continuar. Es aquí cuando los marcos resultan útiles para indicar que no se continuará con la evaluación del programa hasta no resolver los cómputos pendientes en dicho marco.

[2]Este término es una traducción directa de [122]

Adicionalmente tendremos una pila en la que almacenaremos todos los marcos pendientes para conservar el orden en el que se fueron encontrando en el programa. Cada marco será empujado al tope de la pila para evaluar las subexpresiones asociados a el.

Una vez concluida la evaluación de las subexpresiones de éste computo, el valor obtenido es empujado al siguiente marco en el tope de la pila y detenido hasta que todas las subexpresiones hayan sido terminadas de evaluar o se regresa como valor final si la pila está vacía (según sea el caso).

Definición 1.1 (Marcos)**.** Los marcos son marcadores estructurales que registran los cómputos pendientes de una expresión. En nuestra definición, el símbolo $\square$ indica el lugar en donde se está llevando acabo la evaluación actual[a].

Operadores primitivos

$$\frac{}{O(\square, e_2)\ marco} \qquad \frac{}{O(v_1, \square)\ marco}$$

Condicional

$$\frac{}{if(\square, e_2, e_3)\ marco}$$

Aplicación de función

$$\frac{}{app(\square, e_2)\ marco}$$

Notemos que los operadores fix, fun y recfun no tienen una definición con un marco. Esto es porque estas expresiones solo pueden ser aplicadas con la expresión app que define el marcador de cómputo pendiente para el argumento de la función.

[a] Definición formulada de [5], [8], [12], y [122]

Definición 1.2 (Pila de control)**.** Una pila de control está formada a partir de marcos, y se define recursivamente como sigue[a]:

$$\frac{}{\diamond\ pila}\ \text{vacia} \qquad \frac{\text{m}\ marco \quad \text{p}\ pila}{\text{m;p}\ pila}\ \text{top}$$

[a] Definición formulada de [5], [8] [12], y [122]

1.2 Estados

Los estados para la máquina $\mathcal{H}$ estarán agrupados en dos categorías; de evaluación y de retorno. Los estados de evaluación exhaustan una expresión hasta obtener un valor y los de retorno empujarán dicho valor a la pila de control (los valores pueden ser retornados a un marco o regresados como valores finales).

Los estados iniciales comenzarán la evaluación de una expresión con una pila vacía y los estados finales retornarán un valor a una pila vacía.

Definición 1.3 (Estados de la máquina $\mathcal{H}$)**.** Los estados están compuestos de una pila de control P y una expresión e cerrada y son de alguna de las siguientes formas[a]:

- **Estados de evaluación**: Se evalúa e siendo P la pila de control y lo denotamos como $P \succ e$
- **Estados de retorno**: Devuelve el valor v a la pila de control P, que denotamos como $P \prec v$

En donde se distinguen dos tipos de estados en particular:

- **Estados iniciales**: comienzan la evaluación con la pila vacía denotados como $\diamond \succ e$.
- **Estados finales**: regresan un valor a la pila vacía y se denota $\diamond \prec v$

[a]Definición formulada de [5], [8] y [12]

1.3 Transiciones

La función de transición vincula la pila de ejecución con los marcos, definiendo las transiciones posibles entre cada uno de los estados.

Sí el cómputo pendiente de un marco ha terminado su evaluación, el valor regresa al marcador de lugar y se continúa con el resto de los cómputos. Sí todos los cómputos han sido resueltos entonces se evalúa lo operación con los valores retornados al marco.

Definición 1.4 (Transiciones para la máquina $\mathcal{H}$)**.** Las transiciones se definen por medio de la relación $\rightarrow_{\mathcal{H}}$ y son de la forma[a] :

$$P \succ e \rightarrow_{\mathcal{H}} P' \succ e'$$

Valores Los valores del lenguaje son números, booleanos y funciones. La evaluación de un valor simplemente lo regresa como resultado a la pila, pues un valor ya finalizó su proceso de evaluación.

$$\frac{}{P \succ v \rightarrow_{\mathcal{H}} P \prec v}$$

Operaciones Definimos el proceso para evaluar cualquier operador de la siguiente forma: Para evaluar $O(e_1, e_2)$ agregamos el marco $O(\square, e_2)$ a la pila y evaluamos e_1.

$$\frac{}{P \succ O(e_1, e_2) \rightarrow_{\mathcal{H}} O(\square, e_2); P \succ e_1}$$

Si tenemos en el tope de la pila el marco $O(\square, e_2)$ y se regresa como resultado un valor v, entonces, evaluamos e_2 y sustituimos el tope de la pila por el marco $O(v_1, \square)$.

$$\frac{}{O(\square, e_2); P \prec v_1 \rightarrow_{\mathcal{H}} O(v_1, \square); P \succ e_2}$$

Si se devuelve un valor a la pila que tiene como tope el marco $O(v_1, \square)$ entonces

podemos devolver al resto de la pila el resultado de la operación de ambos valores. Aplicando este principio a la operación sum tenemos la siguiente regla.

$$\frac{}{sum(num[n], \square); P \prec num[m] \rightarrow_{\mathcal{H}} P \prec num[n+m]}$$

Condicional Para evaluar la expresión $if(e_1, e_2, e_3)$ agregamos el marco $if(\square, e_2, e_3)$ al tope de la pila y evaluamos e_1.

$$\frac{}{P \succ if(e_1, e_2, e_3) \rightarrow_{\mathcal{H}} if(\square, e_2, e_3); P \succ e_1}$$

Si se regresa true a la pila con el marco $if(\square, e_2, e_3)$ en el tope, entonces evaluamos e_2 con el resto de la pila.

$$\frac{}{if(\square, e_2, e_3); P \prec \mathsf{true} \rightarrow_{\mathcal{H}} P \succ e_2}$$

Si se regresa false a la pila con el marco $if(\square, e_2, e_3)$ en el tope, entonces evaluamos e_3 con el resto de la pila.

$$\frac{}{if(\square, e_2, e_3); P \prec \mathsf{false} \rightarrow_{\mathcal{H}} P \succ e_3}$$

Asignaciones locales Si se quiere evaluar la expresión $let(e_1, x.e_2)$ con la pila P entonces se evalúa e_2 en donde se sustituyen las apariciones de x por e_1.

$$\frac{}{P \succ let(e_1, x.e_2) \rightarrow_{\mathcal{H}} P \succ e_2[x := e_1]}$$

Aplicación de función Para evaluar una aplicación $app(e_1, e_2)$ en una pila P se agrega el marco $app(\square, e_2)$ como tope y se evalúa e_1. Si se regresa un valor $fun(x.e_1)$ a la pila con tope $app(\square, e_2)$ entonces se quita el tope y se evalúa e_1 sustituyendo x por e_2.

$$\frac{}{P \succ app(e_1, e_2) \rightarrow_{\mathcal{H}} P; app(\square, e_2) \succ e_1}$$

$$\frac{}{app(\square, e_2); P \prec fun(x.e_1) \rightarrow_{\mathcal{H}} P \succ e_1[x := e_2]}$$

Si se regresa un valor $recfun(f.x.e_1)$ a la pila con tope $app(\square, e_2)$ entonces se quita el tope y se evalúa e_1 sustituyendo f por su punto fijo y x por e_2.

$$\frac{}{app(\square, e_2); P \prec recfun(f.x.e_1) \rightarrow_{\mathcal{H}} P \succ e_1[f := fix(f.x.e_1), x := e_2]}$$

El operador de punto fijo Para evaluar la expresión $fix(f.e)$ en la pila P se evalúa e sustituyendo f por $fix(f.e)$.

$$\frac{}{P \succ fix(f.e) \rightarrow_{\mathcal{H}} P \succ e[f := fix(f.e)]}$$

[a] Definición formulada de [5], [8] y [12]

Ejercicio 1.1 (Ejecución máquina $\mathcal{H}$ con *marcos*)**.** Para ver el funcionamiento de la máquina $\mathcal{H}$ vamos a evaluar la siguiente expresión:

```
let x = false in
    let y = 4 in
       if x then y + 1 else y - 1
    end
end
```

En sintaxis abstracta:

$$let(\mathsf{false}, x.let(4, y.if(x, sum(y,1), res(y,1)))))$$

La evaluamos en la máquina $\mathcal{H}$.

$$
\begin{array}{rcccl}
\diamond & \succ & let(\mathsf{false}, x.let(4, y.if(x, sum(y,1), res(y,1)))) & \to_{\mathcal{H}} \\
\diamond & \succ & let(4, y.if(x, sum(y,1), res(y,1)))[x := \mathsf{false}] & \to_{\mathcal{H}} \\
\diamond & \succ & let(4, y.if(\mathsf{false}, sum(y,1), res(y,1))) & \to_{\mathcal{H}} \\
\diamond & \succ & if(\mathsf{false}, sum(y,1), res(y,1))[y := 4] & \to_{\mathcal{H}} \\
\diamond & \succ & if(\mathsf{false}, sum(4,1), res(4,1)) & \to_{\mathcal{H}} \\
if(\square, sum(4,1), res(4,1)) : \diamond & \succ & \mathsf{false} & \to_{\mathcal{H}} \\
if(\square, sum(4,1), res(4,1)) : \diamond & \prec & \mathsf{false} & \to_{\mathcal{H}} \\
\diamond & \succ & res(4,1) & \to_{\mathcal{H}} \\
res(\square, 1) : \diamond & \succ & 4 & \to_{\mathcal{H}} \\
res(\square, 1) : \diamond & \prec & 4 & \to_{\mathcal{H}} \\
res(4, \square) : \diamond & \succ & 1 & \to_{\mathcal{H}} \\
res(4, \square) : \diamond & \prec & 1 & \to_{\mathcal{H}} \\
\diamond & \succ & 4 - 1 & \to_{\mathcal{H}} \\
\diamond & \prec & 3 &
\end{array}
$$

Ejercicio 1.2 (Ejecución máquina $\mathcal{H}$ con *marcos*)**.** Para ver el funcionamiento de la máquina $\mathcal{H}$ vamos a evaluar la siguiente expresión:

```
let x = fun z -> z + 1 in
    let y = 4 in
       x y
    end
end
```

En sintaxis abstracta:

$$let(fun(z.sum(z,1)), x.let(4, y.app(x,y)))$$

La evaluamos en la máquina $\mathcal{H}$.

$$\begin{array}{rcccl}
 & \diamond & \succ & let(fun(z.sum(z,1)), x.let(4, y.app(x,y))) & \to_{\mathcal{H}} \\
 & \diamond & \succ & let(4, y.app(x,y))[x := fun(z.sum(z,1))] & \to_{\mathcal{H}} \\
 & \diamond & \succ & let(4, y.app(fun(z.sum(z,1)), y)) & \to_{\mathcal{H}} \\
 & \diamond & \succ & app(fun(z.sum(z,1)), y)[y := 4] & \to_{\mathcal{H}} \\
 & \diamond & \succ & app(fun(z.sum(z,1)), 4) & \to_{\mathcal{H}} \\
app(\square, 4) : & \diamond & \succ & fun(z.sum(z,1)) & \to_{\mathcal{H}} \\
app(\square, 4) : & \diamond & \prec & fun(z.sum(z,1)) & \to_{\mathcal{H}} \\
 & \diamond & \succ & sum(z,1)[z := 4] & \to_{\mathcal{H}} \\
 & \diamond & \succ & sum(4,1) & \to_{\mathcal{H}} \\
sum(\square, 1) : & \diamond & \succ & 4 & \to_{\mathcal{H}} \\
sum(\square, 1) : & \diamond & \prec & 4 & \to_{\mathcal{H}} \\
sum(4, \square) : & \diamond & \succ & 1 & \to_{\mathcal{H}} \\
sum(4, \square) : & \diamond & \prec & 1 & \to_{\mathcal{H}} \\
 & \diamond & \succ & 4+1 & \to_{\mathcal{H}} \\
 & \diamond & \prec & 5 &
\end{array}$$

2 La máquina $\mathcal{J}$

La máquina $\mathcal{J}$ es una extensión de la máquina $\mathcal{H}$ con la adición de un *cache* para almacenar variables y sus valores al que denominamos entorno. Este entorno será el encargado de regresar el valor asociado a un atributo en las operaciones de sustitución (let, fix, fun, app). Esto con el objetivo de tener una implementación eficiente para la sustitución, operación que hasta el momento posee una complejidad lineal sobre el tamaño de la expresión que queremos evaluar.

Con pequeños ajustes podemos redefinir este nuevo modelo partiendo de las reglas definidas para $\mathcal{H}$ los cuales discutiremos a continuación.

2.1 Marcos

Definición 2.1 (Marcos)**.** En la máquina $\mathcal{J}$ se usa el mismo conjunto de marcos que los presentados para la máquina $\mathcal{H}$, con la única excepción siendo los marcos de let y app para implementar una evaluación ansiosa con el fin de simplificar las operaciones de sustitución.[a].

Asignaciones locales

$$\frac{}{let(\square, x.e_2)\ marco}$$

Aplicación de función

$$\frac{}{app(f, \square)\ marco}$$

[a]Definición formulada de [5], [8] y [12]

2.2 Ambientes

La introducción del concepto de ambientes, nos permitirá almacenar los pares *variable / valor* en un contexto para optimizar la resolución de una asignación en las expresiones evaluadas por $\mathcal{J}$. Los ambientes se definen de forma análoga a las listas como veremos en la siguiente especificación.

Definición 2.2 (Ambientes)**.** Un ambiente es una estructura que almacena asignaciones de variables con su valor y la definimos recursivamente como[a]:

$$\frac{}{\bullet\ env}\ \text{vacio} \qquad \frac{x\quad var \qquad v\quad valor \qquad \mathbb{E}\ env}{x \leftarrow v\ ; \mathbb{E}\ env}\ \text{asig}$$

La forma de acceder a los elementos del ambiente es mediante el nombre de la variable, entonces si el ambiente $\mathbb{E}$ tiene la asignación $x \leftarrow v$. Podemos acceder al valor de x como $\mathbb{E}[x]$ y el resultado es v.

En caso de tener mas de una asignación sobre el mismo nombre de variable $\mathbb{E}[x]$ nos regresa la primera aparición de x en el ambiente. Por ejemplo en el ambiente $\mathbb{E} =_{def} x \leftarrow v_1; x \leftarrow v_2; \bullet$ la operación $\mathbb{E}[x]$ nos arroja como resultado v_1.

[a]Definición formulada de [5], [8] y [12]

2.3 Estados

Definición 2.3 (Estados de la máquina $\mathcal{J}$)**.** Los estados ahora son una relación ternaria de una pila de control P un ambiente $\mathbb{E}$ y una expresión e, denotados como[a]:

- **Estados de evaluación**: $P \mid \mathbb{E} \succ e$
- **Estados de retorno**: $P \mid \mathbb{E} \prec e$

Un estado inicial es de la forma:

$$\diamond \mid \bullet \succ e$$

Mientras que los estados finales son de la forma:

$$\diamond \mid \mathbb{E} \prec v$$

Notemos que en los estados finales no importa que está guardado en el ambiente solo importa que la pila de control esté vacía.

[a] Definición formulada de [5], [8] y [12]

2.4 Transiciones

Definición 2.4 (Transiciones de variables en la máquina $\mathcal{J}$[a]). A continuación brindamos la especificación para la regla de transición buscando el valor asociado a una variable desde el contexto.

Variables En la máquina $\mathcal{H}$ una variable representa un estado bloqueado, pues no hay forma de continuar la evaluación del programa si no se aplica una operación de sustitución sobre la misma.

En la máquina $\mathcal{J}$ no está definida la operación de sustitución. Tenemos que evaluar las variables buscándolas en el ambiente, lo que definimos con la siguiente regla:

$$\frac{}{P \mid \mathbb{E} \succ x \rightarrow_j P \mid \mathbb{E} \prec \mathbb{E}[x]}$$

[a] Definición formulada de [5], [8] y [12]

Queremos utilizar los ambientes para introducir el concepto de alcance de una variable[3]. De manera informal el alcance de una variable es la porción del programa donde la definición de dicha variable es válida, por ejemplo en la expresión:

$$let(e_1, x.e_2)$$

El alcance de x solo estará definido para el cuerpo de la expresión e_2. Una vez que se termine de evaluar la expresión *let* queremos eliminar del ambiente la asignación $x \leftarrow e_1$ para que no figure en el alcance de futuras evaluaciones. Para esto debemos extender la pila de los marcos para que pueda almacenar ambientes y deshacerse de ellos.

Definición 2.5 (Pila de control para $\mathcal{J}$[a]).

$$\frac{}{\diamond\ pila}\ \mathsf{vacia} \qquad \frac{m\ marco \quad P\ pila}{m; P\ pila}\ \mathsf{top} \qquad \frac{\mathbb{E}\ env \quad P\ pila}{\mathbb{E}\ ; P\ pila}\ \mathsf{top}$$

[3] Exploraremos de manera formal este concepto mas adelante en este capítulo.

De esta forma cuando evaluemos una expresión que genera un ambiente distinto (llamémosle ambiente B), se guarda el ambiente actual en la pila de control (llamémosle ambiente A) y se definen las nuevas asignaciones dentro del ambiente B.

Una vez que termine la ejecución y se regrese un valor a la pila, recuperamos el ambiente A y continuamos con la ejecución del programa.

[a] Definición formulada de [5], [8] y [12]

Finalmente enlistaremos las transiciones para la máquina $\mathcal{J}$ que difieren con la máquina $\mathcal{H}$. El resto de transiciones que no aparecen en esta definición se heredan directamente de esta última máquina.

Estas reglas permiten manipular los entornos para guardar, definir y sustituir variables durante la ejecución del programa. Una vez concluida los entornos serán descartados hasta liberar la pila de ejecución para retornar el valor final.

Definición 2.6 (Transiciones con alcance en la máquina $\mathcal{J}$[a])**.** A continuación brindamos la especificación de las reglas de evaluación que involucran la aplicación de la sustitución para resolverse.

Asignaciones locales

$$\frac{}{P \mid \mathbb{E} \succ let(e_1, x.e_2) \rightarrow_{\mathcal{J}} let(\square, x.e_2); P \mid \mathbb{E} \succ e_1}$$

$$\frac{}{let(\square, x.e_2)\ ;\ P \mid \mathbb{E} \prec v \rightarrow_{\mathcal{J}} \mathbb{E}\ ;\ P \mid x \leftarrow v\ ;\ \mathbb{E} \succ e_2}$$

Aplicación de función

$$\frac{}{app(\square, e_2); P \mid \mathbb{E} \prec v_1 \rightarrow_{\mathcal{J}} app(v_1, \square); P \mid \mathbb{E} \succ e_2}$$

$$\frac{}{app(fun(x.e_1), \square)\ ;\ P \mid \mathbb{E} \prec v_2 \rightarrow_{\mathcal{J}} \mathbb{E}\ ;\ P \mid x \leftarrow v_2; \mathbb{E} \succ e_1}$$

$$\frac{}{app(recfun(f.x.e_1), \square); P|E \prec v_2 \rightarrow_{\mathcal{J}} \mathbb{E}; P|f \leftarrow fix(f.x.e_1); x \leftarrow v_2; \mathbb{E} \succ e_1}$$

Liberación del ambiente

$$\frac{}{\mathbb{E}; P \mid E_1 \prec v \rightarrow_{\mathcal{J}} P \mid \mathbb{E} \prec v}$$

Esta regla se agrega para liberar el ambiente de la pila de control.

[a] Definición formulada de [5], [8] y [12]

Ejercicio 2.1. Ejecución de la máquina $\mathcal{J}$.[a] Tomemos como ejemplo la siguiente expresión de MinHS

```
let x = 3 in
    let f = fun y => x + y in
        let x = 5 in
            f 4
        end
    end
end
```

La cual nos da la siguiente representación en sintaxis abstracta:

$$let(3, x.let(fun(y.sum(x, y)), f.let(5, x.app(f, 4))))$$

La evaluación en la máquina $\mathcal{J}$ es de la siguiente forma:

$$\begin{array}{rcl}
\diamond|\ \bullet & \succ & let(3, x. \dots) \\
let(\square, x. \dots); \diamond|\ \bullet & \succ & 3 \\
let(\square, x. \dots); \diamond|\ \bullet & \prec & 3 \\
\bullet; \diamond|\ x \leftarrow 3; \bullet & \succ & let(lam(y.sum(x, y)), \dots) \\
let(\square, \dots); \bullet; \diamond|\underline{x \leftarrow 3; \bullet}_{E_1} & \succ & lam(y.sum(x, y)) \\
let(\square, \dots); \bullet; \diamond|\ x \leftarrow 3; \bullet & \prec & lam(y.sum(x, y)) \\
E_1; \bullet; \diamond|\ f \leftarrow lam(y.sum(x, y)); x \leftarrow 3; \bullet & \succ & let(5, x.app(f, 4)) \\
let(\square, x.app(f, 4)); E_1; \bullet; \diamond|\ f \leftarrow lam(y.sum(x, y)); x \leftarrow 3; \bullet & \succ & 5 \\
let(\square, x.app(f, 4)); E_1; \bullet; \diamond|\underline{f \leftarrow lam(y.sum(x, y)); x \leftarrow 3; \bullet}_{E_2} & \prec & 5 \\
E_2; E_1; \bullet; \diamond|\ x \leftarrow 5; f \leftarrow lam(y.sum(x, y)); x \leftarrow 3; \bullet & \succ & app(f, 4) \\
app(\square, 4); E_2; E_1; \bullet; \diamond|\ x \leftarrow 5; f \leftarrow lam(y.sum(x, y)); x \leftarrow 3; \bullet & \succ & f \\
app(\square, 4); E_2; E_1; \bullet; \diamond|\underline{x \leftarrow 5; f \leftarrow lam(y.sum(x, y)); x \leftarrow 3; \bullet}_{E_3} & \prec & lam(y.sum(x, y)) \\
E_3; E_2; E_1; \bullet; \diamond|\ y \leftarrow 4; x \leftarrow 5; f \leftarrow lam(y.sum(x, y)); x \leftarrow 3; \bullet & \succ & sum(x, y) \\
sum(\square, y); E_3; E_2; E_1; \bullet; \diamond|\ y \leftarrow 4; x \leftarrow 5; f \leftarrow lam(y.sum(x, y)); x \leftarrow 3; \bullet & \succ & x \\
sum(\square, y); E_3; E_2; E_1; \bullet; \diamond|\ y \leftarrow 4; x \leftarrow 5; f \leftarrow lam(y.sum(x, y)); x \leftarrow 3; \bullet & \prec & 5 \\
sum(5, \square); E_3; E_2; E_1; \bullet; \diamond|\ y \leftarrow 4; x \leftarrow 5; f \leftarrow lam(y.sum(x, y)); x \leftarrow 3; \bullet & \succ & y \\
sum(5, \square); E_3; E_2; E_1; \bullet; \diamond|\ y \leftarrow 4; x \leftarrow 5; f \leftarrow lam(y.sum(x, y)); x \leftarrow 3; \bullet & \prec & 4 \\
E_3; E_2; E_1; \bullet; \diamond|\ y \leftarrow 4; x \leftarrow 5; f \leftarrow lam(y.sum(x, y)); x \leftarrow 3; \bullet & \prec & 5 + 4 \\
E_2; E_1; \bullet; \diamond|E_3 & \prec & 9 \\
E_1; \bullet; \diamond|E_2 & \prec & 9 \\
\bullet; \diamond|E_1 & \prec & 9 \\
\diamond|\ \bullet & \prec & 9
\end{array}$$

[a] Ejemplo extraído de [12]

2.5 Alcance

El alcance de una variable es la porción del programa en la cual esta está definida. Dependiendo de la implementación del lenguaje este alcance puede ser estático o dinámico.

Definición 2.7 (Alcance estático)**.** Para los programas que implementan alcance estático, las variables solo estarán definidas en un área delimitada por bloques de indentación, marcadores de lugar como el operador let, in, end, cuerpo de una función etc[a].

[a] Definición extraída de [2] y [107]

Definición 2.8 (Alcance dinámico)**.** En un lenguaje que implementa el alcance dinámico, el alcance de una variable es todo el programa y se toma la última asignación hecha a la misma cuando se requiera evaluar[a].

[a] Definición extraída de [2] y [107]

2.6 Cerraduras

Los alcances de las máquinas $\mathcal{H}$ y $\mathcal{J}$ suponen un problema dado que los resultados obtenidos al evaluar la misma expresión son distintos para las expresiones de tipo función.

Esto se debe de corregir para la definición de la máquina $\mathcal{J}$ mediante la introducción de cerraduras, que encapsulan el contexto de evaluación y la expresión para la que dicho contexto será aplicado cuando se evalúe.

Las cerraduras nos ayudan a imitar la definición de un alcance estático en la implementación de $\mathcal{J}$, para esto deberemos modificar las reglas de evaluación.

Definición 2.9 (Cerraduras)**.** Una cerradura es una pareja de una expresión de función de MinHS y un ambiente que se denota[a]:

$$\ll \mathbb{E}, f \gg$$

Cuya interpretación es que el ambiente adecuado para evaluar la función f es $\mathbb{E}$.

De esta forma se respeta el ambiente en el que se define una función para usar el mismo en su ejecución y tener una evaluación con alcance estático.

[a] Definición formulada de [5], [8] y [12]

2.7 Transiciones con cerradura para $\mathcal{J}$

Una vez hecho el análisis de la diferencia de alcances en ambas máquinas, necesitamos homogeneizar la evaluación para que el ambiente no introduzca efectos secundarios y el resultado de la evaluación de un programa en ambas máquinas difiera.

Solo las reglas que modifican el ambiente serán candidatas para ser ajustadas en la definición de las transiciones para la máquina $\mathcal{J}$, en particular app, fun, recfun y fix.

Definición 2.10 (Transición para funciones)**.** En lugar de regresar las funciones como una expresión, se guardará como una pareja de la expresión y el ambiente en el que fue definido[a].

$$\frac{}{P \mid \mathbb{E} \succ fun(x.e) \rightarrow_{\mathcal{J}} P \mid \mathbb{E} \prec\!\ll \mathbb{E}, x.e \gg}$$

$$\frac{}{app(\square, e_2); P \mid \mathbb{E} \prec\!\ll E_f, x.e_1 \gg \rightarrow_{\mathcal{J}} app(\ll E_f, x.e_1 \gg, \square); P \mid \mathbb{E} \succ e_2}$$

$$\frac{}{app(\ll E_f, x.e_1 \gg, \square); P \mid \mathbb{E} \prec v \rightarrow_{\mathcal{J}} \mathbb{E}; P \mid x \leftarrow v; E_f \succ e_1}$$

$$\frac{}{P \mid \mathbb{E} \succ recfun(f.x.e) \rightarrow_{\mathcal{J}} P \mid \mathbb{E} \prec fix(f. \ll \mathbb{E}, x.e \gg)}$$

$$\frac{}{app(\square, e_2); P | \mathbb{E} \prec fix(f. \ll E_f, x.e_1 \gg) \rightarrow_{\mathcal{J}} app(fix(f. \ll E_f, x.e_1 \gg), \square); P \mid \mathbb{E} \succ e_2}$$

$$\frac{}{app(fix(f. \ll E_f, x.e_1 \gg), \square); P \mid \mathbb{E} \prec v \rightarrow_{\mathcal{J}} \mathbb{E}; P \mid x \leftarrow v; f \leftarrow fix(f. \ll E_f, x.e_1 \gg); E_f \succ e_1}$$

[a] Definición formulada de [5], [8] y [12]

Para finalizar el capítulo presentamos un ejemplo de la evaluación en la máquina $\mathcal{J}$ con las nuevas reglas para mostrar como ahora se pueden guardar ambientes y sustituirlos según se necesite durante el proceso de evaluación.

Esto nos da la flexibilidad de capturar un ambiente para una porción específica del programa sin perder el resto de las asignaciones contenidas en el.

Una vez concluida la evaluación de la instrucción para la cual un ambiente fue guardado, se restaura el ambiente general para continuar con el resto del programa.

Ejercicio 2.2. Ejecución de la máquina $\mathcal{J}$ implementando cerraduras.[a] Tomemos como ejemplo la siguiente expresión de MinHS

```
let x = 3 in
    let f = fun y => x + y in
        let x = 5 in
            f 4
        end
    end
end
```

La cual nos da la siguiente representación en sintaxis abstracta:

$$let(3, x.let(fun(y.sum(x, y)), f.let(5, x.app(f, 4))))$$

La evaluación en la máquina $\mathcal{J}$ es de la siguiente forma:

$$\begin{array}{rcl}
\diamond|\ \bullet & \succ & let(3, x.\dots) \\
let(\square, x.\dots); \diamond|\ \bullet & \succ & 3 \\
let(\square, x.\dots); \diamond|\ \bullet & \prec & 3 \\
\bullet; \diamond|\ x \leftarrow 3; \bullet & \succ & let(fun(y.sum(x, y)), \dots) \\
let(\square, \dots); \bullet; \diamond|\underline{x \leftarrow 3; \bullet}_{E_1} & \succ & fun(y.sum(x, y)) \\
let(\square, \dots); \bullet; \diamond|\ x \leftarrow 3; \bullet & \prec & \ll E_1, y.sum(x, y) \gg \\
E_1; \bullet; \diamond|\underline{f \leftarrow \ll E_1, y.sum(x, y) \gg; x \leftarrow 3; \bullet}_{E_2} & \succ & let(5, x \dots) \\
let(\square, \dots); E_1; \bullet; \diamond|\ f \leftarrow \ll E_1, y.sum(x, y) \gg; x \leftarrow 3; \bullet & \succ & 5 \\
let(\square, \dots); E_1; \bullet; \diamond|\ f \leftarrow \ll E_1, y.sum(x, y) \gg; x \leftarrow 3; \bullet & \prec & 5 \\
E_2; E_1; \bullet; \diamond|\underline{x \leftarrow 5; f \leftarrow \ll E_1, y.sum(x, y) \gg; x \leftarrow 3; \bullet}_{E_3} & \succ & app(f, 4) \\
app(\square, 4); E_2; E_1; \bullet; \diamond|E_3 & \succ & f \\
app(\square, 4); E_2; E_1; \bullet; \diamond|E_3 & \prec & \ll E_1, y.sum(x, y) \gg \\
app(\ll E_1, y.sum(x, y) \gg, \square); E_2; E_1; \bullet; \diamond|E_3 & \succ & 4 \\
app(\ll E_1, y.sum(x, y) \gg, \square); E_2; E_1; \bullet; \diamond|E_3 & \prec & 4 \\
E_3; E_2; E_1; \bullet; \diamond|\ y \leftarrow 4; E_1 & \succ & sum(x, y) \\
sum(\square, y); E_3; E_2; E_1; \bullet; \diamond|\ y \leftarrow 4; E_1 & \succ & x \\
sum(\square, y); E_3; E_2; E_1; \bullet; \diamond|\ y \leftarrow 4; E_1 & \prec & 3 \\
sum(3, \square); E_3; E_2; E_1; \bullet; \diamond|\ y \leftarrow 4; E_1 & \succ & y \\
sum(3, \square); E_3; E_2; E_1; \bullet; \diamond|\ y \leftarrow 4; E_1 & \prec & 4 \\
E_3; E_2; E_1; \bullet; \diamond|E_1 & \prec & 3 + 4 \\
E_2; E_1; \bullet; \diamond|E_3 & \prec & 7 \\
E_1; \bullet; \diamond|E_2 & \prec & 7 \\
\bullet; \diamond|E_1 & \prec & 7 \\
\diamond|\ \bullet & \prec & 7
\end{array}$$

[a] Ejemplo extraído de [12]

De esta forma concluimos el estudio de la implementación de las máquinas abstractas para MinHS. A continuación se encuentran los ejercicios de comprensión para el lector.

3 Ejercicios para el lector

Ejercicio 3.1. Dada la siguiente expresión de MinHS contesta lo siguiente:

```
let x = false in
    let y = fun z => leq(z,0) in
        let x = true in
            if x then y 0 else false
        end
    end
end
```

- Evalúa la expresión de acuerdo a la definición de la máquina $\mathcal{H}$.
- Evalúa la expresión de acuerdo a la definición de la máquina $\mathcal{J}$.
- Los resultados obtenidos son los mismos o son diferentes?.
- Evalúa la expresión utilizando la definición extendida para la máquina $\mathcal{J}$ con cerraduras
- El resultado obtenido es el mismo?.

Ejercicio 3.2. Dada la siguiente expresión de MinHS contesta lo siguiente:

```
let recfun pow x n =
    if  n = 0 then 1 else x * pow x  n-1
       in
        pow 2 4
end
```

- Evalúa la expresión de acuerdo a la definición de la máquina $\mathcal{H}$.
- Evalúa la expresión de acuerdo a la definición de la máquina $\mathcal{J}$.
- Los resultados obtenidos son los mismos o son diferentes?.
- Evalúa la expresión utilizando la definición extendida para la máquina $\mathcal{J}$ con cerraduras.
- El resultado obtenido es el mismo?.

Ejercicio 3.3. Dada la siguiente expresión de MinHS contesta lo siguiente:

```
let recfun fib n =
    if n < 1 then 1 else fib n -1 +  fib n
        - 2 in
        fib 5
end
```

- Evalúa la expresión de acuerdo a la definición de la máquina $\mathcal{H}$.
- Evalúa la expresión de acuerdo a la definición de la máquina $\mathcal{J}$.
- Los resultados obtenidos son los mismos o son diferentes?.
- Evalúa la expresión utilizando la definición extendida para la máquina $\mathcal{J}$ con cerraduras.
- El resultado obtenido es el mismo?.

Ejercicio 3.4. Dada la siguiente expresión de MinHS contesta lo siguiente:

```
let recfun fac n =
    if n < 1 then 1 else n * fac n - 1 in
        fac 4
end
```

- Evalúa la expresión de acuerdo a la definición de la máquina $\mathcal{H}$.
- Evalúa la expresión de acuerdo a la definición de la máquina $\mathcal{J}$.
- Los resultados obtenidos son los mismos o son diferentes?.
- Evalúa la expresión utilizando la definición extendida para la máquina $\mathcal{J}$ con cerraduras.
- El resultado obtenido es el mismo?.

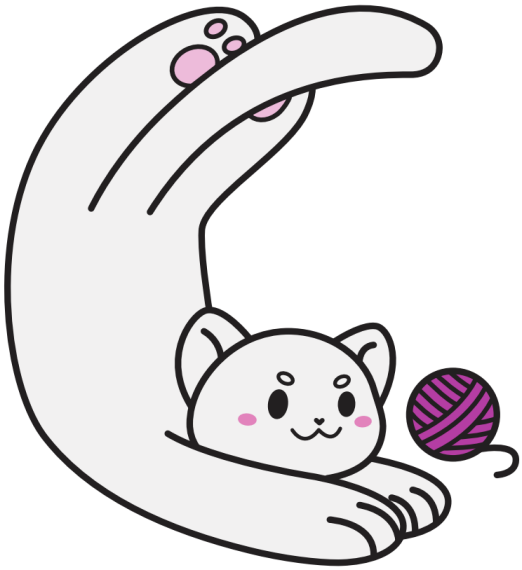
Ejercicio 3.5. Dada la siguiente expresión de MinHS contesta lo siguiente:

```
let x = true in
    let x = false in
        if x then 1 else 0
    end
end
```

- Evalúa la expresión de acuerdo a la definición de la máquina $\mathcal{H}$.
- Evalúa la expresión de acuerdo a la definición de la máquina $\mathcal{J}$.
- Los resultados obtenidos son los mismos o son diferentes?.
- Evalúa la expresión utilizando la definición extendida para la máquina $\mathcal{J}$ con cerraduras.
- El resultado obtenido es el mismo?.

Capítulo 9

TinyC



Hasta este punto en el manual hemos discutido la implementación de diferentes modelos de cómputo como MinHS, que nos fue de utilidad para ilustrar el paradigma funcional. La realidad es que muchos de los lenguajes que utilizamos actualmente no se implementan de esta forma.

Es aquí cuando haremos un cambio abrupto en nuestro caso de estudio para poner nuestra atención en el paradigma procedimental cuyo primer ejemplo será una implementación de un lenguaje de programación basado en C conocido como TinyC. Este tipo de lenguajes requieren del manejo de estados los cuales son afectados por el contexto del programa, los cambios en la memoria y las instrucciones que se ejecutan en un determinado momento.

Objetivo

En este capítulo comenzamos el estudio del paradigma procedimental, para esto nuestro interés principal será el de definir el primer caso de estudio conocido como TinyC. Proporcionando su sintaxis, reglas de transición y manejo de memoria.

Planteamiento

Se revisarán los componentes básicos del lenguaje TinyC, a saber: la sintaxis concreta y la semántica operacional mediante la implementación de la máquina $\mathcal{C}$. proporcionando una definición completa incluyendo marcos, estados y transiciones.

1 Sintaxis

En TinyC tendremos las declaraciones de variables y funciones que se pueden identificar por nombre, las expresiones aritméticas y booleanas para construir la lógica de los programas y por último las sentencias encargadas de la ejecución y control del estado del programa.

Definición 1.1 (Sintaxis Concreta de TinyC). [a].
[b]

$$
\begin{array}{rcl}
\text{progam} & ::= & \text{global-decs}\ \ \text{stmt} \\
\text{global-decs} & ::= & \varepsilon \mid \text{global-dec}\ \ \text{global-decs} \\
\text{global-dec} & ::= & \text{fun-dec} \mid \text{var-dec} \\
\text{var-decs} & ::= & \varepsilon \mid \text{var-dec}\ \ \text{var-decs} \\
\text{var-dec} & ::= & \text{type}\ \ \text{ident} = \text{expr}\ ; \\
\text{fun-dec} & ::= & \text{type}\ \ \text{ident}\ (\text{arguments})\ \ \text{stmt}\ ; \\
\text{stmt} & ::= & \text{expr}\ ; \mid \text{if}\ \ \text{expr}\ \ \texttt{then}\ \ \text{stmt}\ \texttt{else}\ \ \text{stmt}\ ; \mid \text{if}\ \ \text{expr}\ \ \texttt{then}\ \ \text{stmt}; \\
 & & \mid \text{return}\ \ \text{expr}\ ; \mid \{\text{var-decs}\ \ \text{stmts}\} \mid \text{while}\ (\text{expr})\ \ \text{stmt} \\
\text{stmts} & ::= & \varepsilon \mid \text{stmt}\ \ \text{stmts} \\
\text{expr} & ::= & \text{num} \mid \text{ident} \mid \text{asig} \mid \text{expr} + \text{expr} \mid \text{expr}\ - \ \text{expr} \\
 & & \mid \text{expr} > \text{expr} \mid \text{expr}\ < \ \text{expr} \mid \text{ident}\ (\text{exprs}) \\
\text{asig} & ::= & \text{ident} = \text{expr} \\
\text{exprs} & ::= & \text{expr} \mid \text{expr}, \text{exprs} \\
\text{arguments} & ::= & \varepsilon \mid \text{type}\ \ \text{ident}, \text{arguments} \\
\text{type} & ::= & \mathit{Int} \mid \mathit{Bool}
\end{array}
$$

[a] Definición formulada de [12], [126], y [127]

[b] De esta definición es importante notar la omisión entera de apuntadores dado que el tema escapa del enfoque de este capítulo quedándonos únicamente con un subconjunto de expresiones del lenguaje de programación C.

Es importante remarcar el impacto que la introducción de las variables tienen en el estado de un programa para el paradigma procedimental. Dado que los variables representan locaciones de memoria, estas puede ser modificadas alterando así el estado mismo del programa en cualquier punto. Esto no ocurría con la introducción de las variables en la sintaxis

de orden superior para MinHS pues el valor ligado a una variable en este sistema permanece constante.

En TinyC los constructores del lenguaje se pueden agrupar en tres categorías:

- **Declaraciones:** para especificar funciones, y asignar valores a variables.
- **Expresiones:** similares al paradigma funcional en el cual siempre regresan un valor una vez concluida la evaluación de la expresión.
- **Sentencias:** que son los constructores encargados de modelar los efectos del programa en el estado.

Los programas en TinyC son una serie de declaraciones globales seguidas de una sentencia.

Ejercicio 1.1. Escribe un programa en TinyC que implemente una función que nos permita calcular el enésimo número de la sucesión de Fibonacci.

```
Int result = 0;
Int fibonacci(Int n){

    Int i = 1;
    Int pre = 1;
    Int post = 1;
    Int fib = 0;

    while(i < n){
        fib = pre + post;
        pre = post;
        post = fib;
        i++;
    }
    return fib;
};

result = fibonacci(9);
```

2 La Máquina $\mathcal{C}$

Para la semántica operacional de TinyC vamos a definir una máquina abstracta siguiendo la misma línea del capítulo anterior, donde contaremos con una pila para guardar los marcos de ejecución y cómputos pendientes, así como un contexto para las variables y los valores asignados a estas.

2.1 Marcos

En nuestro modelo de cómputo para TinyC vamos a heredar los marcos de las expresiones de la máquina $\mathcal{J}$ dado que los elementos pertenecientes a la categoría *expr* se comportan

de forma análoga regresando un valor al concluir su evaluación.

De esta definición únicamente excluiremos a los marcos para funciones ya que su comportamiento difiere entre ambas máquinas. En $\mathcal{J}$ la declaración se hace mediante funciones anónimas que pueden ir en cualquier parte del programa, mientras que en en $\mathcal{C}$ deben ser declaradas antes de ser llamadas. Adicionalmente estas declaraciones de funciones pueden ser multiparamétricas mientras que en $\mathcal{J}$ solo pueden declarase un argumento a la vez.

Definición 2.1 (Marcos para la pila de control de la máquina $\mathcal{C}$)**.** [a].

Se definen los siguientes marcos que usaremos en la pila de control de la máquina $\mathcal{C}$. Estos marcos corresponden a la evaluación de nuevas instrucciones como return que evalúa el contenido hasta obtener un valor, secu que nos ayuda a escribir una secuencia de instrucciones y call que es el equivalente a app de MinHS.

Declaraciones

$$\frac{}{vardec(T, x, \square)\ marco}$$

Asignaciones

$$\frac{}{asig(x, \square)\ marco}$$

Secuencia

$$\frac{}{secu(\square, e_2)\ marco}$$

Condicionales

$$\frac{}{if(\square, e_2, e_3)\ marco} \qquad \frac{}{if(\square, e_2)\ marco}$$

Return

$$\frac{}{return(\square)\ marco}$$

Llamada a función

$$\frac{}{call(f, \square, e_2, \ldots, e_n)\ marco} \quad \cdots \quad \frac{}{call(f, v_1, v_2, \ldots, \square)\ marco}$$

[a] Definición extraída de [12]

2.2 Estados

Esta categoría introduce una diferencia sustancial con las máquinas abstractas hasta ahora estudiadas. Tendremos una categoría para la pila de cómputos pendientes y dos categorías para el contexto de evaluación de las variables, el primero local representando el alcance de las variables declaradas en el cuerpo de una función y el segundo global para aquellas declaraciones cuyo contexto sea todo el programa.

Definición 2.2 (Estados de la máquina $\mathcal{C}$)**.** [a].

Los estados de la máquina $\mathcal{C}$ son de la siguiente forma:

$$P \mid L \mid G \succ e \qquad\qquad P \mid L \mid G \prec e$$

En donde P es una pila de control, L y G son los ambientes local y global respectivamente y e es una expresión en sintaxis abstracta de TinyC.

[a] Definición formulada de [5], y [8] y [12]

2.3 Ambientes

Los ambientes serán una lista de asignaciones de la forma:

$$\text{variable} \leftarrow \text{valor}$$

A continuación vamos a definir las funciones para hacer referencia al valor de una variable por nombre (búsqueda) y la función para modificar el contenido de una variable (actualización).

Definición 2.3 (Referencia de variable en ambiente para la máquina $\mathcal{C}$). [a].

$$\frac{}{\bullet[x] = \ fail} \qquad \frac{}{x \leftarrow v; \mathbb{E}[x] = v} \qquad \frac{}{y \leftarrow v; \mathbb{E}[x] = \mathbb{E}[x]}$$

Esta función itera sobre la lista de variables contenidas en el ambiente hasta hacer un *match*. Sí la búsqueda llega hasta la pila vacía esta regresa $fail$.

[a] Definición formulada de [5], y [8] y [12]

Definición 2.4 (Modificación del ambiente para una variable en la máquina $\mathcal{C}$). [a].

$$\frac{}{\bullet[x \rightarrowtail v] = fail} \qquad \frac{}{x \leftarrow u; \mathbb{E}[x \rightarrowtail v] = x \leftarrow v; \mathbb{E}}$$

$$\frac{}{y \leftarrow u; \mathbb{E}[x \rightarrowtail v] = \mathbb{E}[x \rightarrowtail v]}$$

Esta función itera sobre la lista hasta hacer un *match* y actualiza el valor contenido en esta variable. Sí la búsqueda llega hasta la pila vacía sin encontrar nada regresamos $fail$.

[a] Definición formulada de [5], y [8] y [12]

2.4 Transiciones

Definición 2.5 (Transiciones de la máquina $\mathcal{C}$). [a]

Las transiciones de la máquina $\mathcal{C}$ se definen en términos de los programas que se están evaluando. Como vimos en secciones anteriores en el caso de TinyC un programa no tiene un resultado final, es decir no se reduce a un valor. Por lo que se agrega un programa

especifico que indica el final de la ejecución, este programa es el programa vacío y se denota como:

$$\perp$$

Este define el final del proceso de evaluación de una sentencia, por lo que los estados finales de la máquina $\mathcal{C}$ son los que tienen la forma siguiente:

$$\diamond \mid L \mid G \prec \perp$$

Se definen las transiciones con las siguientes reglas:

Secuencia

$$\frac{}{P \mid L \mid G \succ secu(e_1, e_2) \rightarrow_C secu(\square, e_2); P \mid L \mid G \succ e_1}$$

$$\frac{}{secu(\square, e_2); P \mid L \mid G \prec \perp \rightarrow_C P \mid L \mid G \succ e_2}$$

Declaraciones

$$\frac{}{P \mid L \mid G \succ vardec(T, x, e) \rightarrow_C vardec(T, x, \square);\ P \mid L \mid G \succ e}$$

$$\frac{G[x] = fail}{vardec(T, x, \square); P \mid L \mid G \prec v \rightarrow_C P \mid L \mid x \leftarrow v; G \prec \perp}$$

$$\frac{}{P|L|G \succ fundec(T, f, x_1 : T_1. \cdots .x_n : T_n.e) \rightarrow_C P|L|f \leftarrow x_1. \cdots .x_n.e; G \prec \perp}$$

Asignación

$$\frac{}{P \mid L \mid G \succ asig(x, e) \rightarrow_C asig(x, \square); P| \ L \mid G \succ e}$$

$$\frac{G[x \rightarrowtail v] = G'}{asig(x, \square); P \mid L \mid G \prec v \rightarrow_C P \mid L \mid G' \prec \perp}$$

$$\frac{G[x \rightarrowtail v] = fail \qquad L[x \rightarrowtail v] = L'}{asig(x, \square); P \mid L \mid G \prec v \rightarrow_C P \mid L' \mid G \prec \perp}$$

Variables

$$\frac{G[x] = v}{P \mid L \mid G \succ x \rightarrow_C P \mid L \mid G \prec v}$$

$$\frac{G[x] = fail \qquad L[x] = v}{P \mid L \mid G \succ x \rightarrow_C P \mid L \mid G \prec v}$$

Condicionales

$$\frac{}{P \mid L \mid G \succ if(e_1, e_2, e_3) \rightarrow_C if(\square, e_2, e_3); P \mid L \mid G \succ e_1}$$

$$\frac{}{if(\square, e_2, e_3); P \mid L \mid G \prec true \rightarrow_C P \mid L \mid G \succ e_2}$$

$$\frac{}{if(\square, e_2, e_3); P \mid L \mid G \prec false \rightarrow_C P \mid L \mid G \succ e_3}$$

$$\frac{}{P \mid L \mid G \succ if(e_1, e_2) \rightarrow_C if(\square, e_2); P \mid L \mid G \succ e_1}$$

$$\frac{}{if(\square, e_2); P \mid L \mid G \prec true \rightarrow_C P \mid L \mid G \succ e_2}$$

$$\frac{}{if(\square, e_2); P \mid L \mid G \prec false \rightarrow_C P \mid L \mid G \prec \bot}$$

While

$$\frac{}{P \mid L \mid G \succ while(e_1, e_2) \rightarrow_C P \mid L \mid G \succ if(e_1, secu(e_2, while(e_1, e_2)))}$$

Esta regla traduce la evaluación de un *while* a un *if* con un solo caso. Modela la siguiente regla equivalencia entre programas:

$$while(e_1)\{e_2\} \equiv if\,(e_1)\,\{\,e_2\,;\,while\,(e_1)\{\,e_2\,\}\}$$

Observe como la expresión *while* sigue apareciendo en el lado derecho. De esta forma se mantiene el ciclo tantas veces como se cumpla la condición.

Llamada a función

$$\frac{}{P \mid L \mid G \succ call(f, e_1, \cdots, e_n) \rightarrow_C call(\square, e_1, \cdots, e_n); P \mid L \mid G \succ f}$$

$$\frac{}{call(\square, e_1, \cdots, e_n); P \mid L \mid G \prec v \rightarrow_C call(v, \square, e_2, \cdots, e_n); P \mid L \mid G \succ e_1}$$

$$\vdots$$

$$\frac{}{call(x_1. \cdots x_n.e, v_1, \cdots, \square); P \mid L \mid G \prec v_n \rightarrow_C P \mid G \bigstar L \mid x_1 \leftarrow v_1; \cdots; x_n \leftarrow v_n; \bullet \succ e}$$

Para la evaluación de una llamada a función, primero es necesario evaluar cada uno de los parámetros con los que se llama.

Una vez que todos son valores, entonces se ejecuta el cuerpo de la función

usando un ambiente vacío como ambiente principal y agregando a el los parámetros de la llamada.

Guardamos el ambiente principal anterior con un símbolo especial ($\bigstar$) que sirve como separador para saber en donde termina uno y comienza el otro.

Return

$$\frac{}{P \mid L \mid G \succ return(e) \rightarrow_C return(\square);\ P \mid L \mid G \succ e}$$

$$\frac{}{return(\square); P \mid G \bigstar L_1 \mid L_2 \prec v \rightarrow_C P \mid L_1 \mid G \prec v}$$

El constructor *return* indica el final de la ejecución de una llamada a función.

Cuando termina la ejecución de una llamada a función. Se restaura el ambiente global y nos deshacemos del local pues solo era necesario dentro del cuerpo de la función.

[a] Definición formulada de [5], y [8]

En este caso, el interés no se centra en el resultado producido en el último paso de la evaluación ($\perp$), sino en el estado final de la memoria. El cálculo no devuelve un valor explícitamente; en su lugar, el estado del programa y por consiguiente el resultado quedan impresos en una de las variables que forman parte del contexto final.

3 Ejercicios para el lector

Ejercicio 3.1. Utilizando la sintaxis definida para TinyC proporciona la definición de un programa que dados dos enteros n y m revise que:

$$n \,\%\, m \;==\; 0$$

Ejercicio 3.2. Utilizando la definición de la máquina $\mathcal{C}$ evalúa el ejercicio anterior con los valores 3 y 9.

Ejercicio 3.3. Utilizando la sintaxis definida para TinyC proporciona la definición de un programa que dado un número n revise su paridad.

Ejercicio 3.4. Utilizando la definición de la máquina $\mathcal{C}$ evalúa el ejercicio anterior con el valor 4

Ejercicio 3.5. Utilizando la sintaxis definida para TinyC proporciona la definición de un programa que dados dos números n y m nos regrese el resultado de n^m.

Ejercicio 3.6. Utilizando la definición de la máquina $\mathcal{C}$ evalúa el ejercicio anterior con los valores 3 y 2.

Ejercicio 3.7. Utilizando la sintaxis definida para TinyC proporciona la definición del algoritmo *Merge Sort*.

Ejercicio 3.8. Utilizando la definición de la máquina $\mathcal{C}$ evalúa el ejercicio anterior con el arreglo [7,5,1,3]

Ejercicio 3.9. Utilizando la sintaxis definida para TinyC proporciona la definición del algoritmo *Binary Search*.

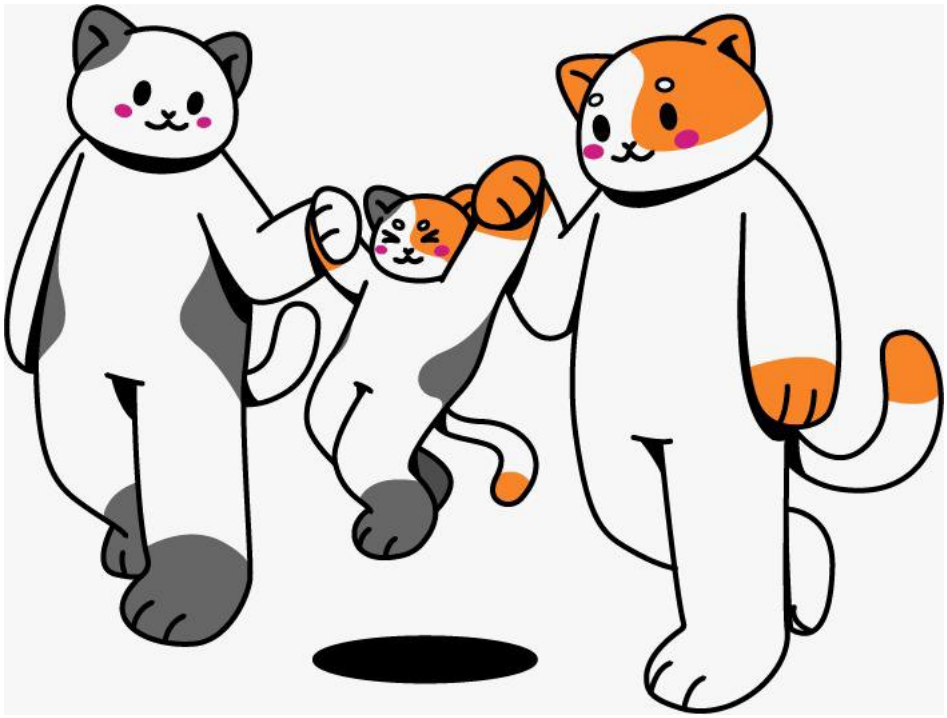
Ejercicio 3.10. Utilizando la definición de la máquina $\mathcal{C}$ evalúa el ejercicio anterior con el arreglo [8,4,1,2,7] y 2.

Ejercicio 3.11. Utilizando la sintaxis definida para TinyC proporciona la definición de un programa que calcule el n-ésimo término de la sucesión de Cataluña.

Ejercicio 3.12. Utilizando la definición de la máquina C evalúa el ejercicio anterior con el número 7

Capítulo 10

Herencia y subtipos



En este capítulo vamos a revisar el último paradigma de programación que estudiaremos en este manual: La orientación a objetos y sus características más importantes como la herencia y los subtipos.

La introducción del paradigma de la orientación a objetos presenta una ventaja al momento de reutilizar código que ya hemos definido anteriormente y que durante los últimos años supuso una revolución en la forma en la que se escriben y desarrollan los programas que corren en nuestros teléfonos y computadoras personales.

Lenguajes como Java, Python, Ruby, Golang, Swift, C# entre muchos otros implementan la herencia, en donde métodos, constructores y variables de la clase padre puede ser reutilizadas por otra clase hija.

Más aún, en este tipo de lenguajes la flexibilidad no solo se extiende a hacer uso de código previamente definido y heredado sino que se puede extender y sobrecargar sobre lo ya existente, proporcionando un mecanismo modular para desarrollar software y evitar la re definición de clases, métodos y variables.

Objetivo

El objetivo de este capítulo es proporcionar una definición de los conceptos principales de la orientación a objetos y su aplicación a la herencia y el subtipificado en el paradigma procedimental para las diferentes categorías definidas en la sintaxis de TinyC.

Planteamiento

En este capítulo se presentan las definiciones principales del paradigma de la orientación a objetos como clase, objeto y subtipo, así como las reglas que nos permiten inferir el tipo de una expresión.

Finalmente se concluye el capítulo introduciendo el concepto de *casting* (en español conversión) junto con los juicios que nos permiten transformar el tipo de una expresión.

1 Orientación a objetos

La orientación a objetos tiene como fundamento la abstracción de los elementos mas importantes de las entidades que se desean modelar en la computadora, su identidad, su comportamiento, relaciones, etc. Para este propósito se utiliza una abstracción conocida como clase.

Definición 1.1 (Definición para clases)**.** [a]
Una clase es una abstracción de una entidad, elemento u objeto que define sus características más importantes como el nombre, los atributos, variables y métodos que describen su comportamiento.

[a] Definición formulada de [128] y [133]

Definición 1.2 (Definición de objeto)**.** [a]
Un objeto es una instancia de una clase.

[a] Definición formulada de [128] y [133]

Las características que se comparten entre los lenguajes que implementan este paradigma incluyen[1]

- **Herencia:** Permite la definición de una clase original dejando que esta sea extendida o sobrecargada por las clases que heredan de ella, constituyendo un mecanismo para reutilizar código previamente escrito y adaptarlo al contexto particular donde se desean

[1] Definición formulada de [133]

instanciar los objetos.

- **Encapsulamiento de datos:** Comprende la seguridad de la información contenida en las variables y métodos de una clase. Permitiendo que solo el objeto mismo sea el encargado de alterar la información contenida en él, proporcionando seguridad y consistencia.

- **Polimorfismo:** Esta característica permite que un objeto que ha heredado de una clase pueda ser identificado por cualquiera de los tipos que posea la cadena de herencia, pudiendo este ser su mismo tipo, o cualquiera de los tipos de la clase o clases padres.

  Esta característica es diferente al polimorfismo que se estudia en lenguajes funcionales, donde por ejemplo la función $\mathsf{id} :: a \to a$ puede ser tipificada con el tipo del argumento que la aplique.

  En Java el polimorfismo se aplica en la sobrecarga de métodos para reutilizar los definidos por la clase que se está extendiendo.

- **Representaciones múltiples:** Permite que dos objetos de una misma clase puedan presentar comportamiento distinto pues este puede cambiar según el estado que posea cada uno.

- **Recursión abierta:** Un objeto es capaz de invocarse a si mismo dentro de la definición de los métodos de su propia clase, utilizando las palabras reservadas como self o this.

2 Subtipos

Los sistemas de tipos que hemos estudiado hasta este momento son restrictivos en el sentido de que muchas operaciones que parecerían intuitivas al desarrollar un programa no serían aceptadas por dicho sistema. Particularmente operaciones como la suma (+) entre los tipos Int y $Float$ por ejemplo.

Necesitamos "relajar" este sistema para tener mas flexibilidad sobre este tipo de casos. En particular necesitamos construir la relación de subtipificado; A $\leq$ B que simboliza: "A es subtipo de B".

Definición 2.1 (Relación de subtipificado). [a]

Reflexividad

$$\frac{}{A \leq A}$$

Transitividad

$$\frac{A \leq B \qquad B \leq C}{A \leq C}$$

Subsunción

$$\frac{\Gamma \vdash e : A \qquad A \leq B}{\Gamma \vdash e : B}$$

Esta propiedad expresa que si una expresión tiene tipo A y A es subtipo de B entonces puede usarse en cualquier contexto en donde sea necesaria una expresión de tipo B.

[a]Definición formulada de [130], [131] y [132]

Ejercicio 2.1. Para este ejemplo consideremos la expresión:

$$5{,}5 + 2$$

Donde podemos observar que $5{,}5 : Float$ y $2 : Int$, suponemos entonces que los tipos $Float$ e Int están relacionados bajo subtipificado como

$$Int \ \leq \ Float$$

por lo que la derivación de tipos de la expresión anterior queda como sigue:

$$\dfrac{\dfrac{}{\varnothing \vdash 5{,}5 : Float} \qquad \dfrac{\dfrac{}{\varnothing \vdash 2 : Int} \qquad Int \leq Float}{\varnothing \vdash 2 : Float}}{\varnothing \vdash 5{,}5 + 2 : Float}$$

Gracias a la propiedad de subsunción podemos reemplazar Int por $Float$ en la derivación de los tipos conservando la propiedad de ser una expresión válida[a].

[a]Ejemplo extraído de [5]

La relación de subtipificado puede ser aplicada a partir de dos interpretaciones diferentes[2]:

- **Interpretación por subconjuntos:** sí A $\leq$ B entonces toda expresión de tipo A también es una expresión de tipo B pues A está contenido en B.

- **Interpretación por coerción:** entendemos coerción como la acción de ejercer presión sobre un objeto para forzar su conducta. Esta acción aplicada al contexto de software nos permite modelar el siguiente comportamiento: sí $s \ : \ A$ y $A \ \leq \ B$ entonces s se puede convertir de forma única a una expresión de tipo B, es decir forzamos el cambio de tipo mediante una conversión explícita.

  En el ejemplo anterior teníamos $2 : Int$ el cuál fue coercido a $2.0 : Float$.

2.1 Subtipificado de tipos primitivos

Las reglas para definir la relación de subtipificado entre los tipos primitivos proporcionados en la definición de los lenguajes de programación, constituyen los axiomas de dicho sistema y con ellos se pude derivar el resto de las reglas utilizando las propiedades de subsunción y transitividad.

Para ilustrar este punto se agrega el tipo $Float$ junto con los axiomas:

Definición 2.2 (Relación de subtipificado para Nat, Int y $Float$)**.** [a]

$$\dfrac{}{Nat \leq Int} \qquad \dfrac{}{Int \leq Float}$$

[2]Definición formulada de [5], [12]

[a]Definición formulada de [130], [131] y [132]

2.2 Subtipificado de funciones

Definamos el siguiente tipo para una función f como $f : Int \to Int$, entonces se tiene que $f\,n : Int$, $\forall\ n : Int$. Aplicando la regla de subsunción obtenemos $f\,n : Float$, y por consiguiente:

$$Int \to Int \leq Int \to Float$$

es una derivación de tipo válida. Es decir la relación de subtipificado original se preserva en el codominio de la función. En tal caso se dice que esta posición es covariante.

Por otro lado si definimos el tipo de nuestra función como $f : Float \to Int$. Tenemos que como todas las expresiones $e : Int$ son también de tipo $Float$ podemos utilizar la relación de subtipificado en sentido opuesto para restringir el dominio (de forma inversa a como se usó en el caso anterior), así podemos introducir elementos de tipo Int en el dominio de f. Por lo que se cumple la siguiente relación de subtipificado:

$$Float \to Int \leq Int \to Int$$

Es decir, la relación de subtipos original $Int \leq Float$ usada en la primera derivación se invierte para este caso, decimos entonces que este argumento es contravariante.

Finalmente, usando la propiedad de transitividad tenemos la siguiente relación de subtipificado

$$Float \to Int \leq Int \to Int \leq Int \to Float$$

De esta forma podemos definir la regla de subtipificado para el caso general de los tipos función como sigue[3].

Definición 2.3 (Definición para subtipificación de funciones)**.** [a]

$$\frac{T_1 \leq S_1 \qquad S_2 \leq T_2}{S_1 \to S_2 \leq T_1 \to T_2}$$

[a]Definición formulada de [5], [12], [131] y [132]

2.3 Subtipificado para suma y producto

Para esta aplicación la relación de subtipificado se comporta como contravariante, es decir la dirección de la relación se preserva como es de esperar, de tal forma que se introducen las siguientes reglas al sistema de tipos.

Definición 2.4 (Reglas para subtipificar expresiones aritméticas)**.** [a]

$$\frac{S_1 \leq T_1 \qquad S_2 \leq T_2}{S_1 + S_2 \leq T_1 + T_2} \qquad\qquad \frac{S_1 \leq T_1 \qquad S_2 \leq T_2}{S_1 \times S_2 \leq T_1 \times T_2}$$

[a]Definición formulada de [5], [12], [131] y [132]

[3]fragmento extraído de [5]

2.4 Subtipificado para registros

Los registros, como hemos estudiado en secciones anteriores, son la generalización de una tupla permitiendo extender el concepto para n elementos. Asociadas a estos existen las proyecciones para recuperar el n-ésimo elemento contenido en esta estructura.

Veremos brevemente las características asociadas al tipo de los registros una vez se introduce la relación de subtipificado.

Definición 2.5 (Definición de la relación de subtipificado aplicado a los tipos registros). [a]

Amplitud Dados dos registros A y B donde A posee una cantidad mayor de elementos que B, decimos que el tipo del registro A es subtipo del registro B representado de la siguiente forma:

$$\frac{}{(l_1 : T_1, \ldots, l_{n+k} : T_{n+k}) \leq (l_1 : T_1, \ldots, l_n : T_n)}$$

Profundidad Esta propiedad es la aplicación de la covarianza de la relación de subtipificado, aplicándose a cada uno de los campos contenidos en el registro:

$$\frac{T_1 \leq S_1 \quad \cdots \quad T_n \leq S_n}{(l_1 : T_1, \ldots, l_n : T_n) \leq (l_1 : S_1, \ldots, l_n : S_n)}$$

Permutación El orden de los campos de un valor de tipo registro no importa.

$$\frac{(s_1 : S_1, \ldots, s_n : S_n) \text{ permutación de } (l_1 : T_1, \ldots, l_n : T_n)}{(s_1 : S_1, \ldots, s_n : S_n) \leq (l_1 : T_1, \ldots, l_n : T_n)}$$

[a] Definición formulada de [5], [12], [131] y [132]

2.5 Elementos máximos

En nuestra implementación para el sistema de tipos es importante contar con un elemento máximo, en donde todo elemento distinto al elemento máximo es subtipo de este. Aquí introducimos Top que cumple exactamente con esta característica representada de la siguiente forma:

Definición 2.6 (Definición del elemento *Top* para el sistema de subtipificado). [a]

$$\frac{}{T \leq Top}$$

[a] Definición formulada de [5], [12]

La idea detrás de Top, es que a este tipo pertenecen todos aquellos programas que están correctamente tipificados. En Java a este tipo se le conoce como *Object*.

El contrario, es un tipo que es subtipo de todos y se conoce como Bot, este es inhabita-

do, ninguna expresión debe de existir dentro del mismo. Este tipo ayuda a expresar aquellos programas que no deben de regresar ningún valor, similar a la forma en la que se utiliza el tipo *void* en Java.

La descripción anteriormente dada para *Bot* se representa de la siguiente forma:

Definición 2.7 (Definición del elemento *Bot* para el sistema de subtipificado). [a]

$$\frac{}{Bot \leq T}$$

[a] Definición formulada de [5], [12]

3 Conversión de tipos

En lenguajes de programación como Java hemos hecho uso de este mecanismo para cambiar el tipo de un dato o variable por otro.

Similar al ejemplo que tenemos para pasar de $2 : Int$ a $2.0 : Float$. La relación de subtipificado nos provee la regla de subsunción. Sin embargo definiremos el mecanismo para ambos sentidos de la relación conocidos como conversión ascendente (*Upcasting*) y conversión descendente (*Downcasting*).

3.1 Conversión ascendente *(Upcasting)*

En este tipo de casting (también conocido como ”casting hacia arriba”) el objetivo es que al tipificar un término, a este se le atribuye un supertipo esperado.

De esta forma para los subtipos donde $T \leq R$ entonces usando subsunción podemos concluir que $\Gamma \vdash \langle R \rangle\, e : R$, en donde $\langle R \rangle$ es la notación para el *casting* explícito, representado como:

Definición 3.1 (Definición para Conversión ascendente). [a]:

$$\frac{\Gamma \vdash e : T \qquad T \leq R}{\Gamma \vdash \langle R \rangle\, e : R}$$

[a] Definición formulada de [5], [12], [131] y [132]

3.2 Conversión descendente *(Downcasting)*

En este tipo de casting (también conocido como ”casting hacía abajo”) la asignación de tipos es arbitraria, la relación de subtipificado no necesariamente se tiene que cumplir.

Esto supone una amenaza a la consistencia de tipos en el programa que se escribe con esta instrucción, el lenguaje está forzado a aceptar el *downcasting* en tiempo de compilación pero en tiempo de ejecución un futuro conjunto de verificaciones deben de ser ejecutados para corroborar que el tipo asignado no supone un error. Esta regla se representa de la siguiente manera:

Definición 3.2 (Definición para conversión descendente)**.** [a]:

$$\frac{\Gamma \vdash e : T}{\Gamma \vdash \langle R \rangle\, e : R}$$

[a] Definición formulada de [5], [12], [131] y [132]

4 Ejercicios para el lector

Ejercicio 4.1. Dada la siguiente expresión, utiliza las reglas de tipificado para suma y producto para derivar el tipo de la misma.

$$4.0 \;-\; 1 \;<\; 3.0 \;:\; \mathit{Float}$$

Ejercicio 4.2. Dada la siguiente expresión, utiliza las reglas de tipificado para suma y producto para derivar el tipo de la misma.

$$3 \;+\; 3.0 \;+\; 4 \;-\; 4 \;:\; \mathit{Float}$$

Ejercicio 4.3. Dada la siguiente expresión, utiliza las reglas de tipificado para el operador *if* y deriva el tipo de la misma.

$$\mathsf{if}\ 7 \;*\; 0.0 < 0\ \mathsf{then}\ 1\ \mathsf{else}\ 0 : \;\mathit{Float}$$

Ejercicio 4.4. Dada la siguiente expresión, utiliza las reglas de tipificado para el operador fun y deriva el tipo de la misma.

$$\mathsf{fun}(x : \mathsf{Int} \rightarrow\; x + 1\ ) : \mathit{Int} \rightarrow \mathit{Float}$$

Ejercicio 4.5. Utiliza la regla de *Upcasting* para obtener el tipo de la siguiente expresión basado en las reglas para tipos primitivos.

$$\mathsf{fun}(x : \mathsf{Int} \rightarrow \mathsf{if}\ x < 0\ \mathsf{then}\ x - 1.0\ \mathsf{else}\ x) : \mathit{Int} \rightarrow \mathit{Int}$$

Ejercicio 4.6. Utiliza las reglas de elementos máximos para obtener la derivación del tipificado para la siguiente expresión. Si no es posible argumenta por qué.

$$\mathsf{fun}(x : \mathsf{Int} \rightarrow \mathsf{if}\ x \leq 0\ \mathsf{then}\ x - 1.0\ \mathsf{else}\ x) : \mathit{Bot}$$

Ejercicio 4.7. Utiliza las reglas de elementos máximos para obtener la derivación del tipificado para la siguiente expresión. Si no es posible argumenta por qué.

$$\mathsf{fun}(x : \mathsf{Int} \rightarrow \mathsf{if}\ x \leq 0\ \mathsf{then}\ x - 1.0\ \mathsf{else}\ x) : \mathit{Top}$$

Capítulo 11

Java Peso Pluma

El presente capítulo constituye el final de este manual, aquí proporcionaremos la última implementación que revisaremos de un lenguaje orientado a objetos para estudiar las propiedades listadas en el capítulo anterior. Para ello vamos a definir al lenguaje orientado a objetos Java Peso Pluma (en inglés a este lenguaje se le conoce como *Featherweight Java*), que contiene un subconjunto de expresiones del lenguaje de programación Java.

Estudiar lenguajes de programación que no pertenecen al paradigma funcional es una tarea no trivial, dado que la compaginación entre la sintaxis, semántica y manejo de la memoria se implementa de forma distinta en comparación a MinHS donde el mapeo es uno a uno entre la definición y la implementación.

Objetivo

Estudiar la implementación de Java Peso Pluma definiendo y discutiendo sus características mas importantes como: herencia, subtipificado, clases, objetos así como la sintaxis del lenguaje junto con la semántica estática y dinámica.

Planteamiento

Siguiendo la misma estructura de los capítulos anteriores comenzaremos el estudio de Java Peso Pluma proporcionando la definición para la sintaxis concreta de los programas que escribiremos en este lenguaje.

Discutiremos la semántica dinámica para evaluarlos, la semántica estática para explorar el sistema de tipos, su interacción con la herencia, la implementación de métodos y la relación de subtipificado.

Por último se estudiará brevemente la propiedad de seguridad aplicada en este lenguaje.

1 Sintaxis de Java Peso Pluma

A continuación presentamos la definición de Java Peso Pluma en donde se discuten las categorías principales de los lenguajes orientados a objetos: clases, instancias, métodos, atributos, herencia, polimorfismo y subtipificado junto con la recursión abierta.

Definición 1.1 (Sintaxis de Java Peso Pluma)**.** Se presenta la sintaxis del lenguaje separado en categorías y usando las siguientes meta-variables[a]

- Nombres de variables: `x,y,z`
- Nombres de atributos: `f,g`
- Nombres de clases: `C,A,B`
- Nombres de métodos: `m`

Expresiones

$$
\begin{array}{rcll}
\mathtt{e} & ::= & \mathtt{x} & \text{Variables} \\
 & | & \mathtt{e.f} & \text{Acceso a atributo} \\
 & | & \mathtt{e.m}(\vec{\mathtt{e}}) & \text{Invocación de método} \\
 & | & \mathtt{new}\ \mathtt{C}(\vec{\mathtt{e}}) & \text{Instancia de objeto} \\
 & | & (\mathtt{C})\ \mathtt{e} & \text{Casting}
\end{array}
$$

Valores

$$
\mathtt{v} \quad ::= \quad \mathtt{new}\ \mathtt{C}(\vec{\mathtt{v}}) \quad \text{Instancia de objeto}
$$

Métodos y Clases

$$
\begin{array}{rcll}
\mathtt{K} & ::= & \mathtt{C}\,(\vec{\mathtt{C}}\,\vec{\mathtt{x}})\ \{\ \mathtt{super}(\vec{\mathtt{x}});\mathtt{this}.\vec{\mathtt{f}} = \vec{\mathtt{x}}\ \} & \text{Constructores} \\
\mathtt{M} & ::= & \mathtt{C}\ \mathtt{m}\,(\vec{\mathtt{C}}\,\vec{\mathtt{x}})\ \{\ \mathtt{return}\ \mathtt{e};\ \} & \text{Métodos} \\
\mathtt{L} & ::= & \mathtt{class}\ \mathtt{C}\ \mathtt{extends}\ \mathtt{B}\ \{\ \vec{\mathtt{A}}\,\vec{\mathtt{f}}\,;\ \mathtt{K}\,\vec{\mathtt{M}}\ \} & \text{Clases}
\end{array}
$$

En las definiciones dadas previamente hacemos abuso de notación donde $\vec{\mathtt{t}}$ se conoce como notación vectorial y se interpreta como:

$$\vec{\mathtt{t}} =_{def} \mathtt{t_1}, \mathtt{t_2}, \ldots, \mathtt{t_n}$$

$\vec{t}$ representa entonces una sucesión de n términos indexados.

Cuando se tienen dos vectores sin separación, esta notación se interpreta como el intercalado entre un valor del primero y uno del segundo separando el siguiente par de elementos por una coma:

$$\vec{\mathtt{C}}\,\vec{\mathtt{x}} =_{def} \mathtt{C_1}\,\mathtt{x_1}, \mathtt{C_2}\,\mathtt{x_2}, \ldots, \mathtt{C_n}\,\mathtt{x_n}$$

Para los constructores los vectores de atributo y valores se pueden escribir de la siguiente forma:

$$\mathtt{this}.\vec{\mathtt{f}} = \vec{\mathtt{x}} =_{def} \mathtt{this}.\mathtt{f_1} = \mathtt{x_1}; \mathtt{this}.\mathtt{f_2} = \mathtt{x_2}; \ldots; \mathtt{this}.\mathtt{f_n} = \mathtt{x_n}$$

Donde la correspondencia entre cada atributo y valor es uno a uno, de manera implícita denotando la misma cardinalidad en ambos vectores.

[a] Definición formulada de [128] y [129]

Esta definición presenta algunas restricciones que no existen en Java:

- Las clases siempre extienden a una super-clase (puede ser Object).
- Los constructores se declaran explícitamente.
- Los constructores siempre llaman al constructor de la super-clase que se hereda mediante le instrucción super().
- El objeto el cuál hace referencia a sus atributos tiene que ser enunciado de forma explícita.
- Los métodos están constituidos únicamente por expresiones return.
- Los constructores solo pueden definir al constructor trivial de la clase, recibiendo y asignando un valor por cada atributo de la clase.
- Solo puede existir un único constructor por clase.
- En la definición de clase los atributos solo pueden ser declarados sin asignar ningún valor.

Revisemos la sintaxis concreta de Java Peso Pluma con los siguientes ejemplos.

Ejercicio 1.1. Proporciona la definición de una clase para definir la dirección de una persona, puedes dar por definida las clases String y Integer

```
class Direccion extends Object {
    String calle;
    String colonia;
    String ciudad;
    Integer numero;
    Integer telefono;

    Direccion(String calle, String colonia,
        String ciudad, Integer numero, Integer telefono){
        super();
        this.calle = calle;
        this.colonia = colonia;
        this.ciudad = ciudad;
        this.numero = numero
    }
}
```

Ejercicio 1.2. Proporciona la definición de una clase TrianguloRec que modele un triángulo rectángulo y calcule su perímetro (puedes suponer la existencia de la clase Integer incluyendo operaciones aritméticas).

```
class TrianguloRec extends Object {
    Integer adyacente;
    Integer opuesto;
    Integer hipotenusa;

    TrianguloRec(Integer adyacente, Integer opuesto, Integer hipotenusa){
        super();
        this.adyacente = adyacente;
        this.opuesto = opuesto;
        this.hipotenusa = hipotenusa;
    }

    Integer perimetro (){
        return this.adyacente + this.opuesto + this.hipotenusa;
    }
}
```

Ejercicio 1.3. Proporciona la definición de una clase MinHeap que modele una estructura de datos de tipo heap mínimo. Puedes suponer la existencia de la clase Integer que permite comparar valores.

```
class HeapNode extends Object {
 Integer value;
 HeapNode left;
 HeapNode right;

 HeapNode(Integer value, HeapNode left, HeapNode right){
  super();
  this.value = value;
  this.left = left;
  this.right = right;
 }

 Integer getValue(){
  return this.value;
 }

 HeapNode getLeft(){
  return this.left;
 }

 HeapNode getRight(){
  return this.right;
 }
}

class MinHeap extends Object {
 HeapNode root;

 MinHeap(HeapNode root){
  super();
  this.root = root;
 }

 HeapNode getRoot(){
  return this.root;
 }

 Integer findMin(){
  return this.root.getValue();
 }
}
```

2 Tablas de clase

Los programas de Java Peso Pluma son pares constituidos por una expresión e y una tabla de clases T cuya generación no es explícita.

Como su nombre indica, esta tabla contiene la información especificada en la declaración de clase. con esta podemos recuperar sus atributos, la firma de los métodos y su definición.

Definición 2.1 (Tabla de clase)**.** Una tabla de clase es una función finita que mapea el nombre de una clase con su definición[a]. Es decir T es una sucesión finita de declaraciones de la forma

$$T(\mathtt{C}) = \mathtt{class\ C\ extends\ B}\ \{\ \vec{\mathtt{C}}\,\vec{\mathtt{f}}\ ;\ \mathtt{K}\ \vec{\mathtt{M}}\ \}$$

En donde $\vec{\mathtt{C}}\,\vec{\mathtt{f}}$ representa los vectores del tipo de la variable de clase y su nombre y $\vec{\mathtt{M}}$ representa la lista de métodos definidos para la clase C de la forma $\mathtt{B\ m}\,(\tilde{\mathtt{B}}, \tilde{\mathtt{x}})\ \{\ \mathtt{return\ e;}\}$

[a]Definición formulada de [5], [12] y [129]

Definición 2.2 (Operaciones de búsqueda para la tabla de clase)**.** A continuación enunciamos las reglas para retribuir elementos de la tabla de clase[a]

Búsqueda de Atributos

$$\frac{}{fields(object) = \varnothing} \qquad \frac{\begin{array}{c} T(\mathtt{C}) = \mathtt{class\ C\ extends\ B}\ \{\ \vec{\mathtt{C}}\,\vec{\mathtt{f}}\ ;\ \mathtt{K}\ \vec{\mathtt{M}}\ \} \\ fields(\mathtt{B}) = \vec{\mathtt{B}}\,\vec{\mathtt{g}} \end{array}}{fields(\mathtt{C}) = \vec{\mathtt{B}}\,\vec{\mathtt{g}}, \vec{\mathtt{C}}\,\vec{\mathtt{f}}}$$

Búsqueda del tipo de un método

$$\frac{\begin{array}{c} T(\mathtt{C}) = \mathtt{class\ C\ extends\ D}\ \{\ \vec{\mathtt{C}}\,\vec{\mathtt{f}}\ ;\ \mathtt{K}\ \vec{\mathtt{M}}\ \} \\ \mathtt{B\ m}\,(\vec{\mathtt{B}}, \vec{\mathtt{x}})\ \{\ \mathtt{return\ e;}\ \}\ \text{figura en}\ \vec{\mathtt{M}} \end{array}}{mtype(\mathtt{m}, \mathtt{C}) = \vec{\mathtt{B}} \to \mathtt{B}} \qquad \frac{\begin{array}{c} T(\mathtt{C}) = \mathtt{class\ C\ extends\ D}\ \{\ \vec{\mathtt{C}}\,\vec{\mathtt{f}}\ ;\ \mathtt{K}\ \vec{\mathtt{M}}\ \} \\ \mathtt{m}\ \text{no figura en}\ \vec{\mathtt{M}} \end{array}}{mtype(\mathtt{m}, \mathtt{C}) = mtype(\mathtt{m}, \mathtt{D})}$$

Búsqueda del cuerpo de un método

$$\frac{\begin{array}{c} T(\mathtt{C}) = \mathtt{class\ C\ extends\ D}\ \{\ \vec{\mathtt{C}}\,\vec{\mathtt{f}}\ ;\ \mathtt{K}\ \vec{\mathtt{M}}\ \} \\ \mathtt{B\ m}\,(\vec{\mathtt{B}}, \vec{\mathtt{x}})\ \{\ \mathtt{return\ e;}\ \}\ \text{figura en}\ \vec{\mathtt{M}} \end{array}}{mbody(\mathtt{m}, \mathtt{C}) = (\vec{\mathtt{x}}, \mathtt{e})} \qquad \frac{\begin{array}{c} T(\mathtt{C}) = \mathtt{class\ C\ extends\ D}\ \{\ \vec{\mathtt{C}}\,\vec{\mathtt{f}}\ ;\ \mathtt{K}\ \vec{\mathtt{M}}\ \} \\ \mathtt{m}\ \text{no figura en}\ \vec{\mathtt{M}} \end{array}}{mbody(\mathtt{m}, \mathtt{C}) = mbody(\mathtt{m}, \mathtt{D})}$$

[a]Definición formulada de [5], [12] y [128]

3 Semántica dinámica

La semántica operacional de Java Peso Pluma se representa mediante la aplicación de la relación $\to_{\mathtt{fj}}$ y está basada en la tabla de clase para llevar acabo la evaluación. Aún cuando la utilización de la tabla de clase sea de forma implícita siempre está presente en la aplicación de cada una de las reglas.

En la definición de Java Peso Pluma los únicos valores permitidos son aquellos objetos ins-

tanciados usando la instrucción new, esto se verá reflejado en la manera en la que las reglas interactúan con dicha instancia de clase.

Definición 3.1 (Semántica Operacional de Java Peso Pluma)**.** Presentamos entonces las reglas de evaluación para el lenguaje como sigue[a]:

Selección de Atributos

$$\frac{fields(\mathtt{C}) = \vec{\mathtt{C}}\,\vec{\mathtt{f}}}{(\mathtt{new}\,\mathtt{C}\,(\vec{\mathtt{v}}))\mathtt{.f_i} \rightarrow_{\mathtt{fj}} \mathtt{v_i}}$$

Invocación de métodos

$$\frac{mbody(\mathtt{m}, \mathtt{C}) = (\vec{\mathtt{x}}, \mathtt{e})}{(\mathtt{new}\,\mathtt{C}\,(\vec{\mathtt{v}}))\mathtt{.m}(\vec{\mathtt{w}}) \rightarrow_{\mathtt{fj}} e[\vec{\mathtt{x}}, \mathtt{this} := \vec{\mathtt{w}}, (\mathtt{new}\,\mathtt{C}\,(\vec{\mathtt{v}}))]}$$

Casting

$$\frac{\mathtt{C} \leq \mathtt{D}}{(\mathtt{D})\ (\mathtt{new}\,\mathtt{C}\,(\vec{\mathtt{v}})) \rightarrow_{\mathtt{fj}} \mathtt{new}\,\mathtt{C}\,(\vec{\mathtt{v}})}$$

Reglas de congruencia Siguiendo el estándar de evaluación de derecha a izquierda tenemos las siguientes reglas de evaluación para expresiones:

$$\frac{\mathtt{e} \rightarrow_{\mathtt{fj}} \mathtt{e}'}{\mathtt{e.f} \rightarrow_{\mathtt{fj}} \mathtt{e}'\mathtt{.f}}$$

$$\frac{\mathtt{e} \rightarrow_{\mathtt{fj}} \mathtt{e}'}{\mathtt{e.m}(\vec{\mathtt{e}}) \rightarrow_{\mathtt{fj}} \mathtt{e}'\mathtt{.m}(\vec{\mathtt{e}})}$$

$$\frac{\mathtt{e_i} \rightarrow_{\mathtt{fj}} \mathtt{e_i'}}{\mathtt{e.m}(\ldots, \mathtt{e_i}, \ldots) \rightarrow_{\mathtt{fj}} \mathtt{e.m}(\ldots, \mathtt{e_i'}, \ldots)}$$

$$\frac{\mathtt{e_i} \rightarrow_{\mathtt{fj}} \mathtt{e_i'}}{\mathtt{new}\ \mathtt{C}(\ldots, \mathtt{e_i}, \ldots) \rightarrow_{\mathtt{fj}} \mathtt{new}\ \mathtt{C}(\ldots, \mathtt{e_i'}, \ldots)}$$

$$\frac{\mathtt{e} \rightarrow_{\mathtt{fj}} \mathtt{e}'}{(\mathtt{C})\ \mathtt{e} \rightarrow_{\mathtt{fj}} (\mathtt{C})\ \mathtt{e}'}$$

[a] Definición formulada de [5], [12] y [129]

4 Semántica estática

El sistema de tipos de Java Peso Pluma no cuenta con tipos primitivos, es decir que todos los tipos con los que trabajemos tendrán que ser definidos mediante su respectiva clase. La única excepción a esta regla es el tipo *Object*.

En este lenguaje *Top* será *Object*, el único tipo primitivo con el cual podemos definir construcciones más complejas. Este constituye también una palabra reservada en la definición que propusimos de la sintaxis.

Definición 4.1 (Semántica Estática de Java Peso Pluma)**.** A continuación enunciamos las reglas de tipificado[a]

Reglas de Subtipificado

$$\frac{T(\mathtt{C}) = \mathtt{class\ C\ extends\ D}\ \{\ \vec{\mathtt{C}}\,\vec{\mathtt{f}}\,;\ \mathtt{K}\,\vec{\mathtt{M}}\ \}}{\mathtt{C} \leq \mathtt{D}}$$

Es decir, si la clase `C` extiende a la clase `D` entonces `C` es subtipo de `D`.
La relación de subtipificado debe cumplir las propiedades de transitividad y reflexividad discutidas en el capítulo anterior.

Reglas de Tipificado

$$\frac{}{\Gamma, x : C \vdash x : C} \qquad \text{Variables}$$

$$\frac{\Gamma \vdash e : C \qquad fields(C) = \vec{C}\,\vec{\mathtt{f}}}{\Gamma \vdash e.\mathtt{f_1} : \mathtt{C_i}} \qquad \text{Acceso a atributos}$$

$$\frac{\Gamma \vdash e_0 : C_0 \qquad \Gamma \vdash \vec{e} : \vec{C} \qquad mtype(\mathtt{m}, C_0) = \vec{D} \to C \qquad \vec{C} \leq \vec{D}}{\Gamma \vdash e_0.\mathtt{m}(\vec{\mathtt{e}}) : \mathtt{C}} \qquad \text{Invocación de métodos}$$

$$\frac{\Gamma \vdash \vec{e} : \vec{C} \qquad fields(C) = \vec{D}\,\vec{\mathtt{f}} \qquad \vec{C} \leq \vec{D}}{\Gamma \vdash \mathtt{new\ C}(\vec{e}) : C} \qquad \text{Creación de objetos}$$

Reglas de casting

$$\frac{\Gamma \vdash e : D \qquad D \leq C}{\Gamma \vdash (C)\ e : C} \qquad \textit{Upcasting}$$

$$\frac{\Gamma \vdash e : D \qquad C \leq D \qquad C \neq D}{\Gamma \vdash (C)\ e : C} \qquad \textit{Downcasting}$$

$$\frac{\Gamma \vdash e : D \qquad C \not\leq D \qquad D \not\leq C \qquad \textsf{stupid warning}}{\Gamma \vdash (C)\ e : C} \qquad \textit{Casting}\ \text{estúpido}$$

Esta última regla es un tecnicismo necesario para poder probar la preservación de tipos con la semántica operacional estructural.

Formación de Clases

Introducimos los juicios $\mathtt{M}$ ok in C para indicar que el método `M` está bien formado en la clase C, C *ok* que indica que la clase C está bien formada y el juicio T *ok* para decir que la tabla de clases T está bien formada.

$$\frac{\begin{array}{c}\mathtt{K} = \mathtt{C}\,(\vec{\mathtt{D}}\,\vec{\mathtt{y}}, \vec{\mathtt{C}}\,\vec{\mathtt{x}})\ \{\ \mathtt{super}(\vec{\mathtt{y}}); \mathtt{this}.\vec{\mathtt{f}} = \vec{\mathtt{x}}\ \} \\ fields(D) = \vec{D}\ \vec{\mathtt{g}} \\ \vec{\mathtt{M}}\ ok\ \textsf{in}\ C\end{array}}{\mathtt{class\ C\ extends\ D}\ \{\ \vec{\mathtt{C}}\,\vec{\mathtt{f}}\,;\ \mathtt{K}\,\vec{\mathtt{M}}\ \}\ ok}$$

Formación de métodos

$$\frac{\begin{array}{c}T(C) = \mathtt{class\ C\ extends\ D}\,\{\ldots\} \\ mtype(\mathtt{m}, D) = \vec{C} \to C_0 \\ \vec{x} : \vec{C}, \mathtt{this} : C \vdash \mathtt{e} : C_0' \\ C_0' \leq C_0\end{array}}{C_0\ \mathtt{m}\,(\vec{C}\,\vec{x})\{\ \mathtt{return\ e;}\ \}\ ok\ \textsf{in}\ C}$$

Formación de Tablas

Decimos que una tabla de clases está bien formada si todas las clases declaradas en ella están bien formadas, modelado con la regla:

$$\frac{\forall C \in \mathsf{dom}(T), T(C)\ ok}{T\ ok}$$

[a]Definición formulada de [5], [12], [128], [129] y [132]

5 Propiedades de Java Peso Pluma

Como hemos acostumbrado en el estudio de cada uno de los lenguajes definidos en este manual brevemente mencionaremos las dos propiedades que más interesan en este curso: progreso de la función $\rightarrow_{\mathtt{fj}}$ y preservación de tipos.

5.1 Preservación de tipos

La preservación de tipos que hemos estudiado en implementaciones anteriores difiere en que ahora tenemos que considerar su relación con el mecanismo de herencia y los *castings* que no son correctos. Esto se puede observar en la siguiente regla.

Proposición 5.1 (Preservación de tipos)**.** Si T es una tabla de clases bien formada, $\Gamma \vdash e : C$ y $e \rightarrow_{\mathtt{fj}} e'$, entonces existe C' tal que $C' \leq C$ y $\Gamma \vdash e' : C'$[a]

[a]Definición formulada de [12] y [128]

5.2 Progreso

Esta propiedad tiene un pequeño detalle cuando se utiliza dentro del contexto de Java Peso Pluma, En general la propiedad es válida salvo en el caso único en el que no se puede computar un downcasting.

Proposición 5.2 (Progreso de la relación $\rightarrow_{\mathtt{fj}}$)**.** Sea T una tabla de clases bien formada, si $\varnothing \vdash e : C$ entonces sucede una y solo una de las siguientes condiciones:

- e es un valor.
- e contiene una expresión de la forma $(C)\ \mathtt{new}\ D(\vec{\mathtt{v}})$ en donde $D \leq C$, es decir, no se puede realizar el *downcast*.
- Existe e' tal que $e \rightarrow_{\mathtt{fj}} e'$[a].

[a]Definición formulada de [12] y [128]

5.3 Seguridad

Combinando las dos propiedades anteriores obtenemos la relga de seguridad, esta enuncia que dada una expresión e de Java Peso Pluma se obtiene una expresión en su forma normal, valor o bien encontramos un *downcasting* incorrecto.

Proposición 5.3 (Seguridad de Java Peso Pluma)**.** Si T es una tabla de clases bien formada, $\varnothing \vdash e : C$ y $e \rightarrow^{\star}_{\mathtt{fj}} e'$ con e' en forma normal, entonces se cumple una y solo una de las siguientes condiciones:

- e' es un valor v tal que $\varnothing \vdash v : D$ y $D \leq C$.
- e' contiene como subexpresión (C) `new` $D(\vec{\mathtt{v}})$ en donde $D \leq C$.

Dada cualquier expresión e, o bien esta eventualmente llega a ser evaluada como un valor o se bloquea en un *downcasting* que no se puede resolver[a].

[a] Definición formulada de [12] y [128]

La demostración de cada propiedad queda fuera del alcance de este manual pero pueden ser consultadas de [128].

6 Cómo se relaciona Java Peso Pluma con Java?

Java Peso Pluma es un lenguaje con propósitos ilustrativos, muchas de las características que robustecen a Java y que son englobadas por el paradigma de la orientación a objetos se dejan fuera del enfoque de la definición del mismo.

Es natural entonces preguntarse por la relación que guardan ambos lenguajes. Qué sucede cuando escribimos un programa en Java Peso Pluma y lo queremos ejecutar en la JVM[1].

Para dar respuesta a esta pregunta presentamos las siguientes proposiciones que hacen explícita la correspondencia entre ambos lenguajes.

- Cada programa sintácticamente correcto en Java Peso Pluma es también sintácticamente correcto en Java.
- Un programa sintácticamente correcto es tipificable en Java Peso Pluma si y sólo si es tipificable en Java.
- La ejecución de un programa bien tipificado en Java Peso Pluma se comporta de la misma forma en Java.
- La evaluación de un programa en Java Peso Pluma no termina si y sólo si compilarlo y ejecutarlo en Java causa no terminación.

La demostración de estas últimas cuatro proposiciones no es posible dado que no hay una formalización de Java que nos permita razonar sobre si mismo, sin embargo enunciar estas proposiciones ilustra la importancia de estudiar los lenguajes de programación formalmente para inferir propiedades y características deseadas bajo un modelo de razonamiento lógico.

7 Ejercicios para el lector

Ejercicio 7.1. Dada la definición de Java Peso Pluma proporciona un programa para modelar árboles binarios cuyo contenido de sus nodos sean enteros (Puedes dar por definido el tipo Integer).

[1] Java Virtual Machine.

Ejercicio 7.2. Dada la definición de Java Peso Pluma proporciona un programa para modelar Tries junto con las operaciones para insertar y buscar letras (Puedes dar por definido el tipo Char).

Ejercicio 7.3. Dada la definición de Java Peso Pluma proporciona un programa para modelar LinkedList junto con las operaciones para insertar, eliminar y obtener elementos dado un índice.

Ejercicio 7.4. Dada la definición de Java Peso Pluma proporciona un programa para modelar un nuevo objeto llamado Empleado que contenga un campo de clase para su dirección, su número de seguridad social (NSS) y su salario (puedes dar por definidas las clases Integer, String y Direccion del ejemplo 1.2).

Ejercicio 7.5. Dadas las reglas para la sintaxis y las reglas de la semántica dinámica y estática de Java Peso Pluma, Demuestra la proposición 5.1: Preservación de tipos.

Ejercicio 7.6. Dadas las reglas para sintaxis y las reglas de la semántica dinámica y estática de Java Peso Pluma, Demuestra la proposición 5.2: Progreso.

Ejercicio 7.7. Dadas las reglas para sintaxis y las reglas de la semántica dinámica y estática de Java Peso Pluma, Demuestra la proposición 5.3: Seguridad.

Conclusión

A lo largo de este manual se presentaron los conceptos fundamentales que permiten estudiar a los lenguajes de programación desde una perspectiva formal. Más allá de entenderlos únicamente como herramientas prácticas para escribir programas, el objetivo fue analizarlos como estructuras bien definidas que pueden describirse mediante modelos matemáticos y reglas precisas.

El recorrido comenzó introduciendo algunas herramientas matemáticas básicas que nos permiten razonar sobre los programas y sus propiedades. A partir de estas ideas fue posible presentar la sintaxis y la semántica de un lenguaje, dos componentes esenciales que determinan tanto la forma en la que se escriben los programas como el significado que tienen cuando se ejecutan.

Posteriormente se estudiaron distintos modelos que ayudan a comprender el comportamiento de los programas. El cálculo lambda permitió analizar la noción de función y evaluación desde un punto de vista abstracto, mientras que el lenguaje MinHS sirvió como un primer ejemplo concreto de cómo estas ideas pueden materializarse en un lenguaje funcional. De manera complementaria se discutió el problema de la inferencia de tipos, mostrando cómo es posible deducir propiedades importantes de los programas a partir de reglas formales.

Más adelante se introdujeron modelos operacionales para describir la ejecución de programas mediante máquinas abstractas, así como el lenguaje imperativo TinyC, que incorpora el manejo explícito de estado y memoria. Finalmente se exploraron conceptos propios del paradigma orientado a objetos, como la herencia y el subtipado, culminando con el estudio de *Featherweight Java*, un modelo que captura los elementos esenciales de este estilo de programación.

En conjunto, los temas presentados buscan ofrecer una visión más clara de los principios que dan forma a los lenguajes de programación modernos. Comprender estos fundamentos no solo permite analizar con mayor profundidad los lenguajes existentes, sino también contar con las herramientas necesarias para estudiar, diseñar e incluso proponer nuevos lenguajes de programación.

Bibliografía

[1] K. Ramírez *et al.*, *Nota de Clase del curso de Lenguajes de Programación,* Facultad de Ciencias, Universidad Nacional Autónoma de México, Ciudad de México, 2022.

[2] A. Brooks, *Modern Programming Languages: A Practical Introduction*, 2nd ed. Sherwood, Oregon: Franklin, Beedle & Associates, 2011.

[3] P. Deepika, *Selection Sort,Geeks for geeks*, 2021. [Online]. Available: https://www.geeksforgeeks.org/selection-sort/. [Accessed: Nov. 14, 2022].

[4] M. Lipovaca, *Learn You a Haskell for Great Good!: A Beginner's Guide*, digital publication. San Francisco, California, 2011.

[5] F. Miranda *et al.*, *Nota de Clase del curso de Lenguajes de Programación,* Facultad de Ciencias, Universidad Nacional Autónoma de México, Ciudad de México, 2021.

[6] G. Keller *et al.*, *Class Notes from the course Concepts of programming language design,* Department of Information and Computing Sciences, Utrecht University, The Netherlands, 2020. [Online]. Available: https://courses.cs.vt.edu/ cs3304/Spring02/lectures/.

[7] F. Nielson, *Semantics with Applications: An Appetizer,* Springer Publishing, Berlin, 2007. [Online]. Available: https://archive.org/details/Hanne_Riis_Nielson_Flemming_Nielson__Semantics

[8] R. Harper, *Practical Foundations for Programming Languages*, Working draft. San Francisco, California: Carnegie Mellon University Press, 2010. [Online]. Available: https://moss.cs.iit.edu/cs440/readings/harper.pdf.

[9] J. Mitchell, *Foundations for Programming Languages*, Massachusetts Institute of Technology Press, Cambridge, 1996.

[10] S. Krishnamurthi, *Programming Languages Application and Interpretation*, Brown University press, Rhode Island, 2007.

[11] A. Spector-Zabusky, *How would the Lambda Calculus add numbers? Stack Overflow*, 2021. [Online]. Available: https://stackoverflow.com/questions/29756732/how-would-the-lambda-calculus-add-numbers. [Accessed: Apr. 16, 2023].

[12] J. Enríquez, *Lenguajes de Programación Nota de clase,* Facultad de Ciencias, Universidad Nacional Autónoma de México, Ciudad de México, 2022.

[13] G. Keller *et al.*, *Concepts of Programming Languages: Data types in Explicitly Typed Languages,* Department of Information and Computing Sciences, Utrecht University, The Netherlands, 2022.

[14] V. Karavirta *et al.*, *Formal Languages Spring Chapter 1 Introduction Grammar Exercises,* online tool. [Online]. Available: https://opendsa-server.cs.vt.edu/OpenDSA/Books/PIFLAS21/html/IntroGrammarEx.html. [Accessed: Oct. 7, 2022].

[15] D. Jerrett, *Binary Search, a haskell approach,* 2022. [Online]. Available: https://programming-idioms.org/idiom/124/binary-search-for-a-value-in-sorted-array/2120/haskell. [Accessed: Nov. 29, 2022].

[16] P. Kirankumarambati, *Binary Search Data Structure and Algorithm Tutorials, Geeks for geeks*, 2023. [Online]. Available: https://www.geeksforgeeks.org/binary-search/. [Accessed: Nov. 29, 2022].

[17] A. Dahiya *et al.*, *CIS 194: Introduction to Haskell, Homework 1,* University of Pennsylvania, Philadelphia, Pennsylvania, 2013. [Online]. Available: https://www.seas.upenn.edu/ cis1940/spring13/hw/01-intro.pdf. [Accessed: Nov. 4, 2022].

[18] B. Yorgey, *Typeclassopedia, Wiki Haskell*, 2011. [Online]. Available: https://wiki.haskell.org/Typeclassopedia. [Accessed: Nov. 24, 2022].

[19] K. King, *Haskell List Problem Set, Github*, 2018. [Online]. Available: https://github.com/JD95/haskell-problem-sets/blob/master/Lists/Problems.hs. [Accessed: Dec. 10, 2022].

[20] A. Goguen, *Category Theory Applied to Computation and Control. Lecture Notes in Computer Science*, vol. 25. Springer, Berlin, 1975.

[21] W. Floyd, *Mathematical Aspects of Computer Science. Proceedings of Symposium on Applied Mathematics*, J.T. Schwartz, Ed., vol. 19. American Mathematical Society, 1967. [Online]. Available: https://www.cs.tau.ac.il/ nachumd/term/FloydMeaning.pdf.

[22] G. Winskel, *The formal semantics of programming languages: an introduction*, Massachusetts Institute of Technology Press, Cambridge, 1993. [Online]. Available: https://cin.ufpe.br/ if721/intranet/TheFormalSemanticsofProgrammingLanguages.pdf.

[23] A. Schmidt, *Denotational Semantics: A Methodology for Language Development*, William C. Brown Publishers, 1986.

[24] D. Plotkin, *A structural approach to operational semantics,* Tech. Rep. DAIMI FN-19, Computer Science Department, Aarhus University, Denmark, 1981.

[25] P. Deransart *et al.*, *Attribute Grammars: Definitions, Systems and Bibliography*, Lecture Notes in Computer Science 323. Springer-Verlag, Berlin, 1988.

[26] S. Krishnamurthi, *Programming Languages: Application and Interpretation*, 2nd ed. Providence, Rhode Island: Brown University Press, 2012.

[27] K. Slonneger *et al.*, *Formal Syntax and Semantics of Programming Languages*, United States: Addison-Wesley Publishing Co., 1995.

[28] *Semantics (computer science), Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Semantics_(computer_science). [Accessed: Oct. 18, 2023].

[29] *Syntax (programming languages), Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Syntax_(programming_languages). [Accessed: Oct. 18, 2023].

[30] P. Friedman *et al.*, *Essentials of Programming Languages*, 1st ed. The Massachusetts Institute of Technology Press, Cambridge, 1992.

[31] D. Smith, *Designing Maintainable Software*. United States: Springer Science & Business Media, 1999.

[32] V. Aho *et al.*, *Compilers: Principles, Techniques, and Tools*, 2nd ed. United States: Addison Wesley Publishing Co., 2017.

[33] C. Louden, *Compiler Construction: Principles and Practice*, Brooks-Cole Publishers, United States, 1997.

[34] M. Sipser, *Introduction to the Theory of Computation*, PWS Publishing Co., United States, 1997.

[35] *Pragmatics, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Pragmatics. [Accessed: Oct. 19, 2023].

[36] E. Coppock *et al.*, *Invitation to Formal Semantics (manuscript draft),* eClass NKUA digital platform, 2019.

[37] L. Mey, *Pragmatics: An Introduction*, 2nd ed. United Kingdom: Oxford-Blackwell Publishing, 2001.

[38] Y. Winter, *Flexibility principles in Boolean semantics*, Massachusetts Institute of Technology Press, Cambridge, 2001.

[39] M. Felleisen *et al.*, *How to Design Programs*, 1st ed. Massachusetts Institute of Technology Press, Cambridge, 2003.

[40] *Ambiguous grammar,Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Ambiguous_grammar. [Accessed: Oct. 19, 2023].

[41] W. Levelt, *An Introduction to the Theory of Formal Languages and Automata*, John Benjamins Publishing, United States, 2008.

[42] E. Scott, *SPPF-Style Parsing From Earley Recognizers,Electronic Notes in Theoretical Computer Science*, Elsevier B.V., United Kingdom, 2008.

[43] *Compiled Language, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Compiled_language. [Accessed: Oct. 19, 2023].

[44] *Interpreter (computing), Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Interpreter_(computing). [Accessed: Oct. 19, 2023].

[45] P. Terence *et al.*, *The Difference Between Compilers and Interpreters,Wayback Machine Archive*, United States, 2014.

[46] *Compilers vs. interpreters: explanation and differences, IONOS Digital Guide*, 2023. [Online]. Available: https://www.ionos.com/digitalguide/websites/web-development/compilers-vs-interpreters. [Accessed: Nov. 26, 2023].

[47] *Object-oriented programming, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Object-oriented_programming. [Accessed: Oct. 19, 2023].

[48] A. Martin *et al.*, *A Theory of Objects*. Springer Verlag, United States, 1998.

[49] J. Armstrong, *The Quarks of Object-Oriented Development,Communications of the ACM*, vol. 49, no. 2, pp. 123–128, 2006. [Online]. Available: https://www.researchgate.net/publication/220425366_The_quarks_of_object-oriented_development.

[50] *Programación por procedimientos, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://es.wikipedia.org/wiki/Programacion_por_procedimientos. [Accessed: Oct. 19, 2023].

[51] *Functional programming, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Functional_programming. [Accessed: Oct. 23, 2023].

[52] P. Hudak, *Conception, evolution, and application of functional programming languages,ACM Computing Surveys*, vol. 21, no. 3, pp. 359–411, 1989.

[53] A. Jain, *Javascript Promises: Is There a Better Approach?Medium*, 2023. [Online]. Available: https://medium.datadriveninvestor.com/javascript-promises-is-there-a-better-approach. [Accessed: Nov. 29, 2023].

[54] *Imperative programming, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Imperative_programming. [Accessed: Oct. 23, 2023].

[55] *Imperative programming: Overview of the oldest programming paradigm, IONOS Digital Guide*, 2021. [Online]. Available: https://www.ionos.com/digitalguide/websites/web-development/imperative-programming/. [Accessed: Apr. 21, 2022].

[56] B. Eckel, *Thinking in Java*. Pearson Education Publishers, United States, 2006.

[57] *Logic programming, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Logic_programming. [Accessed: Oct. 24, 2023].

[58] *Mathematical object, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Mathematical_object. [Accessed: Oct. 25, 2023].

[59] J. Azzouni, *Metaphysical Myths, Mathematical Practice*. Cambridge University Press, United States, 1994. :

[60] J. Burgess *et al.*, *A Subject with No Object*, Oxford University Press, United Kingdom, 1997.

[61] *Judgment (mathematical logic), Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Judgment_(mathematical_logic). [Accessed: Oct. 25, 2023].

[62] P. Martin-Löf, *On the meanings of the logical constants and the justifications of the logical laws,Nordic Journal of Philosophical Logic*, vol. 1, no. 1, pp. 11–60, 1996.

[63] *Rule of inference, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Rule_of_inference. [Accessed: Oct. 25, 2023].

[64] G. Boolos *et al.*, *Computability and logic*. Cambridge University Press, United States, 2007.

[65] C. R. John, *Theories of Programming Languages*, Cambridge University Press, United States, 2009.

[66] M. Bergmann, *An introduction to many-valued and fuzzy logic: semantics, algebras, and derivation systems*, Cambridge University Press, United States, 2008.

[67] F. Miranda *et al.*, *Matemáticas Discretas*, Facultad de Ciencias, Universidad Nacional Autónoma de México, Ciudad de México, 2016.

[68] A. Dossey *et al.*, *Discrete Mathematics*, 5th ed. Pearson-Addison-Wesley Publishing Co., United States, 2006.

[69] L. Gersting, *Mathematical Structures for Computer Science*, 3rd ed. Computer Science Press, W.H. Freeman and Company, United States, 1993.

[70] K. Grassman *et al.*, *Logic and Discrete Mathematics, A computer Science Perspective*, Prentice-Hall Inc., United States, 1996.

[71] D. Gries *et al.*, *A Logical Approach to Discrete Mathematics*, Springer-Verlag, United States, 1994.

[72] W. Grossman, *Discrete Mathematics, An introduction to concepts, methods and applications*, Macmillan Publishing Company, United States, 1990.

[73] T. Koshy, *Discrete Mathematics with Applications*, Elsevier Academic Press, 2004.

[74] H. Rossen, *Discrete Mathematics and its Applications*, 6th ed. McGraw Hill, 2006.

[75] S. Jessica, *Introduction to Haskell,* University of Pennsylvania, Philadelphia, Pennsylvania, 2013. [Online]. Available: https://www.seas.upenn.edu/ cis1940. [Accessed: Nov. 4, 2022].

[76] H. Krahn *et al.*, *Model Driven Engineering Languages and Systems,*Technische Universität Braunschweig, Braunschweig, Germany, 2007.

[77] N. Chomsky, *Aspects of the Theory of Syntax*, Massachusetts Institute of Technology Press, Cambridge, 2014.

[78] *Parse tree, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Parse_tree. [Accessed: Oct. 30, 2023].

[79] *Abstract syntax, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Abstract_syntax. [Accessed: Oct. 30, 2023].

[80] *Scope, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Scope_(computer_science). [Accessed: Oct. 31, 2023].

[81] *Let expression, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Let_expression. [Accessed: Oct. 31, 2023].

[82] *Lambda Calculus, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Lambda_calculus. [Accessed: Oct. 31, 2023].

[83] A. Turing, *Computability and λ-Definability,The Journal of Symbolic Logic*, vol. 2, no. 4, pp. 153–163, 1937.

[84] C. Thierry *et al.*, *Type Theory,*in *The Stanford Encyclopedia of Philosophy*, Department of Philosophy, Stanford University, United States, 2013.

[85] C. Mitchell, *Concepts in Programming Languages*, Cambridge University Press, Cambridge, 2003.

[86] C. Pierce, *Basic Category Theory for Computer Scientists*, The MIT Press, Cambridge, 1991.

[87] A. Church, *A set of postulates for the foundation of logic,Annals of Mathematics Archive*, Series 2, vol. 33, no. 2, pp. 346–366, Mathematics Department, Princeton University Press, Nueva Jersey, 1932.

[88] P. Selinger, *Lecture Notes on the Lambda Calculus*, vol. 0804. Department of Mathematics and Statistics, University of Ottawa Press, Canada, 2018.

[89] F. Turbak *et al.*, *Design concepts in programming languages*, The MIT press, Cambridge, 2008.

[90] W. Abrahams, *A final solution to the Dangling else of ALGOL 60 and related languages,Communications of the ACM*, vol. 9, no. 9, pp. 679–682, 1986.

[91] *Mathematical induction, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Mathematical_induction. [Accessed: Nov. 14, 2023].

[92] M. DeVos, *Mathematical Induction*, University Press, Canada, 2023.

[93] G. Diaz, *Mathematical Induction*, Wayback Machine Archive, Harvard University Press, United States, 2023.

[94] *Recursion, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Recursion. [Accessed: Nov. 14, 2023].

[95] L. Causey, *Logic, sets, and recursion*, 2nd ed. Jones and Bartlett Publishers, New England, 2006.

[96] user207421, *Static Semantics meaning?Stack Overflow*, 2016. [Online]. Available: https://stackoverflow.com/questions/40430578/static-semantics-meaning. [Accessed: Nov. 14, 2023].

[97] F. Remer, *Compiler Construction course,* University of California Press, United States, 1979.

[98] *Dynamic syntax, Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Dynamic_syntax. [Accessed: Nov. 28, 2023].

[99] R. Cann *et al.*, *The dynamics of language: an introduction*, Elsevier, Amsterdam, 2005.

[100] *Operational semantics,Wikipedia, La enciclopedia libre*, 2023. [Online]. Available: https://en.wikipedia.org/wiki/Operational_semantics. [Accessed: Nov. 28, 2023].

[101] K. Gilles, *Natural Semantics,* in *Proceedings of the 4th Annual Symposium on Theoretical Aspects of Computer Science*, Springer-Verlag, United Kingdom, 1987.

[102] A. Myers, *IMP: Big-Step and Small-Step Semantics,*Cornell University Online Repository, Ithaca, NY, Estados Unidos, 2007. [Online]. Available: https://www.cs.cornell.edu/courses/cs6110/2009sp/lectures/lec05-fa07.pdf. [Accessed: Apr. 30, 2024].

[103] M. Beckam, *Big Step Semantics,*University of Illinois Online Repository, Champaign, IL, Estados Unidos, 2020. [Online]. Available: https://courses.engr.illinois.edu/cs421/sp2020/slides/04.1.2-big-step-semantics.pdf. [Accessed: Apr. 30, 2024].

[104] B. Pierce *et al.*, *Programming Language Foundations*, The MIT Press, Cambridge, 2021. [Online]. Available: https://softwarefoundations.cis.upenn.edu/plf-current/Smallstep.html. [Accessed: Apr. 30, 2024].

[105] J. Aldrich, *Lecture Notes: Small-Step Operational Semantics,* Carnegie Mellon University Online Repository, Pittsburgh, PA, Estados Unidos, 2022. [Online]. Available: https://www.cs.cmu.edu/ aldrich/courses/17-363/notes/lecture06-small-step.pdf. [Accessed: Apr. 30, 2024].

[106] S. Chong, *Large-step semantics, continued,* Harvard Online Repository, Cambridge, Massachusetts, 2016. [Online]. Available: https://groups.seas.harvard.edu/courses/cs152/2016sp/lectures/lec04-largestep.pdf. [Accessed: Apr. 30, 2024].

[107] G. Keller *et al.*, *Concepts of Programming Language Design,* Utrecht University Online Repository, Utrecht, Países Bajos, 2022. [Online]. Available: https://github.com/jaeem006/Lenguajes/blob/main/Gabrielle [Accessed: May 14, 2024].

[108] J. Korbmacher *et al.*, *The Lambda Calculus,Stanford Encyclopedia of Philosophy*, California, Estados Unidos, 2023. [Online]. Available: https://plato.stanford.edu/entries/lambda-calculus/. [Accessed: May 21, 2024].

[109] R. Rojas, *A Tutorial Introduction to the Lambda Calculus,* Dallas University Online Repository, Texas, Estados Unidos, 2021. [Online]. Available: https://personal.utdallas.edu/g̃upta/courses/apl/lambda.pdf. [Accessed: May 21, 2024].

[110] B. Bonacci, *Lambda Calculus Boolean logic, Bruno Bonacci Blog*, 2007. [Online]. Available: https://blog.brunobonacci.com/2017/10/08/lambda-calculus-and-boolean-logic/. [Accessed: May 21, 2024].

[111] R. Cartwright, *Comp 311 - Review 2,* Rice University Online Repository, Texas, Estados Unidos, 2010. [Online]. Available: https://www.cs.rice.edu/ javaplt/311/Readings/supplemental.pdf. [Accessed: May 21, 2024].

[112] D. Chiang, *Data structures in the lambda calculus,* Notre Dame Online Repository, Indiana, Estados Unidos, 2010. [Online]. Available: https://www3.nd.edu/ dchiang/-teaching/pl/2019/church.html. [Accessed: May 21, 2024].

[113] A. Sampson, *Recursion and Fixed-Point Combinators,* Cornell University Online Repository, Nueva York, Estados Unidos, 2017. [Online]. Available: https://www.cs.cornell.edu/courses/cs6110/2017sp/lectures/lec05.pdf. [Accessed: May 21, 2024].

[114] G. Richards, *Proof of the Church-Rosser Theorem,* Waterloo University Online Repository, Ontario, Canadá, 2022. [Online]. Available: https://student.cs.uwaterloo.ca/ cs442/W22/extras/c-r-thm-proof.pdf. [Accessed: Jun. 4, 2024].

[115] J. Pohjola, *COMP3161/9164 23T3 Assignment 1,* UNSW Online Repository, Sydney, Australia, 2023. [Online]. Available: https://www.cse.unsw.edu.au/ cs3161/23T3/Assignment %201/Spec.pdf. [Accessed: Jun. 12, 2024].

[116] J. Pohjola, *Functional Programming Languages: MinHs,* UNSW Online Repository, Sydney, Australia, 2023. [Online]. Available: https://www.cse.unsw.edu.au/ cs3161/23T3/Week %2004/Tuesday/Slides.pdf. [Accessed: Jun. 12, 2024].

[117] F. Pfenning, *Foundations of Programming Languages: Lectures Notes on Progress,* Carnegie-Mellon University Online Repository, Pittsburgh, Pensilvania, Estados Unidos, 2004. [Online]. Available: https://www.cs.cmu.edu/ fp/courses/15312-f04/handouts/07-progress.pdf. [Accessed: Jun. 18, 2024].

[118] F. Pfenning, *Foundations of Programming Languages: Lectures Notes on Type Safety,* Carnegie-Mellon University Online Repository, Pittsburgh, Pensilvania, Estados Unidos, 2004. [Online]. Available: https://www.cs.cmu.edu/ fp/courses/15312-f04/handouts/06-safety.pdf. [Accessed: Jun. 18, 2024].

[119] M. Biernacka *et al.*, *Non-Deterministic Abstract Machines,* in *CONCUR 2022 - 33rd International Conference on Concurrency Theory*, Varsovia, Polonia, 2022. [Online]. Available: https://inria.hal.science/hal-03772712/document. [Accessed: Jun. 18, 2024].

[120] A. Erwan, *Self-application in Church's untyped lambda calculus, Stack Overflow*, 2012. [Online]. Available: https://math.stackexchange.com/questions/1316377/self-application-in-churchs-untyped-lambda-calculus. [Accessed: Jun. 18, 2024].

[121] J. Enriquez *et al.*, *Notas para Lenguajes de Programación 2023-1: Boletín de ejercicios 4,* Facultad de Ciencias, Universidad Nacional Autónoma de México, Ciudad de México, 2023.

[122] D. Watt, *Programming Language Design Concepts*, John Wiley & Sons, Ltd., University of Glasgow, Scotland, 2004.

[123] D. Kozen, *CS3110 Notes on Data Structures and Functional Programming: lecture 26: Type Inference and Unification,* Cornell University Online Repository, Cornell University, New York, United States, 2011.

[124] R. Ribeiro *et al.*, *A Mechanized Textbook Proof of a Type Unification Algorithm,* Universidade Federal de Ouro Preto Online Repository, Universidade Federal de Ouro Preto, Minas Gerais, Brazil, 2015.

[125] M. Amaro *et al.*, *A Mechanized Textbook Proof of a Type Unification Algorithm, Revista de Informática Teórica e Aplicada - RITA*, vol. 27, no. 3, pp. 13-24, 2020.

[126] M. Feeley, *Compiler for the Tiny-C language,* Université de Montreal Online Repository, 2002. [Online]. Available: https://www.iro.umontreal.ca/f̃elipe/IFT2030-Automne2002/Complements/tinyc.c. [Accessed: Oct. 29, 2024].

[127] M. Slávik, *TinyC Optimizing Compiler,* Faculty of Information Technology CTU, Prague, 2023.

[128] A. Igarashi *et al.*, *Featherweight Java: A Minimal Core Calculus for Java and GJ,* UPenn Online Repository, Pennsylvania, Estados Unidos, 2021. [Online]. Available: https://www.cis.upenn.edu/ bcpierce/papers/fj-toplas.pdf. [Accessed: Aug. 23, 2025].

[129] S. Weirich *et al.*, *A Design for Type-Directed Programming in Java,* University of Pennsylvania, Estados Unidos, 2002.

[130] *Subtyping, Wikipedia, La enciclopedia libre*, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Subtyping. [Accessed: Nov. 15, 2024].

[131] A. Meyers, *Introduction to Compilers: Subtype Polymorphism,* Cornell University Online Repository, Nueva York, Estados Unidos, 2022.

[132] A. Silva, *Advanced Programming Languages: Subtyping,* Cornell University Online Repository, Nueva York, Estados Unidos, 2023.

[133] *Object-oriented programming, Wikipedia, La enciclopedia libre*, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Object-oriented_programming. [Accessed: Nov. 15, 2024].

[134] User76258, *What is a non-ambiguous CFG for generating the set of natural numbers? Computer Science Stack Exchange.* [Online]. Available: https://cs.stackexchange.com/questions/97794/what-is-a-non-ambiguous-cfg-for-generating-the-set-of-natural-numbers. [Accessed: Nov. 26, 2024].

[135] *Lecture 18: Context-Free Grammars (CS103: Mathematical Foundations of Computing),* Stanford University, United States, 2016. [Online]. Available: https://web.stanford.edu/class/archive/cs/cs103/cs103.1164/lectures/18/Slides18.pdf.

[136] J. Almeida *et al.*, *Context-free grammars: Exercise generation and probabilistic assessment,* in *Proceedings of OASIcs-SLATE 2016*, 2016.

[137] L. Snyder, *Practice Problems 07 (CS 320),* Boston University. [Online]. Available: https://www.cs.bu.edu/fac/snyder/cs320/Review

[138] D. Baker, *Homework 11 Context-Free Grammars (CS 341),* Texas University. [Online]. Available: https://www.cs.utexas.edu/ cline/ear/automata/CS341-Fall-2004-Packet/2-Homework/Home11CFGs.pdf.

[139] Guy Coder, *Looking for a Church-encoding (lambda calculus) to define <, >, !=, Stack Overflow*, 2012. [Online]. Available: https://stackoverflow.com/questions/20523625/looking-for-a-church-encoding-lambda-calculus-to-define. [Accessed: May 22, 2025].

[140] user3310334, *How would the Lambda Calculus add numbers? Stack Overflow*, 2015. [Online]. Available: https://stackoverflow.com/questions/29756732/how-would-the-lambda-calculus-add-numbers. [Accessed: May 29, 2025].

[141] B. Gruenbaum, *Lambda calculus predecessor function reduction steps, Stack Overflow*, 2021. [Online]. Available: https://stackoverflow.com/questions/8790249/lambda-calculus-predecessor-function-reduction-steps. [Accessed: Jun. 3, 2025].